Dungeon Roguelike Engine

0 Contents

0 Contents 1

1 Analysis 3

1.1 Statement of Problem 3

1.2 Background 3

1.3 End User 3

1.4 Initial Research 4

1.4.1 Existing, similar programs 4

1.4.2 Potential abstract data types / algorithms 6

1.4.3 First interview 7

1.4.4 Key components 9

1.5 Further Research 9

1.5.1 Prototype 9

1.5.2 Second interview 17

1.6 Objectives 18

1.7 Modelling 20

2 Design 21 22

3 Testing 58

4 Evaluation 66

5 Technical Solution 70

Directory of technical skills 70

Code 71

1 Analysis

1.1 Statement of Problem

Since the creation of the game ‘Rogue’, many other games have replicated the same key-features and have been labelled ‘Rogue-likes’ or ‘Rogue-lites’. However, these games that have followed suit aren’t particularly affordable, and those who may have wanted to explore the roguelike genre could avoid the investment in fear that they might not enjoy the games and find it a waste of money. On top of this, Rogue and other roguelikes can be complicated to newcomers, which could intimidate them from giving the games a go. In conclusion, there is a lack of free and simple roguelike games to provide a comfortable entry into the genre.

1.2 Background

The key features of a roguelike game are:

- The player can collect items that give an offensive or defensive advantage
- The game utilises permadeath to make decisions and moves by the user more meaningful, as dying would mean the player character is dead and cannot be brought back - the user must start with a fresh player character again
- No run of the game is the same. This means that the layout of dungeon floors, monster encounters and any items that may appear are procedurally generated so that each run has an element of randomness
- As the player progresses through the game, enemies become more difficult to defeat.

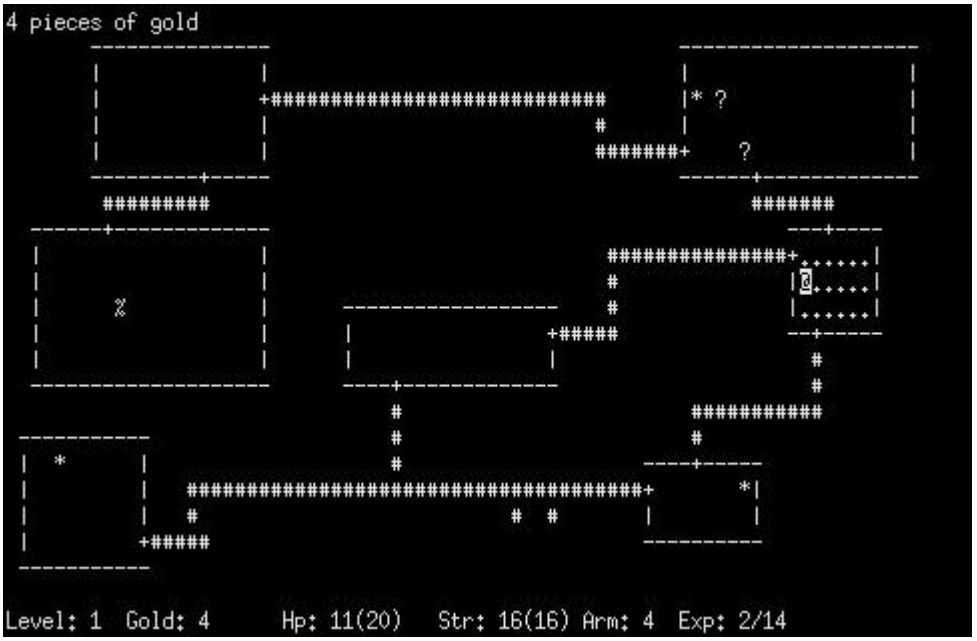
1.3 End User

This program is being developed for people who haven’t yet played roguelike games and want to get a feel for the genre without having to reach into their wallet. The program will be for teens or those that are older, since the game might be too complicated for younger children despite being a simpler version of the original Rogue. The end user I have selected is my older brother, who is a seasoned veteran in regards to gaming, but hasn’t yet given roguelikes a go. Since he’s a family member, he will be readily available and provide honest feedback. From his experience with gaming, I can make good use of my brother’s in-depth insights, and the fact that he hasn’t yet played roguelikes will make him part of my target audience.

1.4 Initial Research

1.4.1 Existing, similar programs

1.4.1.1 Rogue



Rogue is a turn-based dungeon crawler that was originally developed around 1980. The game is free to play on a web browser. The player can move around on a square grid represented by ASCII. Each level starts out unmapped, so the player can only see rooms and corridors after moving into them. The game is turn-based, such that players can devise the best strategy to survive and defeat their enemies.

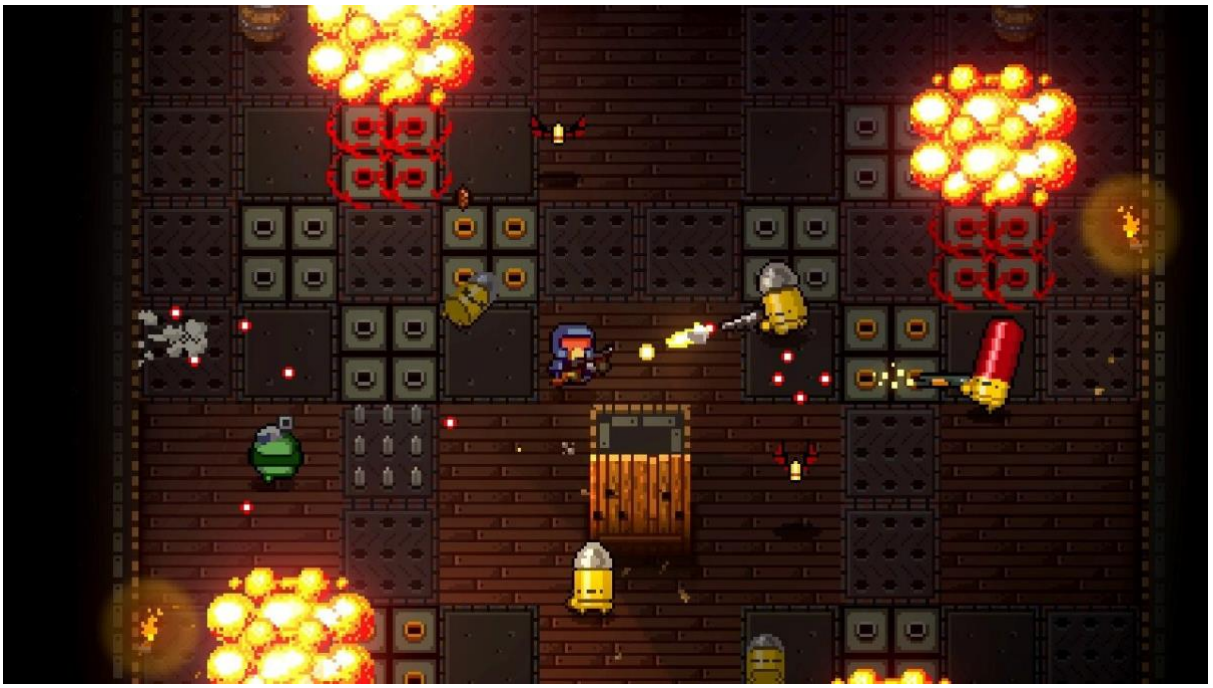
Pros:

- The turn-based aspect provides players with more time to make decisions, which can be very helpful to beginners
- The use of ASCII makes the game display look simple which makes the game easier to understand
- The game is free on the web, so is accessible to anyone who has access to anyone with an internet connection and a computer
- Starting with the dungeon levels unmapped makes the game more challenging, providing a greater skill ceiling which gives players more room to improve Cons:
- There are so many different controls to memorise, which makes it very difficult to play for beginners, and would take a long time to master
- The web version is hard to navigate, and the easier option is on steam which is £2.50
- Starting with the dungeon levels unmapped can make the game too challenging for beginners, and it can be annoying taking a long time navigating corridors to try and find a room
- The simple graphics can make the game lack depth and less interesting

Elements to apply to my solution:

Using ASCII for the game display will make the game simpler and less intimidating for the user, which is great for beginners to the roguelike genre. Furthermore, this simple display will allow the game to render quickly for the user. Another feature I would like to implement in my own solution is that the game is turn-based. This allows the player to be a lot more thoughtful with their movements in complex situations in more difficult levels, and will give beginners more leeway in early levels by providing more time to make decisions. Seeing the feature where the level begins unmapped makes me realize I will allow my user to view their surroundings regardless of if they've discovered certain rooms, as I want my solution to be beginner-friendly.

1.4.1.2 Enter the Gungeon



Enter the Gungeon is a bullet hell roguelike which was developed in 2016. The player can choose from many different characters to play as, which each have different abilities, starter weapons and looks. The player must traverse through 5 floors to reach the bottom floor of the Gungeon in order to get a gun that can 'kill the past'. Each of the 5 floors have a unique set of enemies and rooms. Some rooms are universal to every floor: a starting room, chest rooms, a shop room and a boss room. In order to progress to the next floor, the player must defeat the boss on their current floor. The game contains a wide variety of active items, passive items and weapons, which make each run unique. These items can be dropped by bosses or from chests. Chests are opened by keys, and keys have a chance of being dropped by enemies. Rooms contain traps which can damage the player and obstacles which the player can use to take cover from bullets.

<u>Pros:</u>

- The wide variety of items makes each run unique, which makes the game incredibly replayable
- The graphics of the game are satisfying, but not over the top, which makes the game engaging

- The HUD is very self-explanatory and intuitive, which makes it easy for beginners to start
- The use of different player-characters to choose from provides different playstyles, improving replayability
- The use of 5 distinct floors with a certain set of rooms and enemies in each provides a sense of structure to a procedurally-generated game, giving a sense of predictability to players <u>Cons:</u>
- The game is £13 on steam, which is quite expensive
- The wide variety of enemies, rooms, items and player-characters can make the game intimidating for beginners
- No options to make the game easier, so beginners might become frustrated and quit
- The game isn't turn-based, which gives players less time to make decisions and might be too challenging for people who have slower reaction times

<u>Elements to apply to my solution:</u>
To start, I want to copy how certain rooms are universal to each dungeon floor: the starting room, chest room and boss room. Adding an element of predictability to the dungeon will make it more forgiving for beginners, since they know that there is a structure that the dungeon must follow. I like how there is a boss room in every floor which must be passed in order to reach the next floor - it ensures that each floor must have a challenge and you can't just skip through floors. Using rooms that have obstacles or traps will add more depth to my game by providing different sorts of encounters with enemies. Adding different abilities and items will provide the players with different sorts of playstyles, which will give players different approaches to certain encounters. Using these features will allow players to make a variety of different choices, improving the replayability of my game. Playing through the game and experiencing how dynamic and fast-paced it was has emphasised to me how important it is to make my program turn-based. Turn-based will allow the player to play at the speed they decide - fast if they're experienced and slow if they're less experienced. Without the game being turn-based, you can't make that decision, which will cause some players to lose interest.

1.4.2 Potential abstract data types / algorithms

To represent my game, where the dungeon and player reside, I will use 2D arrays. This allows entities to move around in a square grid, and it will be easy for the user to interpret.

To generate my dungeon, I looked into two different algorithms. First, I looked into binary space partitioning, where you start with a large rectangular space, and then recursively split the space into smaller rectangles, and a room is placed in each rectangular space. This provides a dungeon that is much more structured and much less chaotic. Though I've previously emphasised the importance of simplicity and predictability, I felt as if this algorithm was too far toward the simple end of the complexity spectrum. The dungeons it generated felt too predictable, which could make the user feel bored. Secondly, I looked into the 'random room placement' algorithm. Rooms are randomly generated, and then randomly

placed into a grid, ensuring that they don't overlap with other rooms. The randomness this algorithm provides ensures that there isn't too much regularity to dungeon floors, and the player is always on their toes.

To allow the enemies to track the player by finding the shortest path, I looked into breadth-first search, depth-first search and Dijkstra's. I decided not to choose Dijkstra's, due to the fact that it finds the shortest path between a node and every other node, which will take an astronomical amount of time if the dungeon is large. Between the depth-first and breadth-first searches, I decided to use breadth-first since depth-first doesn't guarantee that the path it finds is the shortest one - it might go around the destination and find a much longer route, which isn't optimal. However, if I decided to add tiles which I wanted some enemies to avoid while traversing, then I would use Dijkstra's, since then the graph would become weighted. To avoid the algorithm taking too long, I will add a limit to how long the algorithm can search for before it settles for any paths it currently has or terminates in case there are no paths yet found.

In order to convert my 2d array into a graph so that I can use BFS or dijkstra's, I will create an algorithm that goes through the entire 2d array, ignoring any tiles which have collision (i.e walls), and adding the coordinates of each cell to a dictionary, where the coordinates of the cell are the key, and then any neighbouring cells which don't have collision are stored in a list which is the value.

1.4.3 First interview

1. **What features would you like in a roguelike?**

I think there should be something new every time, maybe different layouts of levels or different items that could be picked up.
Also, probably some sort of progression. New stages, permanent upgrades, that sort of thing, so there's something to try to aim for.
A story behind why things repeat is a nice plus too.

2. **What games do you enjoy the most and why?**

My favourites right now are Elden Ring, Destiny 2 and Cyberpunk 2077. I'd say it's because they each have such incredible art and a deep story behind them. Elden ring is really unique in its setting design, and you can discover a lot about the story just through the surroundings. Cyberpunk 2077 has amazing world building - you can overhear conversations between strangers and find lore about the Cyberpunk 2077 world through 'data shards' that are scattered around, which makes you feel like you're actually inside the game. The individuality of games makes them engaging: in Cyberpunk 2077 you can choose to be good at strength or be a good hacker, be stealthy or be loud, and can choose different play styles with different types of guns. In Destiny 2 you can be a titan, hunter or warlock, and then there's many different subclasses within these classes. You get to pick an avenue and make it yours, you can really be yourself in the game. Individuality also increases replayability, in one save I could be an assassin using dark magic, but then in my next save I could

choose to be something different like a super heavy guy who uses two giant hammers.

3. **What do you dislike about some of the games you have played?** Having a meta in games, because you could be given a wide array of classes to choose from, which seems fun initially, but then you soon find yourself being directed towards a certain few classes which are more powerful. If you don't choose these classes, then the game might be unnecessarily difficult, or in competitive games you won't stand a chance against others.

4. **Do you think that including randomly generated aspects in a game improves replayability?**

Yes, definitely. I find going through the same areas and enemies and repeating the same thing over and over is boring. You can't really experience many games the same way after you play them for the first time. Adding randomly generated aspects makes you unable to expect what will be in the next run - what new abilities and items you'll get or what enemies you'll face. There's always something new that way.

5. **To what extent do graphics matter in a game that is designed to teach you the skills of the roguelike genre**

Having overly nice graphics might distract me, I might focus on the setting or environment rather than what's actually being taught.
However, having really bad graphics will make the game boring, shallow and not incredibly engaging, so meeting somewhere in the middle is for the best.

6. **In a roguelike game, should there be a compulsory boss at the end of every floor in order to progress?**

I think this would be a good idea, since bosses tend to be more challenging than the regular enemies you'd find throughout the dungeon floor. Adding a boss at the end of every floor will add a bigger obstacle for the player to overcome, which encourages them to think smarter and try different things.

7. **Is it important for there to be different difficulty modes?**

Yes it is, different players want different things from a game - some want to be challenged, and some want to be able to relax. If a player who wants to be challenged finds the game easy, then they might become bored. If a player who wants to relax finds the game challenging, then they might become frustrated and rage quit.

8. **Does having classes that each have unique characteristics and abilities in a game make it more fun?**

It’s good to have different classes, but they can also be restricting at times. If you’re only able to have a certain collection of abilities and weapons that you can use together for certain classes, then you can get tired of each class pretty quickly. It would be nice to mix abilities and weapons as you wished, or maybe randomly doing so would encourage the player to try different playstyles they wouldn’t otherwise.

Conclusions

From my interview I was able to assess what the key features are of a good roguelike game which is friendly to beginners. These features will ease the user into the roguelike genre, which is the purpose of my program. The answer to question 8 has given me the idea of providing the player with a random set of abilities at the start of each run, which will ensure they don’t become comfortable with a certain set of abilities and keep them engaged in the game. Question 7 emphasised that options for difficulty is a must if I want to make the game entertaining for player’s with a wide range of experience. If I make the game just easy, then experienced players won't be interested, and if I make the game just hard, then new players don't want to play. Giving the opportunity to the player to decide solves this problem.

The answers to questions 1 and 4 reinforced what I was thinking before; I know that I will definitely be adding randomly generated dungeon floors and items that the player can access to improve replayability.

1.4.4 Key components

Reflecting on my first interview and research of existing programs, the key components of my game I’ve decided are:

1. Each dungeon floor is procedurally generated to ensure randomness
2. Provide options for different difficulty modes for the user
3. Give the player a random combination of abilities at the start of each run
4. Increase difficulty of the game through making enemies more challenging to give a sense of progression
5. In order to move to the next dungeon floor, a boss must be defeated
6. There will be a variety of different weapons and items that the player can use to provide a uniqueness to each run
7. The game won’t have too many controls so that the user doesn’t get overwhelmed

1.5 Further Research

1.5.1 Prototype

Prototype overview

The features I have decided to implement are:

1. Random dungeon generator
2. Player movement
3. Some enemy generation and movement

4. Player and enemy attacking
5. Player camera

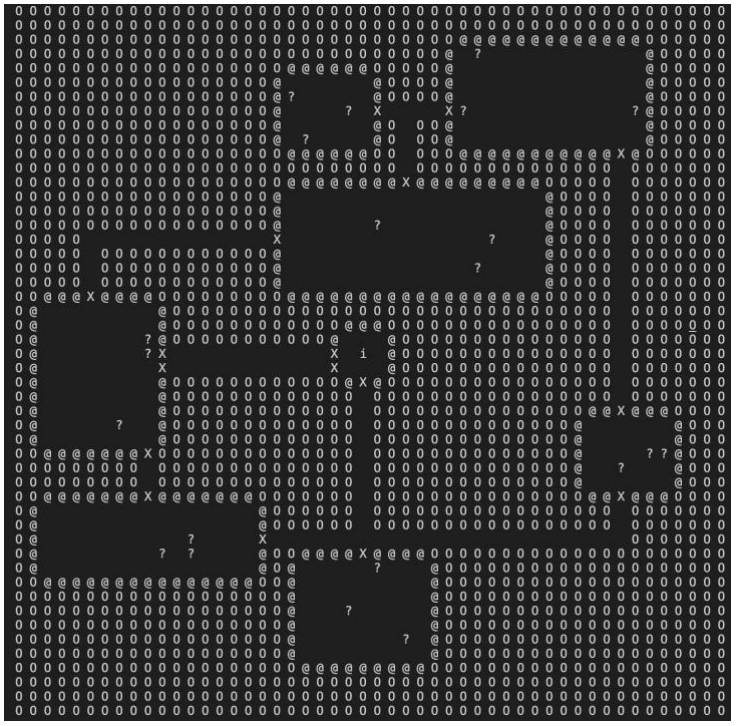
I have decided to implement these features first because they are the foundations of my program: my game can't be played without a few of these features. I've tried to make my prototype scalable, so implementing these features makes it easier for me to implement more specific features later on, as I can build on top of what I've made in my prototype.

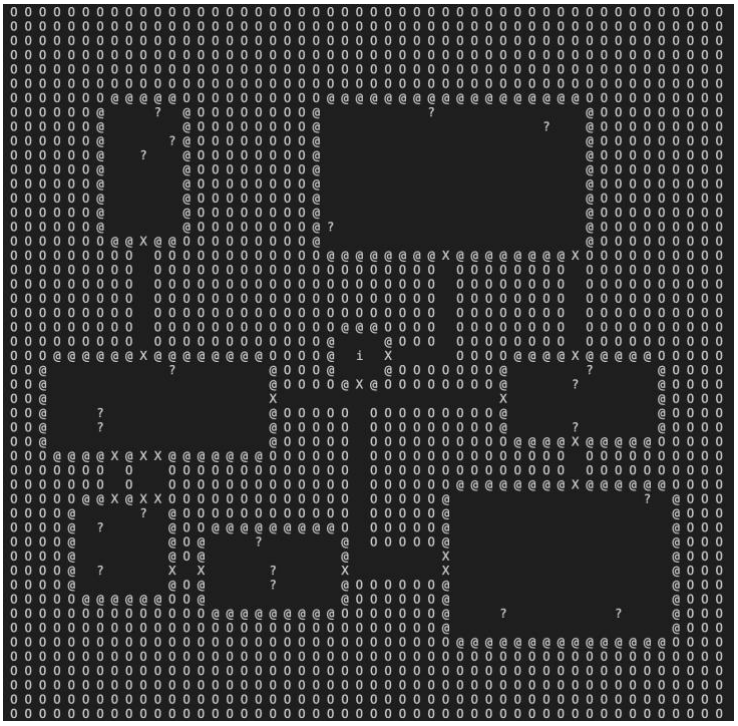
Making a random dungeon generator was essential to my prototype since this provides a forever-changing environment for the player to explore, and the dungeon is where all of my other features are built around: enemies, items, player exploration, etc.
It's important to add player movement early on, so that my end-user can interact with my prototype, providing a more in-depth experience and resulting in better feedback. It also means that I can get a feel for the player experience and gain insights into what other features I must add or existing features I must improve on.
Simple enemy generation, movement and attacking is a must, the enemy is what the player will interact with throughout the game and provide a challenge. I needed to get this basic part done so it's easier in the future when I decide to make enemies more complex. I also decided to implement a player camera, as it's not great if the player starts the game and can see the entire dungeon, so it makes the player feel as if they are actually exploring.
Finally, generating the dungeon (with many different tiles) and enemies (with different movement patterns, hp, dmg, etc) relies heavily on OOP, so I coded the core classes there, using inheritance and abstract method types and variables.

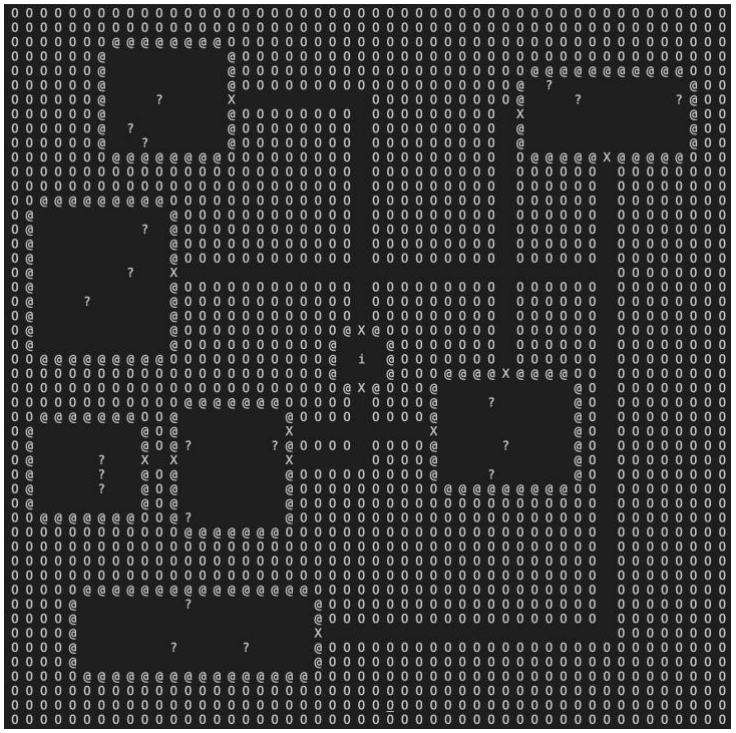
How the prototype works

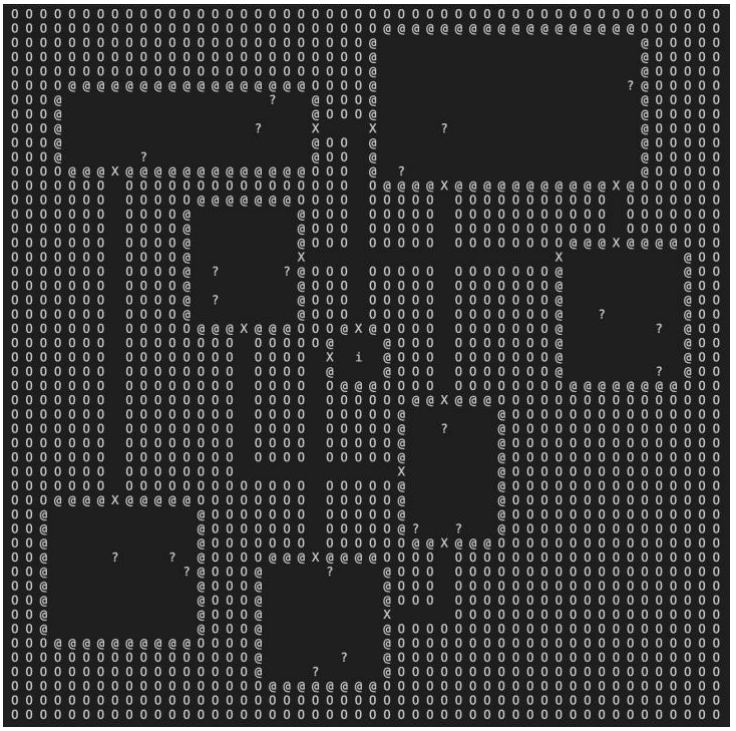
1. Random dungeon generation

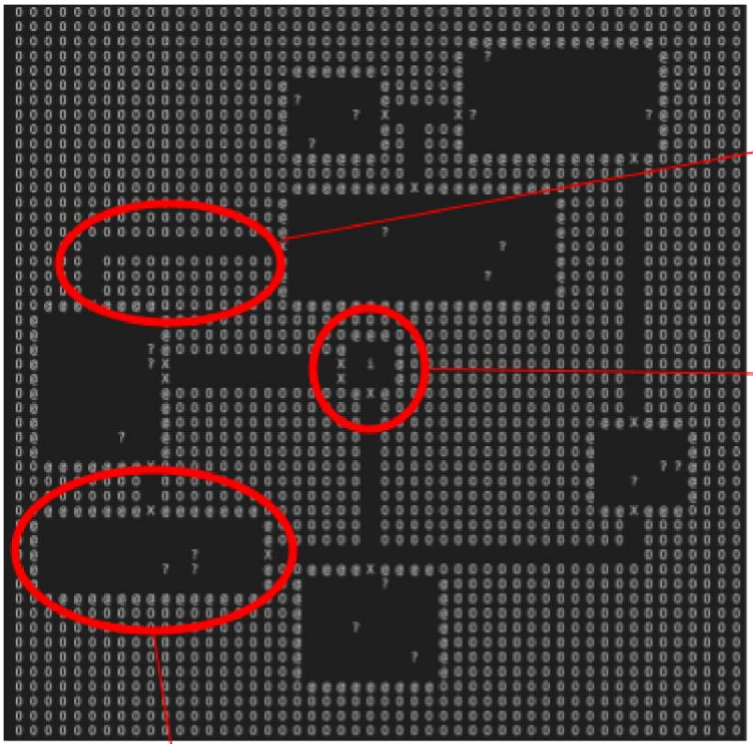
Screenshots:










3. Corridor
1. Starting room
2. Random room

Explanation:

1.0.1. What do the characters represent?

‘ ‘ (empty space) - floor tile
‘@’ - room wall tile
‘O’ - wall tile
‘X’ - door tile
‘i’ - player (more detail later)
‘?’ - enemy (more detail later)

1.0.2. How is the dungeon represented?

To represent the dungeon, I used a 2D array of my ‘Tile’ class. All of the aforementioned tiles inherit from this class. This 2D array is initially ‘empty’, full of null values.

1.1. Starting room

When generating the dungeon, this room is not randomly generated nor randomly placed. It is of a predetermined position and size, and it is where the player starts. For the prototype, I have made the starting room in the center of my dungeon. This is the first room added to the dungeon.

1.2. Random room

These rooms have a random size (width and height) and random position in the dungeon. My algorithm will generate a room with a random width, height and position (for x and y respectively, from the index 0 to the width/height of the dungeon minus by the width/height of the room generated). It will then check the dungeon to see if there are any existing tiles in

the area that the room generated will be in. If there are already tiles in that region, then another room will be generated until one fits. If there isn't, then the room is added to the 2D tile array. The random position generated represents the top left position in the room. From there, the algorithm iterates through each position of the dungeon that the room will be in. If the position is any of the corners of the room, then it makes the position in the array remain null (due to issues with carving corridors and door placement). If the position is the top row, bottom row, leftmost column or rightmost column of the room, then that position in the array becomes a wall tile. Otherwise, the position in the array becomes a floor tile. This whole process repeats until the predetermined number of rooms is reached.

1.3. Corridors

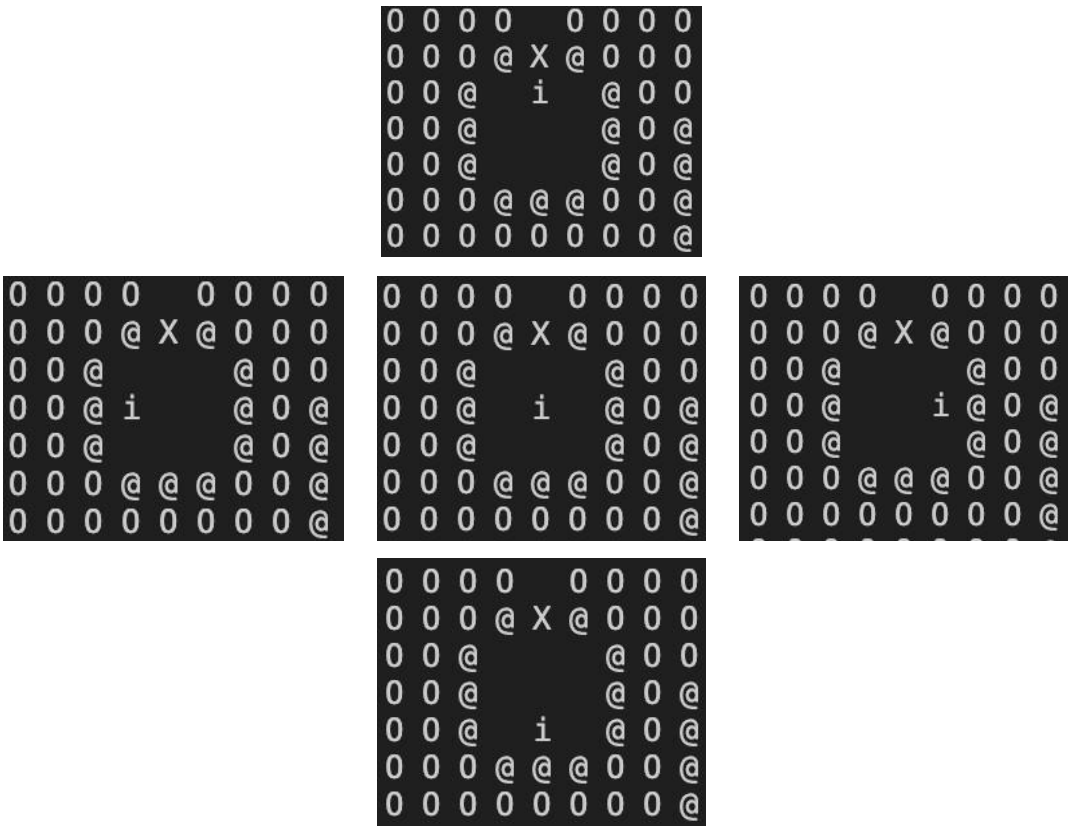
Corridors are generated by finding the center of two rooms, and then either carving a horizontal, then vertical path between the two, or a vertical, then horizontal path, which creates 'L' shaped corridors. When carving a horizontal path, the algorithm iterates through the x positions between the two centers, and for the vertical path through the y positions between the centers. First, the algorithm finds if the path is valid. It does this by checking how many times it consecutively intersects a wall tile. If it happens more than once, then the path isn't valid. This makes the path invalid since I generate door tiles by replacing wall tiles when carving corridors, so if the corridor path happens to intersect an entire side of a room, then that entire side of the room will become a door, which isn't great. If the path is valid, then the corridor is carved in the 2D array, adding floor tiles at each position where there isn't an existing tile, replacing any wall tiles with door tiles and then just leaving existing floor tiles. If the path is invalid, then a different second room is used until the path is valid.

2. Player movement

2.0. Entities

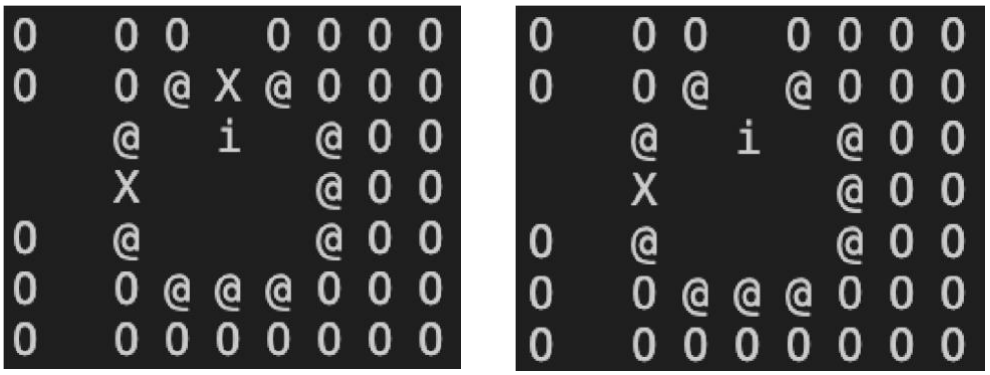
For entities (the player, enemies, etc), I created a 2D array of my 'Entity' class which has the same size as the dungeon 2D array. All entities are stored in this array, representing their position in the dungeon. All entities (player class, enemy classes, etc) inherit from this entity class.

2.1. Player directions Screenshots:



Explanation:
Shown above are the directions that the player can move in, which are up, down, left and right.

2.2 Tiles and collision Screenshots:
Certain tiles have collision, and certain don't. Floor tiles have no collision so the player can always move on these, walls have collision so the player can never move on these, and doors have collision initially, but after the player attempts to move onto it (thus interacting with it), it changes to have no collision.



On the left is a door before being interacted with, on the right is a door after being interacted with. Although the display changes from an 'X' to a ' ' (empty space), the tile does not change from a door to a floor. The tile remains as a door but the collision changes to false so

when the dungeon is displayed, if the collision is false, then it is displayed as a ‘ ‘, if the collision is true, then it is displayed as a ‘X’.

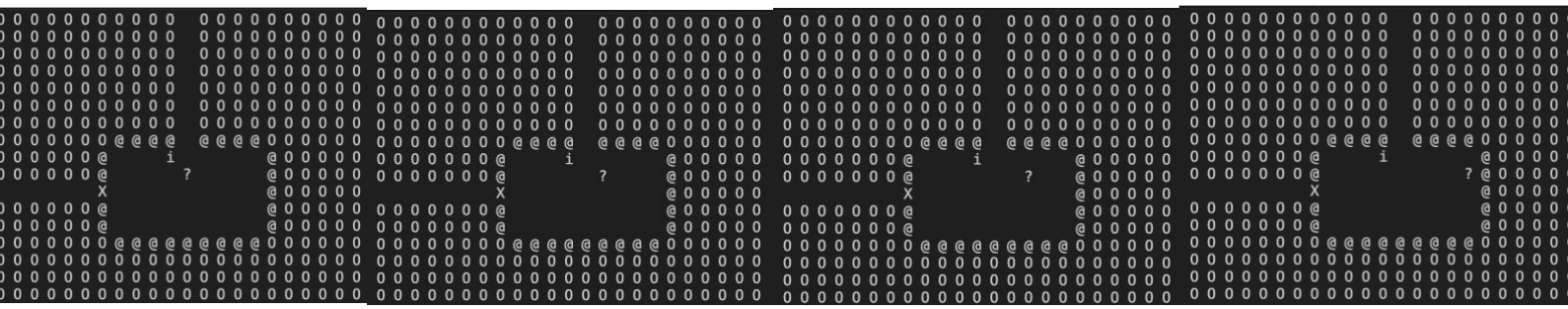
3. Enemies

3.0 Overview

I have created 3 simple enemies: VerticalEnemy, HorizontalEnemy and RandomEnemy. Their names represent how they move, so VerticalEnemy just moves vertically up and down, HorizontalEnemy moves left and right, and RandomEnemy moves in a random direction (up, down, left or right). Each enemy moves every other time the player moves, which makes it easier for the player to predict where they will move next and then decide where to move accordingly, however to increase difficulty later on I can make enemies which will move every time the player moves. For now, each of these enemies do 1 damage and have 1 health, but this can be varied easily (due to using classes).

3.1 Horizontal Enemy

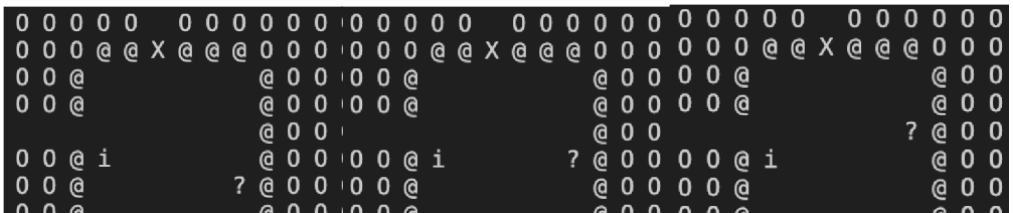
Screenshots:



Explanation:

When the enemy is generated, it is randomly selected if it starts by moving right or left. When it reaches a tile with collision it stops, then changes direction. Shown from left to right is the enemy moving to the right, and in the last screenshot it reaches a wall. From there, it travels to the left, which is shown if you look at the screenshots right to left instead.

3.2 Vertical Enemy Screenshots:



Explanation:
When the enemy is generated, it is randomly selected if it starts by moving up or down. When it reaches a tile with collision it stops, then changes direction. Shown above is the enemy moving up.

3.3 Random Enemy Screenshots:

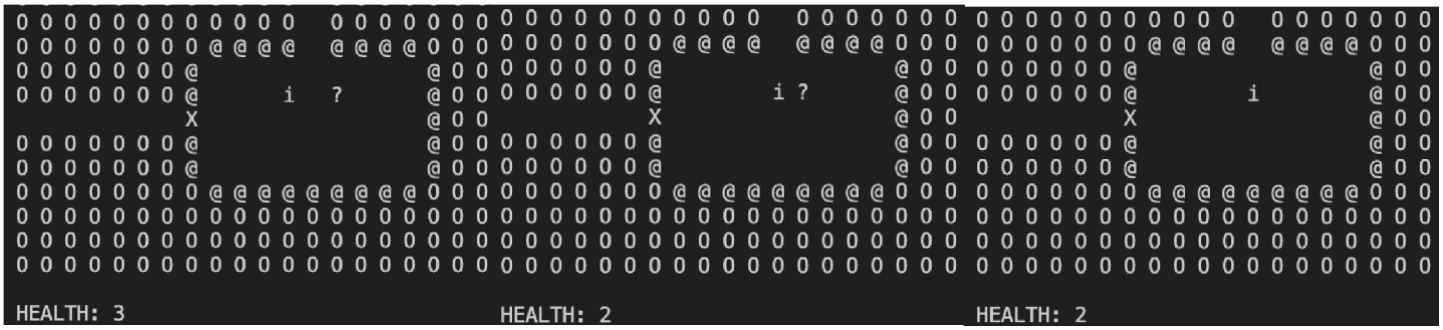
Explanation:



The enemy chooses a random direction to go in (up, down, left or right). If it attempts to move onto a tile with collision it just remains in its current position. Shown above is the enemy moving to the right, and then down, and then to the left.

4. Player and enemy attacking

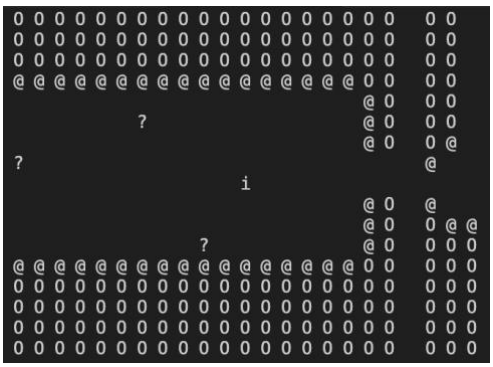
Screenshots:

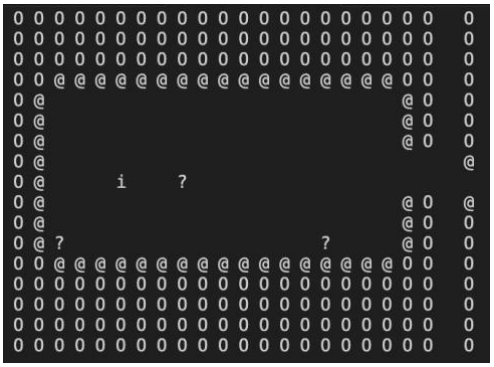


Explanation:
With both the player and enemies, each attack by moving onto a position with an entity already at that position. Instead of the enemy/player moving to that position, they do damage to the entity at that position. Shown above, on the left, is the player and a HorizontalEnemy (moving left). In the middle, the player moves one space to the right, and the HorizontalEnemy attempts to move one space to the left. This results in the enemy attacking the player and the player taking damage. On the right, the player attempts to move right and attacks the HorizontalEnemy, killing it.

5. Player camera

Screenshots





Explanation

The camera has a variable width and height, and keeps the player in the centre. If the player reaches any of the boundaries, then they will no longer be centred. It displays the dungeon around the player.

Takeaways:

When I was first implementing the dungeon, I used a 2D array of characters with unique characters for each tile. This made it very troublesome when tiles had different properties, such as collision or having tiles that could be interacted with, so it made me realize that using a 2D array of classes would be a much better option. I also tried to store the player as a character in the same 2D array, which would cause problems with trying to store the tile that the player was previously on when they moved, so I decided to create a separate 2D array for entities altogether, which worked out much better. Although I am happy with my prototype and think that it's built a solid foundation for my game, it feels very clunky and not very user friendly. I realize now that I need to work hard to make the game feel smoother and look more appealing to the player.

1.5.2 Second interview

1. **What do you think about my key components?**

   I like that you've considered making the game both engaging and easier to play for beginners through adding options for difficulty and then also adding a variety of weapons, items and enemies. Making the controls simple will ensure that beginners don't get frustrated - I've had it where I've bought a game and had to refund it since it felt exhausted from trying to learn all the in's and out's before I could have any fun. Making it so that difficulty will increase as the game progresses will add a nice learning curve so that beginners don't immediately have their hopes destroyed yet also have room to improve.

2. **What features do you think I could improve on in my prototype?**

   I think that there should be some way of distinguishing what type each enemy is. The different types of enemies are all displayed as '?', which could make it very hard to figure out which type of enemy a certain enemy is. This would make it much more challenging to fight, particularly in a room with many enemies.

3. **What existing features do you like in my prototype?**

I love the camera system you've implemented. It makes it so that you can still see a fair amount of the dungeon without the individual tiles being too small. If you had displayed the entire dungeon then I would have a hard time focusing on just the player, and would occasionally lose where I was.

4. **Are the controls in my prototype simple?**
   I found the controls very easy to use - using arrow keys to move around felt very intuitive.

5. **What did you think about the display of my prototype?**
   Simple, which is good and bad. I like how you can immediately assess your surroundings as each tile has a unique character, however the display is just black and white, which feels super boring. Add some more colour. And as I mentioned in question 2, make the display for each enemy different. Maybe making the enemies a different colour to the player would help too.

In summary, the key components of my program are sufficient to solve my problem. In regards to my prototype, I need to make each enemy have a unique display, and I need to add some more colour to the game to make it more engaging. I have successfully made the controls simple so that the player doesn't get overwhelmed, and the camera system I implemented doesn't reveal too much of the dungeon yet doesn't obscure the player's view such that they are surprised when they enter a room.

1.6 Objectives

1. User interface features
    1.1 Main menu
        1.1.1 Player can navigate menu using up and down arrow keys, and select options by using enter key
        1.1.2 An option to change difficulty level, with options for 'easy, 'normal', 'hard' and
        'impossible'
        1.1.3 An option to start game, which will start the game
        1.1.4 An option for a tutorial, which will display information about the game
        1.1.5 An option to exit game, which will end the program

    1.2 HUD
        1.2.1 Health bar in top left of screen, which are hearts
        1.2.2 Weapon display in bottom left of screen
        1.2.3 Item display in bottom left of screen, with a duration counter if it has a duration
        1.2.4 Ability display, with a cooldown and duration counter in bottom left of screen
        1.2.5 Money display in top left of screen
        1.2.6 Key display in bottom left of screen
        1.2.7 Damage display in bottom left of screen

2. Gameplay features

    2.1 Player controls

2.1.1 Player can move up, down, left or right using the arrow keys

2.1.2 Player can activate ability using q key

2.1.3 Player can use item using e key

2.1.4 Player can attack up, down, left or right using wasd keys

2.2 Dungeon features

2.2.0 Dungeon will be randomly generated

2.2.1 Rooms

2.2.1.1 Random rooms have a random size, have walls placed within them randomly and will have enemies

2.2.1.2 Chest rooms will have a constant size, have a chest in the center which can be opened by moving toward them and will not contain enemies

2.2.1.3 Starting room will have a constant size and will not contain enemies

2.2.1.4 Boss room will have a constant size and will contain the boss

2.2.2 Dungeon will have different types of tiles

2.2.2.1 Wall tiles and door tiles will have collision

2.2.2.2 Floor tiles won't have collision

2.2.2.3 Damage tiles won't have collision, and will do damage to any entities that are on it

2.2.2.4 Chest tiles will have collision, and will drop an item when destroyed

2.2.3 Enemies

2.2.1.1 Enemies move in different ways

2.2.1.1.1 Some enemies move around randomly

2.2.1.1.2 Some enemies find the shortest path to the player and move toward them

2.2.1.1.3 Some enemies don't move around at all

2.2.1.1.4 Some enemies can move only up, down, left, right

2.2.1.1.5 Some enemies find the shortest path to the player, but avoid damage tiles

2.2.1.2 Enemies have different assets (look different)

2.2.1.3 Enemies attack differently

2.2.2.3.1 Some enemies will used ranged attacks

2.2.2.3.2 Some enemies will attack by moving directly into player

2.2.2.3.3 Some enemies will leave damage tiles where they move

2.2.2.3.4 Some enemies will do an area of effect attack around them

2.2.1.4 Enemies will drop coins

2.2.4 Levels

2.2.4.1 The player can reach new levels after defeating the boss on the level they are currently on

2.2.4.2 The difficulty will increase as the player progresses through levels by adding more rooms, increasing room size, making the dungeon bigger and adding more types of enemies

2.3 Items

2.3.1 There will be different weapons which attack differently

2.3.1.1 Only one can be held at once

2.3.2 There will be different active items

2.3.2.1 Only one can be held at once

2.3.2.2 Have a uses count which decreases after each use

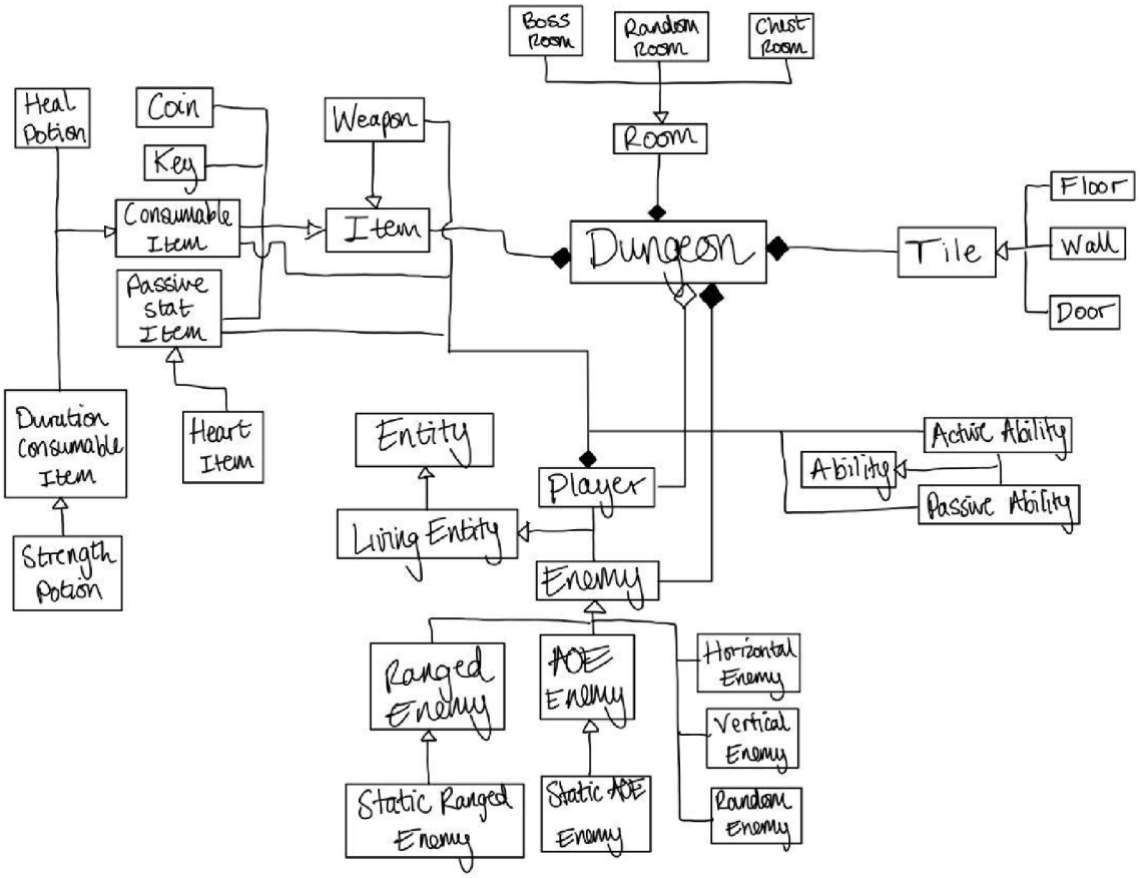
2.3.2.3 Some have a duration, some don't

2.3.3 There will be different passive stat items

2.3.3.1 Many can be held at once

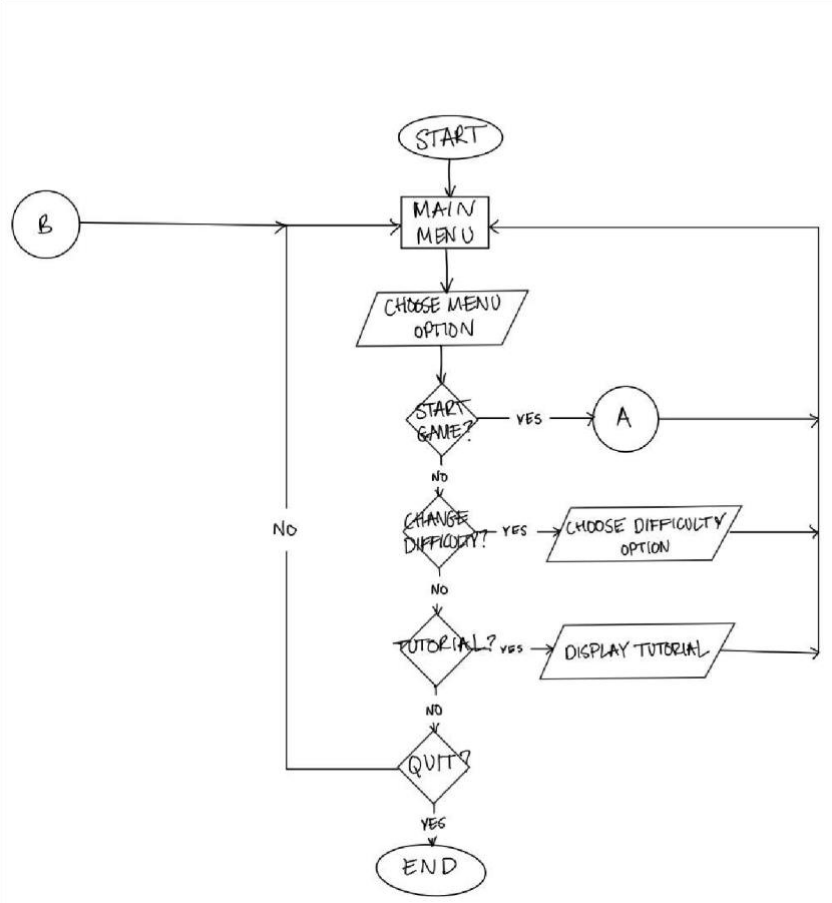
1.7 Modelling

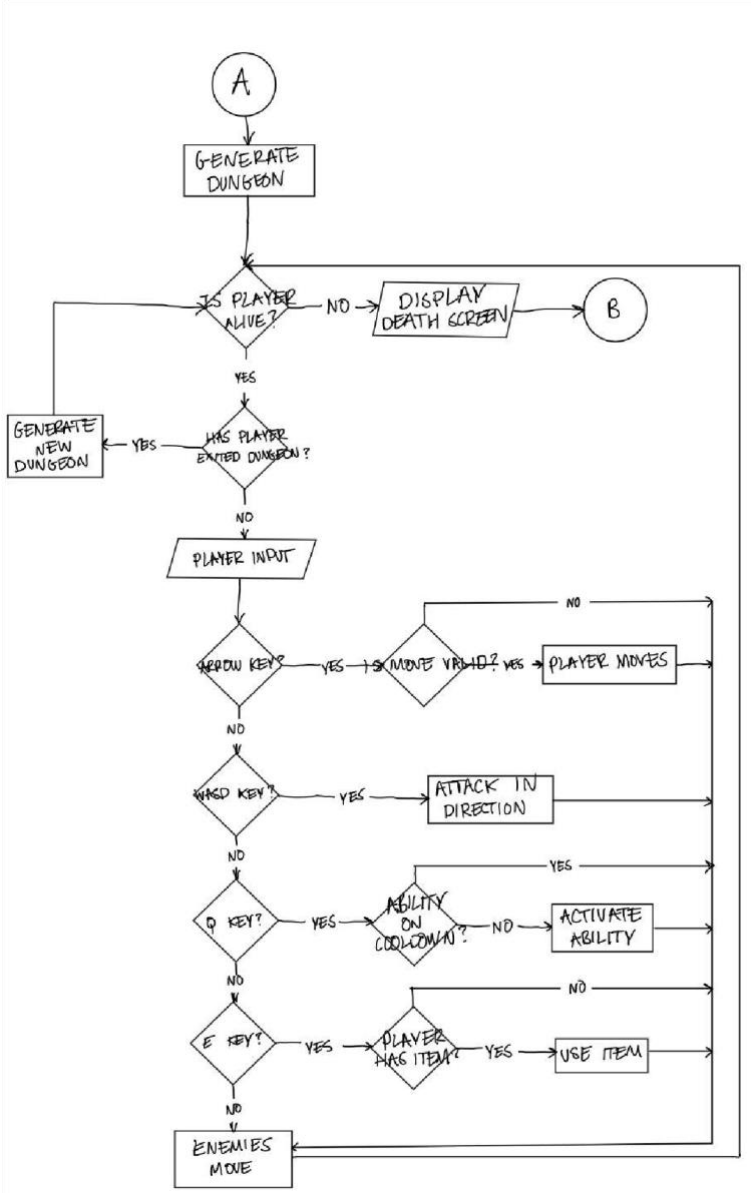
1.7.1 Class diagram



Shown above is my class diagram, which represents how my classes will interact and inherit from others. Adding additional classes would be much easier due to creating this class diagram since I already have a clear framework to work onto.

1.7.2 Flowchart





My flowchart shows the main flow of my program, which will make it much easier to program my project since I've already planned the order in which processes occur. The first flowchart illustrates the flow of my main menu, and the second flowchart illustrates the flow of the game.

2 Design

2.1 High-Level overview

The purpose of my program is to provide the player with a dungeon roguelike game that is beginner-friendly. My game will take place on a 2D grid with an ASCII display that uses different colours and the game will be open-ended such that there is no 'ending' - the game ends once the player dies. Following the nature of roguelikes, my game will include a procedurally generated dungeon, a wide array of items to collect and use, many different enemies which have varying levels of difficulty to defeat and the game will implement permadeath. To prioritise simplicity, the player and enemies can only move orthogonally,

limiting the number of controls for the player and the range of moves that enemies can make.

My menu provides a way to reach the other functions of my program besides the actual game: it allows the player to change the difficulty to provide a more tailored experience, to read the tutorial in order to gain a better understanding of the game, or to exit the program.

2.2 Choice of programming language

To write my program, I chose to use the C# programming language. One reason behind this decision is that it's the language I'm most proficient in, meaning that I can spend more time making my program and less time learning a new language. It also means that I can write much better and efficient code. Another reason is that my program is centered around using classes for entities and items, and C# is renowned for being object-oriented. This allows me to create a comprehensive structure for entities and items for my game, which can easily be built upon if needed.

2.3 Key components

2.3.1 Main menu

The main menu allows the player to access each of the features of my program, and includes the options: 'Start game', 'Change difficulty','Tutorial' and 'Exit Game'. The menu is navigated by using the up and down arrow keys to move a pointer up and down, and then an option is selected by using the enter key.

2.3.1.1 Algorithms

Displaying menu

The menu is displayed by using a for loop over an array of strings which contains the different menu options. If the value of the counter of the for loop is equal to the value of the menu pointer, then the menu option is written to the console with '><' surrounding it, otherwise the menu option is written to the console as is. This is to clearly indicate to the user which option they are selecting.

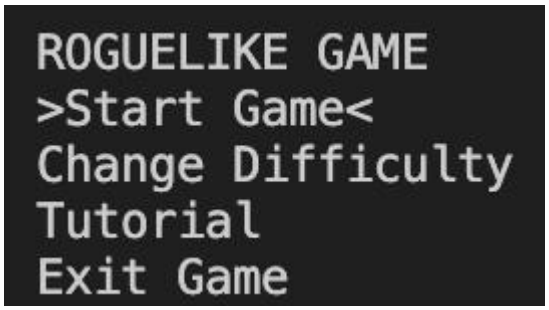
Player input

After the menu has been displayed, the 'GetMenuChoice' subroutine is called, with a reference to the menu pointer and variable 'max' (the index of the last menu choice) passed in as arguments. The user's key input is read and then stored in a variable. An if statement is used such that if the user presses the up arrow key and the value of the menu pointer is greater than 0 (the index of the first menu choice in the array storing menu choices) then the menu pointer is decremented by 1 and the subroutine returns false (to indicate that the user hasn't selected an option), if the user presses the down arrow key and the value of the menu

pointer is smaller than the value of ‘max’ then the menu pointer is incremented by 1 and the subroutine returns false, if the user presses the enter key then the subroutine returns true (to indicate that the user has selected an option) and if the user presses any other key then the subroutine just returns false.
An if statement in the menu subroutine uses a switch statement on the menu pointer if the GetMenuChoice subroutine returns true in order to call the corresponding subroutine for the option selected. If the menu pointer is 0, then the PlayGame subroutine is called, if the menu pointer is 1, then the ChangeDifficulty subroutine is called, if the menu pointer is 2, then the Tutorial is called, if the menu pointer is 3, then the ‘exit’ variable is changed to true, which will break the condition of the while loop in the menu thus ending the program.

To store the menu options in my menu subroutine, I used a 1D array. This allowed me to use my pointer variable as an index pointing to which option in the 1D array the user was currently on. A 1D array is suitable instead of a list, since I don’t need to add any additional options, thus there is already a determined number of options.

2.3.1.2 UI Design



As shown above, my menu is very simplistic, being monochromatic. I chose to do this so as to not take any focus away from the main feature of my program, the dungeon game.

2.3.2 Game display

I used ASCII to display my game and chose to use only a few different colours in order to keep my game simple and beginner friendly.

The game display consists of three different components:

- The top player display, which shows the player’s HP and the number of coins they have
- The dungeon display, which shows the player and the dungeon map.
- The bottom player display, which shows the player’s damage, the number of keys they have, the weapon they’re currently holding, the item they’re currently holding, the player’s active ability and the player’s passive ability

2.3.2.1 Algorithms

Player HP display

To display the player's HP, I used a for loop which repeats as many times as the value of the player's current HP, writing a red '<3' to the console each time, and then followed this for loop with another for loop, which repeats as many times as the difference between the player's max HP and current HP, writing a dark grey '<3' to the console each time.

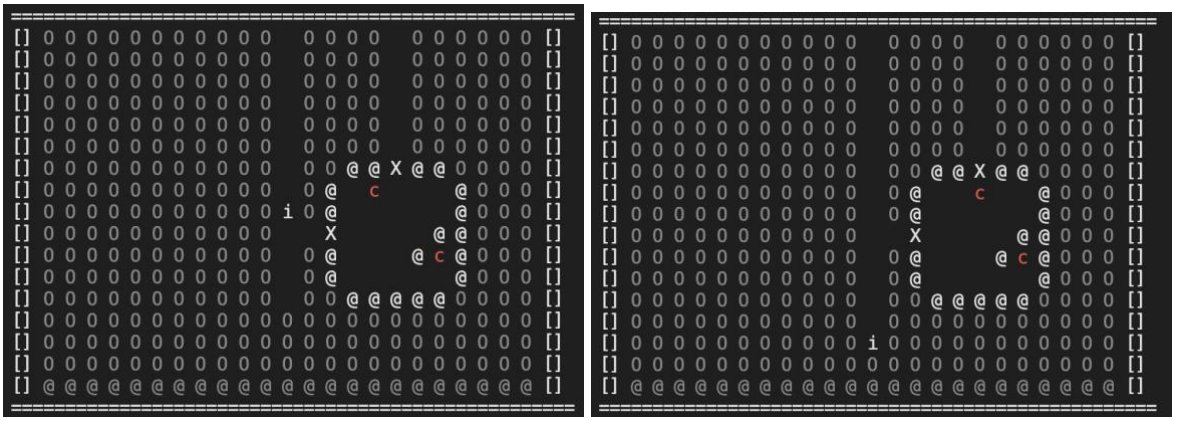
Dungeon display

To find where the edges of the player camera will be, I used an if statement to check if each edge is within the bounds of the dungeon array. For finding the top and bottom rows of the camera, I first checked if the player's y position minused by the camera height divided by two was greater than or equal to 0. If this was true, then I know that the top of the camera is within the bounds, so then I check if the bottom is too: if the player's y position added by the camera height divided by two is smaller than or equal to the height of the dungeon, then the height of the camera is also within the bounds. Thus, the top and bottom rows of the camera are set to half the camera height above and below the player's y position. However, if the bottom row isn't within the bounds but the top is within the bounds, then the top row of the camera is calculated by minusing the height of the dungeon by the height of the camera and the bottom edge is merely the bottom of the dungeon, equal to the height of the dungeon. If the top edge isn't within the bounds but the bottom edge is within bounds, then the bottom row of the camera is the camera height minused by 1, and the top row is merely the top of the dungeon.

Finding the leftmost and rightmost columns of the camera is the same as finding the top and bottom rows, but instead of using the dungeon height, the dungeon width is used, and similarly, instead of using camera height, camera width is used.

After calculating the values of the leftmost and rightmost columns and the top and bottom rows of the player camera, a double for loop is used to iterate through each of the dungeon cells between the bounds of the edges calculated.

An if statement is then used to decide what will be displayed in each cell: if there is a character in the effectsLayer array at the same coordinates then display this character, otherwise if there is an entity in the entityLayer array at the same coordinates then display this entity, otherwise if there is an item in the itemLayer array at the same coordinates then display this item, otherwise display the dungeon tile at this coordinate.

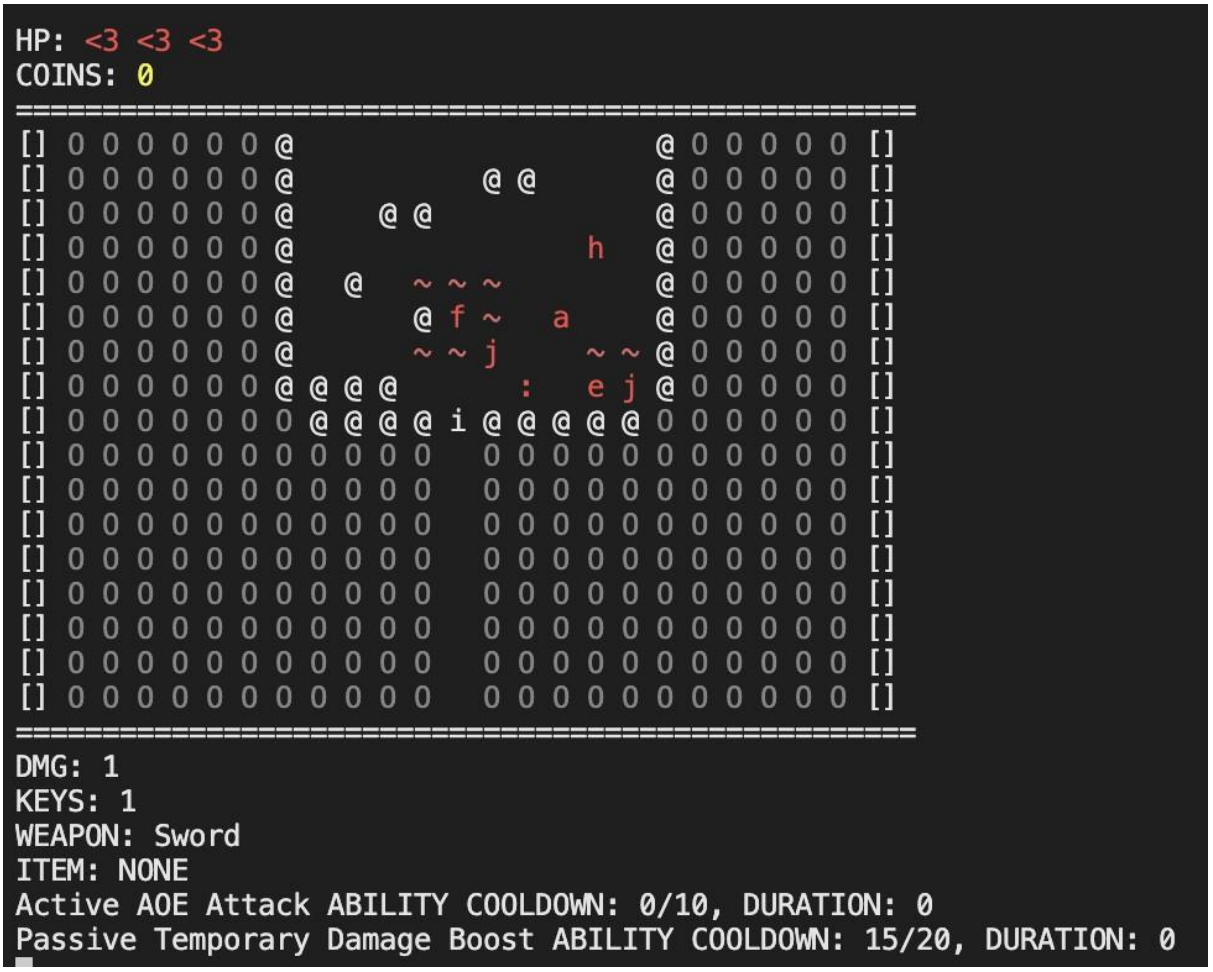


On the left shows the camera centered on the player as all of the edges are within bounds, on the right shows the camera off-centered on the player, as the bottom edge isn't within bounds but all others are

2.3.2.2 Data structures

To display my dungeon, I used a list of the 'DisplayCharacter' structure I created. The DisplayCharacter structure has two attributes: character and colour. This allowed me to add the DisplayCharacter for each cell in the dungeon to my list when iterating through the bounds of the camera. When using a for loop to write each DisplayCharacter to the console, using the colour attribute allowed me to display a particular colour for each cell easily.

2.3.2.3 UI design



Shown above is the game display, consisting of the top player display, dungeon map display and bottom player display

The player is represented by a white ‘i’.
The bright red letters represent enemies, with each unique letter representing a different type of enemy. The darker red characters around some of the enemies shows where the enemies’ next attack will be (for enemies with area of effect attacks or ranged attacks). Blank space represents floor tiles, white ‘@’s represents room walls and grey ‘O’s represent dungeon walls.

2.3.3 Dungeon map

The dungeon consists of many different layers overlapping each other: the tile layer, which stores the tile layout of the dungeon, the entity layer, which stores the enemies and player, the item layer, which stores any items that are dropped and the effects layer, which is used to store visual effects produced by entities for the dungeon display.
The dungeon map itself, or more specifically the tile layer, begins as a blank slate which is then filled by unique rooms that are connected by corridors by using a procedural dungeon generation algorithm.

Tiles

There are 7 different types of tiles that are used throughout the dungeon, each of which has a unique purpose.

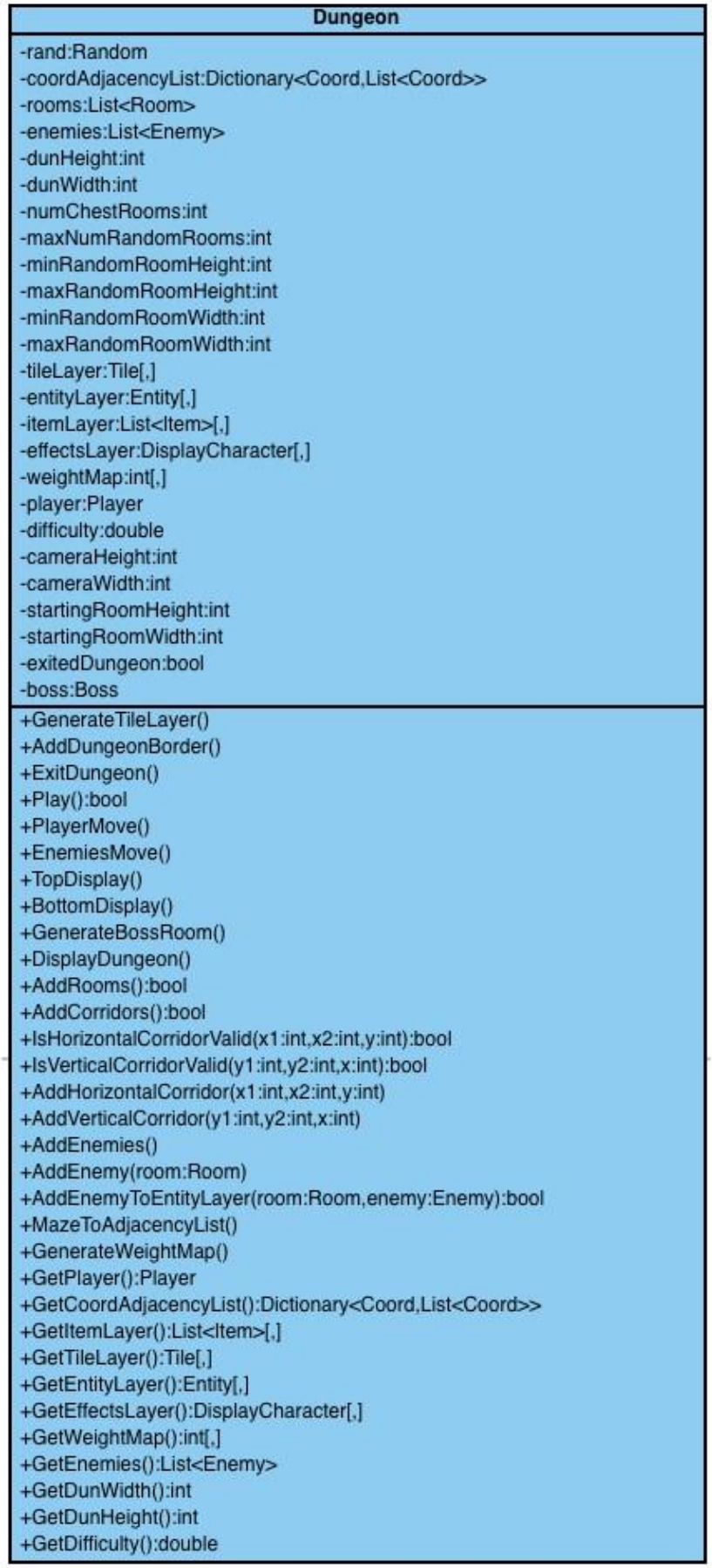
- Floor tile - the main tiles that the user moves on to traverse around the dungeon
- Wall tile - used to distinguish where rooms are located, they cannot be immediately moved onto; they must be destroyed first
- Door tile - used to separate rooms from corridors, they cannot be immediately moved onto; they must be opened first
- Dungeon border tile - used as a border which the player can't ever move across (for example, surrounding the edge of the dungeon)
- Chest tile - drops an item when interacted with, they cannot be immediately moved onto; they must be opened first
- Damage tile - damages any entities standing on it, they can be moved on immediately

Rooms

There are 4 different types of rooms that appear throughout the dungeon, each of which have a different layout:

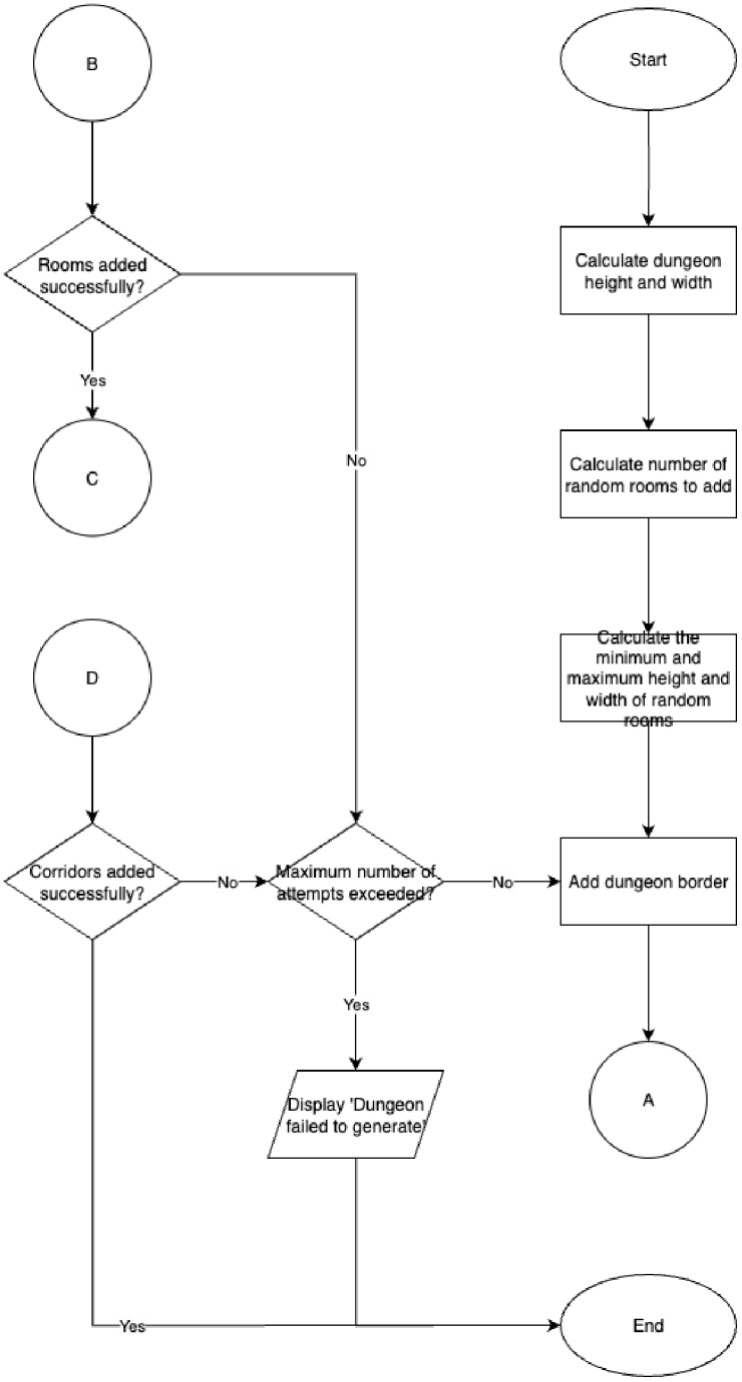
- Starting room - where the player spawns at the start of each dungeon floor, always 5 by 5 tiles
- Random room - where enemies are found, has a randomly generated size
- Chest room - contain a chest in the center, always 7 by 7 tiles
- Boss room - contains the boss which the player must defeat to progress to the next dungeon floor, always 29 by 29 tiles

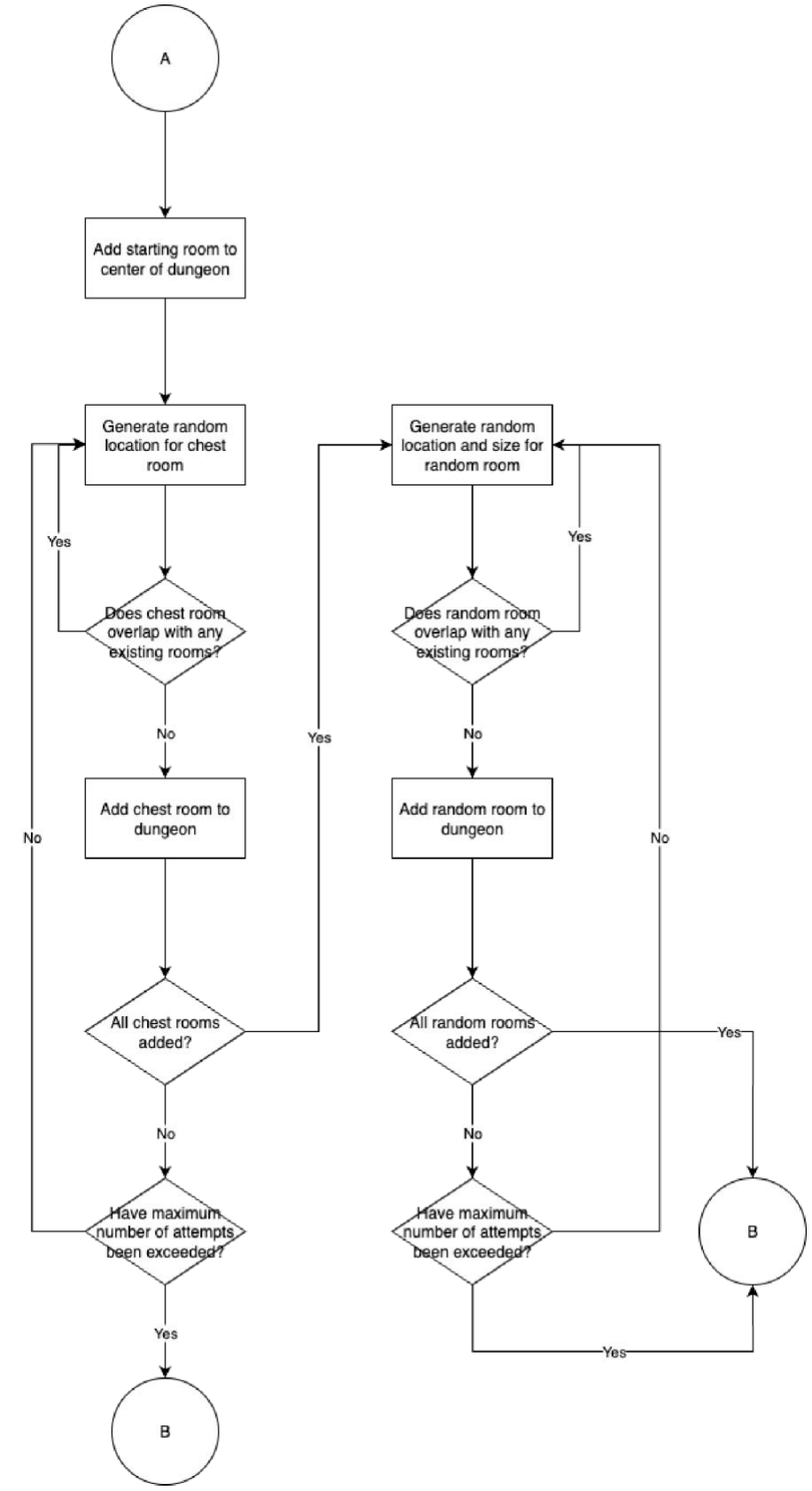
Dungeon class definition

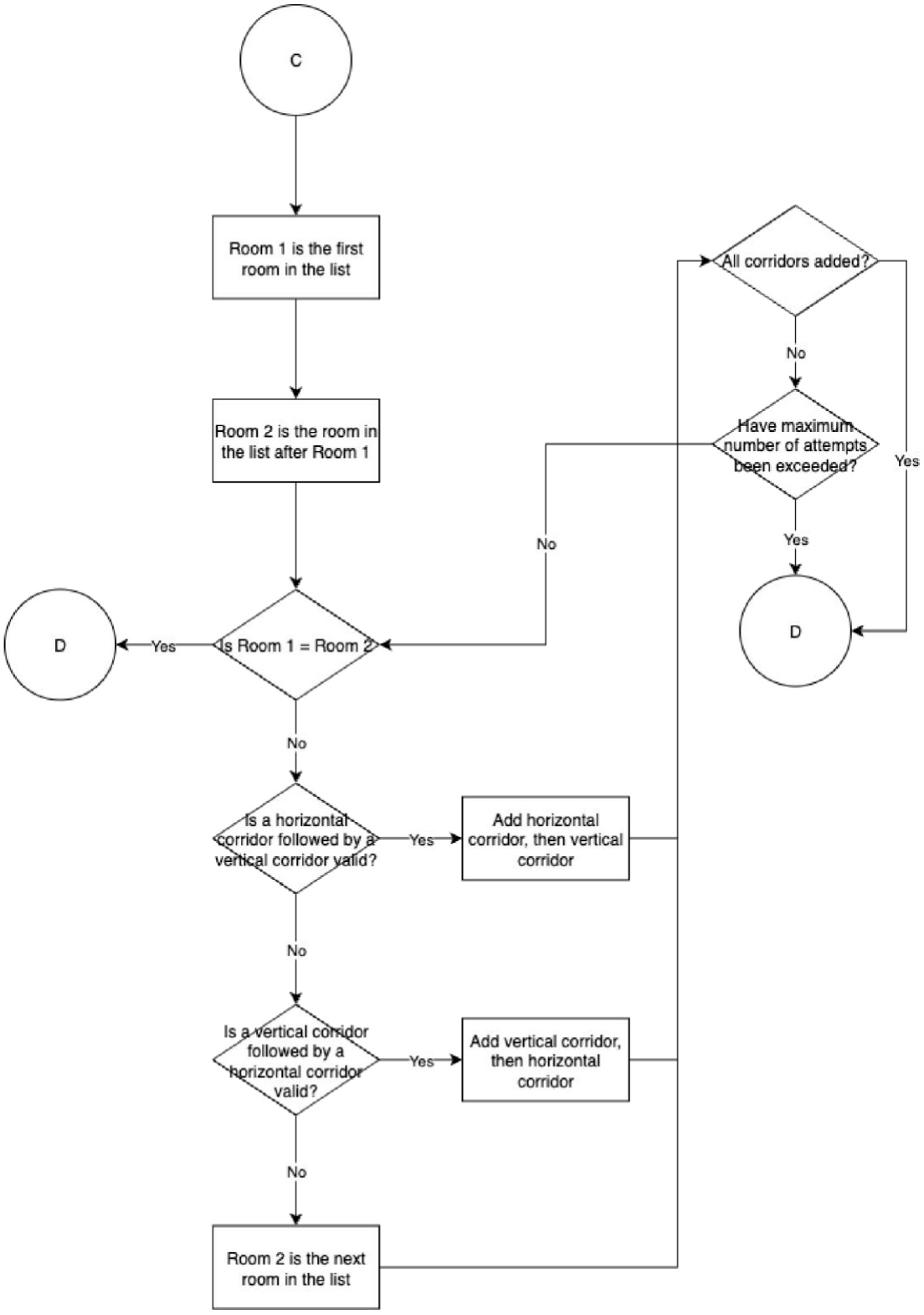


Dungeon generation flowchart

Below is a high-level flowchart which shows how my dungeon generation algorithm works. I needed my dungeon generation to be random in order to provide the user with a unique experience in each run, thus my rooms are placed in the dungeon with a random location. I used while loops for adding sets of rooms and corridors to the dungeon, and for adding individual rooms and corridors.




A
Add starting room to center of dungeon
Generate random location for chest room
Yes
Does chest room overlap with any existing rooms?
No
Add chest room to dungeon
No
All chest rooms added?
Yes
No
Have maximum number of attempts been exceeded?
No
Yes
B
Generate random location and size for random room
Yes
Does random room overlap with any existing rooms?
No
Add random room to dungeon
No
All random rooms added?
Yes
No
Have maximum number of attempts been exceeded?
No
B
Yes



Calculating attributes of the dungeon floor

To decide the width and height of a dungeon floor, 45 is multiplied by the difficulty and the width and height of the dungeon cannot exceed 150. Otherwise, the dungeon would take too long to navigate and there would be problems with so many enemies being in the dungeon and trying to move at once.

Similarly, the number of random rooms in a dungeon floor is calculated by multiplying 3 by the difficulty, and the number cannot exceed 8, since the size of the random rooms also increases with difficulty, and there could be problems with fitting them in the dungeon if there are too many.
To calculate the minimum and maximum width and height of the random rooms, the equations below are used:

minimum width or height = 4 * (**difficulty**+1) / 2 **maximum width or height** = 7 * (**difficulty**+1) / 2

I chose to add 1 to the difficulty and then divide by two so that the size of the rooms grows slower than the dungeon size and number of rooms in the dungeon.
I then made it so that the minimum width and height cannot exceed 7, and the maximum width and height cannot exceed 15, for the same reason as why I put a limit on the number of random rooms.

Room generation

The starting room is added to the center of the dungeon, and its x and y coordinates (which represents the top left corner of the room) are calculated by using the equation:

x or y position = **dungeon width or dungeon height** / 2 - 5 / 2

The significance of 5 is that it is the width and height of the starting room. The dungeon width or height is divided by two to find the center of the dungeon, and then 5 is divided by two and minused from the dungeon width or height so that the center of the starting room is at the center of the dungeon.

To calculate the x and y coordinates of a chest room, the equation used is:

x or y position = **RANDOM NUMBER BETWEEN** 1 **AND dungeon width or dungeon height** - 7 - 2

The ‘between’ is inclusive. The significance of 1 is that it is the first possible tile after the dungeon border (and we don’t want the room to be placed on the dungeon border, otherwise the player could attempt to get out of the bounds of the dungeon floor), and the significance of minusing 7 and 2 from the dungeon width or height is that it is the last tile before the dungeon border at the other end of the dungeon floor. The number 7 represents the width and height of the chest room. The number 2 is used because 1 needs to be minused since arrays are 0 indexed in c#, and another 1 needs to be minused so that the room isn’t built over the dungeon border.

To calculate the x and y coordinates of a random room, its height and width must be found first. The height and width of a random room is calculated using the equation:

height or width = **RANDOM NUMBER BETWEEN minimum width or minimum height AND maximum width or maximum height**

The x and y coordinates are then calculated using the equation:

x or y position = **RANDOM NUMBER BETWEEN** 1 **AND dungeon width or dungeon height** - **width or height** - 2

This is the exact same as the chest room calculation, except the width and height must be calculated first, and then they are used in the equation.

Room validation

To decide whether or not the location of a room is valid, a nested for loop is used to check every cell that the room might occupy, and if a tile already exists in any of the cells, then the location of the room is deemed invalid.
The region to cover is decided by going from the x and y coordinates of the room minused by 1 (to ensure the no rooms are directly bordering each other), to the x and y coordinates added by the width or height of the room added by 1 (for the same reason).

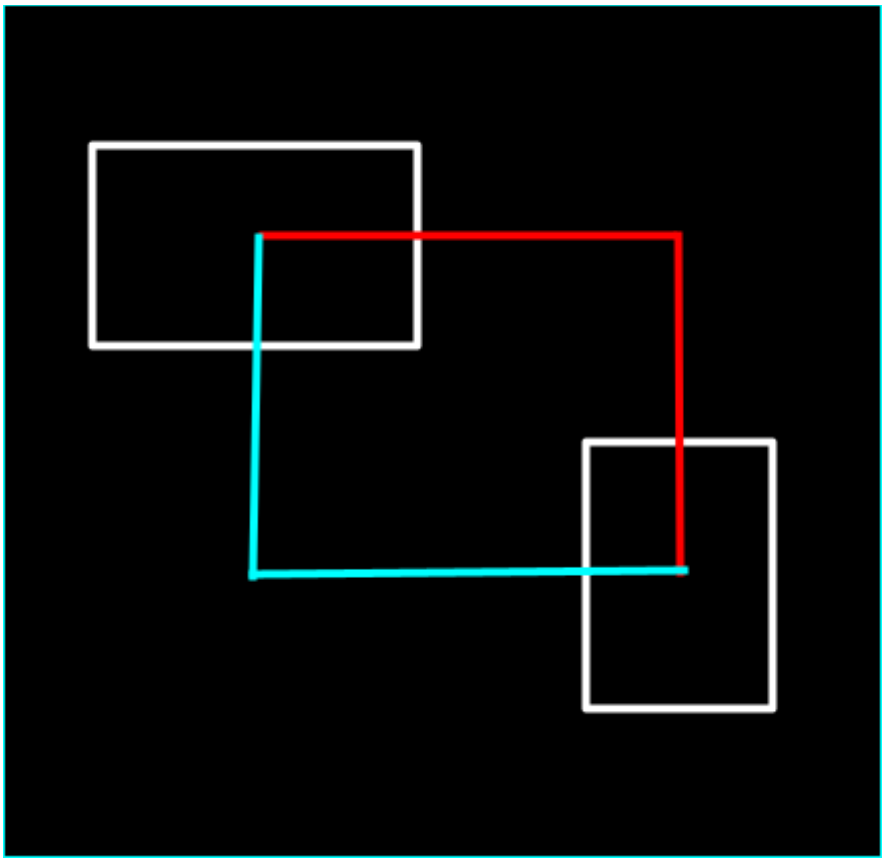
Room addition

Adding each room is slightly different, of course, but they each use a double for loop to cover every cell within its region (going from its x or y coordinates to its x or y coordinates added by the rooms width or height). Then, using if statements:

1. If the current cell is the corner of the room, leave it
2. If the current cell is at the edge of the room, place a wall
3. Otherwise, place a floor

There are exceptions, such as with the chest room, where a chest is placed in the center of the room, and with random rooms and boss rooms there is a chance of cells within the walls of the room to be walls instead of floors (so that there are barriers to evade enemies). Also, after the difficulty reaches a certain threshold, damage tiles are placed within the walls of boss rooms.

Corridor generation



Shown above is a diagram illustrating how a corridor could be generated between two rooms (the rooms are indicated by the white boxes). Corridors can either go horizontally, then vertically, as shown by the red line, or vertically and then horizontally, as shown by the blue line, which creates L shaped corridors.

In order to calculate where corridors will exist, the calculations below are used:

x or y coordinate of center of room = **x or y coordinate of room** + (**room width or height** - 1) / 2

horizontal corridor goes from **x coordinate of center of room 1** to **x coordinate of center of room 2** at **y coordinate of center of room 1 or 2**

vertical corridor goes from **y coordinate of center of room 1** to **y coordinate of center of room 2** at **x coordinate of center of room 1 or 2**

Corridor validation

To check if a corridor is valid, a for loop is used to go from one x value to another x value at a constant y value, or to go from one y value to another y value at a constant x value. Each cell is checked, and the number of consecutive walls are counted. If there is more than one consecutive wall, then the corridor is deemed invalid, since corridors are made such that they turn any walls they go through into doors, and if the corridor goes through more than one wall consecutively then the corridor is going through an entire edge of a room. This

could result in the entire side of a room being doors instead of walls, so I made it such that a corridor would be invalid if it went through more than one wall consecutively.

Corridor addition

To add a corridor, a for loop is used to go over each cell of the dungeon where the corridor will exist. A floor will be placed where there is currently no tile, and a door will be placed where there is currently a wall.

2.3.4 Player

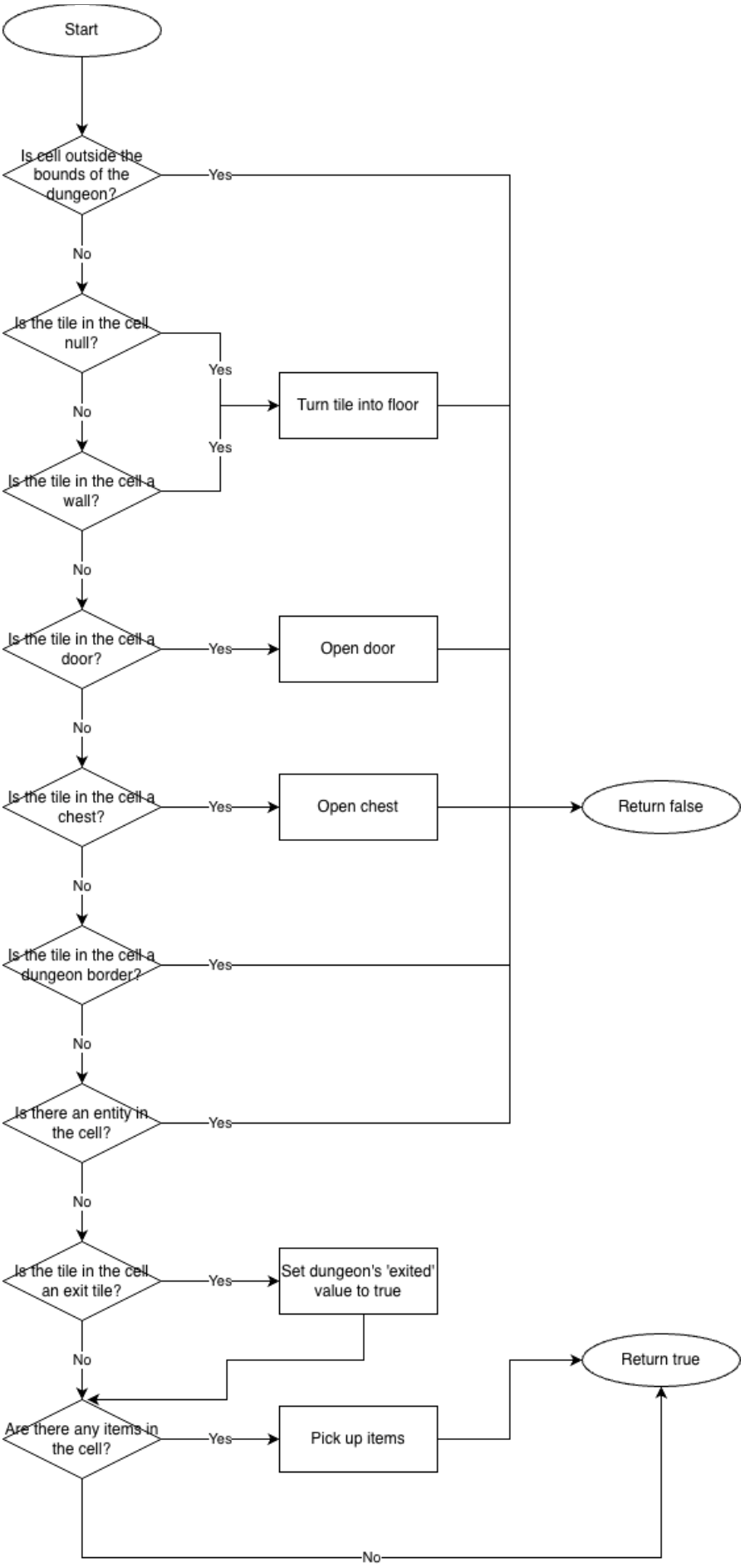
2.3.4.1 Player controls

The player character, represented by a white ‘i’, is controlled by the player by using the following keys:

- Up, down, left and right arrow keys to move the player in each direction
- W, A, S and D keys to allow the player to attack in each direction
- Q key to activate the player’s active ability
- E key to use the player’s active item

2.3.4.2 Movement validation

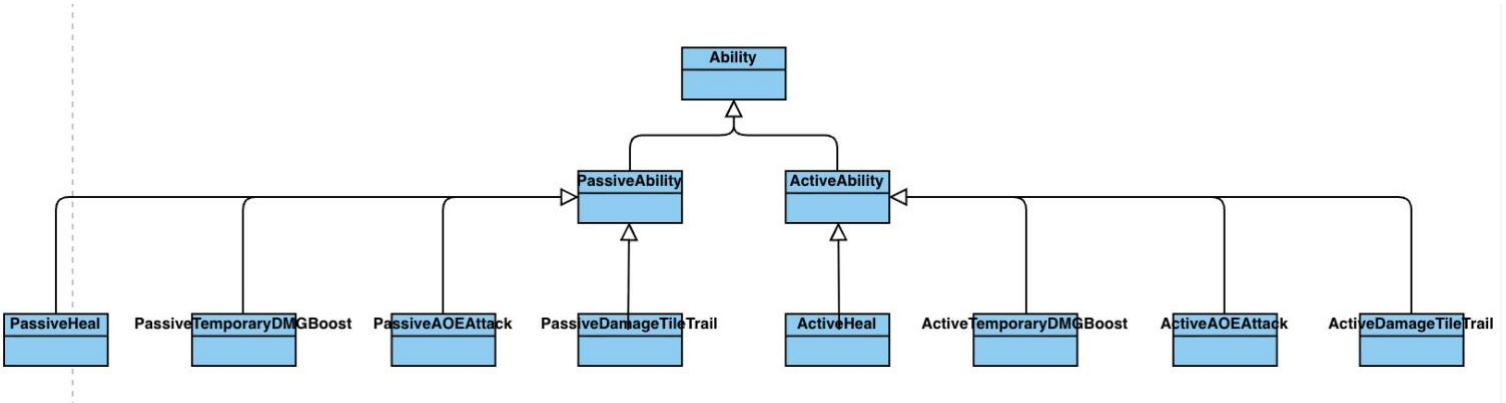
In order to prevent any errors from occurring from the player moving, such as moving out of bounds of the dungeon or moving into walls, validation of any movements that the player makes is required. This validation is shown in the flowchart below:


Start
Is cell outside the bounds of the dungeon?
Yes
No
Is the tile in the cell null?
Yes
Turn tile into floor
No
Is the tile in the cell a wall?
Yes
No
Is the tile in the cell a door?
Yes
Open door
No
Is the tile in the cell a chest?
Yes
Open chest
Return false
No
Is the tile in the cell a dungeon border?
Yes
No
Is there an entity in the cell?
Yes
No
Is the tile in the cell an exit tile?
Yes
Set dungeon's 'exited' value to true
No
Return true
Are there any items in the cell?
Yes
Pick up items
No

2.3.4.3 Abilities

The player has an active ability and passive ability, each of which are randomly decided when the player is created. The active ability only activates when it isn't on cooldown and the player presses the Q key, and the passive ability activates whenever the cooldown decreases to 0.

Class diagram



Class definitions

{Abstract} Ability
#cooldown:int #currentCooldown:int #abilityName:string #abilityDesc:string #duration:int #currentDuration:int #cooldownCap:int #durationCap:int
+Display() +CanUse():bool *+UseAbility(dungeon:Dungeon,player:Player)* *+DecreaseCooldown(dungeon:Dungeon,player:Player)* +ChangeCooldown(amount:int) +ChangeDuration(amount:int) +ChangeCurrentDuration(currentDurationIn:int) +GetDuration():int +GetCurrentDuration():int +GetCooldown():int

{Abstract} PassiveAbility
+DecreaseCooldown(dungeon:Dungeon,player:Player)

{Abstract} ActiveAbility
+DecreaseCooldown(dungeon:Dungeon,player:Player)

The Ability, PassiveAbility and ActiveAbility classes are all abstract, in order to provide a template for actual abilities that the player will use. The DecreaseCooldown subroutine from the ability class is abstract, since passive abilities and active abilities behave differently when decreasing in cooldown, thus they need different implementations of the DecreaseCooldown subroutine. The UseAbility subroutine is also abstract, to allow each ability used by the player to have a unique implementation.

Assigning random abilities

To give the player a random active and passive ability, a list of all the active and passive abilities are created. The player is then assigned their active and passive ability by selecting a random ability from each list.

Decreasing cooldown and duration

The decrease cooldown subroutine is called every time a move is made in the game. For active abilities, decreasing cooldown works as follows:

```
FUNC DecreaseCooldown(Dungeon dungeon, Player player)
        IF currentCooldown GREATER THAN 0
                IF currentDuration GREATER THAN 0
                        UseAbility(dungeon,player)
                ELSE            currentCooldown =
        currentCooldown - 1
                ENDIF
        ELSE
                IF currentDuration GREATER THAN 0
                        UseAbility(dungeon,player)
                ENDIF
        ENDIF
ENDSUB
```

The UseAbility subroutine is called when the duration of the ability is greater than 0, since this indicates that the ability is still active, thus it must be activated every turn until the duration returns to 0.

For passive abilities:

```
FUNC DecreaseCooldown(Dungeon dungeon, Player player)
        IF currentCooldown GREATER THAN 0
                IF currentDuration GREATER THAN 0
                        UseAbility(dungeon,player)
                ELSE
                        currentCooldown = currentCooldown - 1
                ENDIF
```

```
        ELSE
                currentDuration = duration
                UseAbility(dungeon,player)
        ENDIF
ENDSUB
```

As shown above, whenever the current cooldown reaches 0 the passive ability is automatically activated.

Overview of different abilities

- Temporary damage boost - temporarily boosts the player's damage by a certain amount for a certain amount of turns
- Heal - heals the player by a certain amount every turn for a certain amount of turns
- Area of effect attack - attacks any enemies in the cells surrounding the player every turn for a certain amount of turns
- Damage tile trail - places a damage tile on the cell that the player is currently standing on every turn for a certain amount of turns

2.3.4.5 Picking up items

The pseudocode for picking up items is shown below:

```
FUNC PickUpItems(Dungeon dungeon, List<Item> items, int x, int y)
        Entity[,] entityLayer = dungeon.GetEntityLayer()
        List<Item>[,] itemLayer = dungeon.GetItemLayer()
        List<Item> newItems
        FOREACH Item item IN items
                IF item TYPE EQUAL TO Coin TYPE
                Coin coin = item CAST TO Coin TYPE
                numCoins += coin.GetNumCoins()    ELSE IF
        item TYPE EQUAL TO Key TYPE               Key key
        = item CAST TO Key TYPE             numKeys +=
        key.GetNumKeys()
                ELSE IF item TYPE IS SUBCLASS OF Weapon TYPE
                        Weapon temp = weapon                weapon =
        item CAST TO Weapon TYPE
                newItems.Add(temp)
                ELSE IF item TYPE IS SUBCLASS OF ConsumableItem TYPE
                        IF consumableItem IS NULL
                                consumableItem = item CAST TO ConsumableItem TYPE
                        ELSE
                                ConsumableItem temp = consumableItem
                        consumableItem = item CAST TO ConsumableItem TYPE
                                newItems.Add(temp)  ENDIF
                ELSE IF item TYPE IS SUBCLASS OF PassiveStatItem TYPE
```

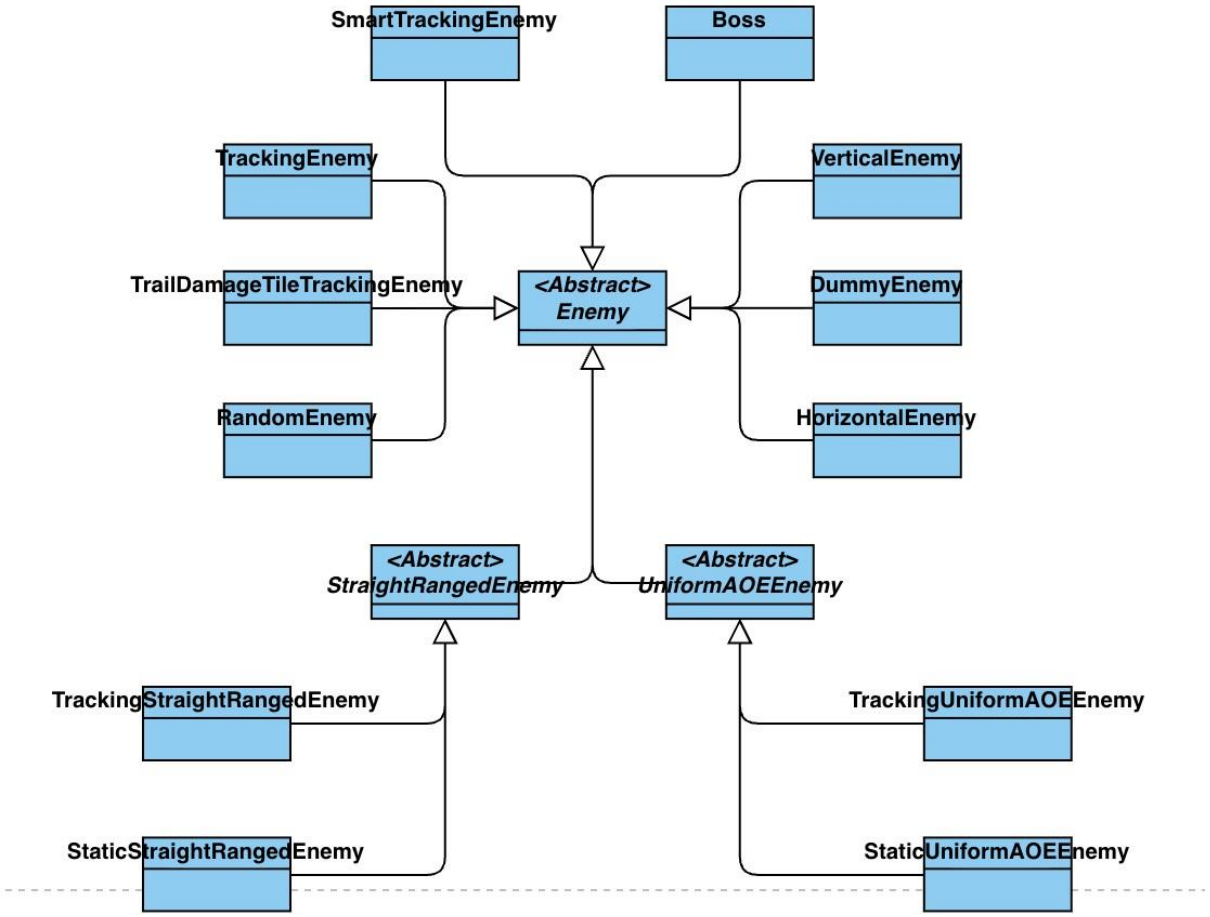
```
                    PassiveStatItem temp = item CAST TO PassiveStatItem TYPE
                temp.BoostPlayerStat(this, entityLayer)          passiveStatItems.Add(temp)
                ENDIF
            ENDFOREACH
            IF newItems.Count SMALLER THAN OR EQUAL TO 0
                itemLayer[y,x] = null ELSE
                itemLayer[y,x].Clear()
                FOREACH Item item IN
            newItems
                itemLayer[y,x].Add(item)
                ENDFOREACH
            ENDIF
ENDFUNC
```

Before giving any items to the player, as shown, the item's type must be checked, and then it must be casted to its corresponding specialised type. The newItem list is used to hold any items that the player drops when picking up new items - such as weapons or consumable items, since the player can only hold one of each of these. After the player has picked up the items in a cell, the items in the cell are cleared and then the contents of newItem are transferred. The reason why consumableItem is checked to be null and weapon isn't is because it is guaranteed that the player will be holding a weapon, whereas the player might not be holding a consumableItem since they could've used their previous item or not have picked one up yet.

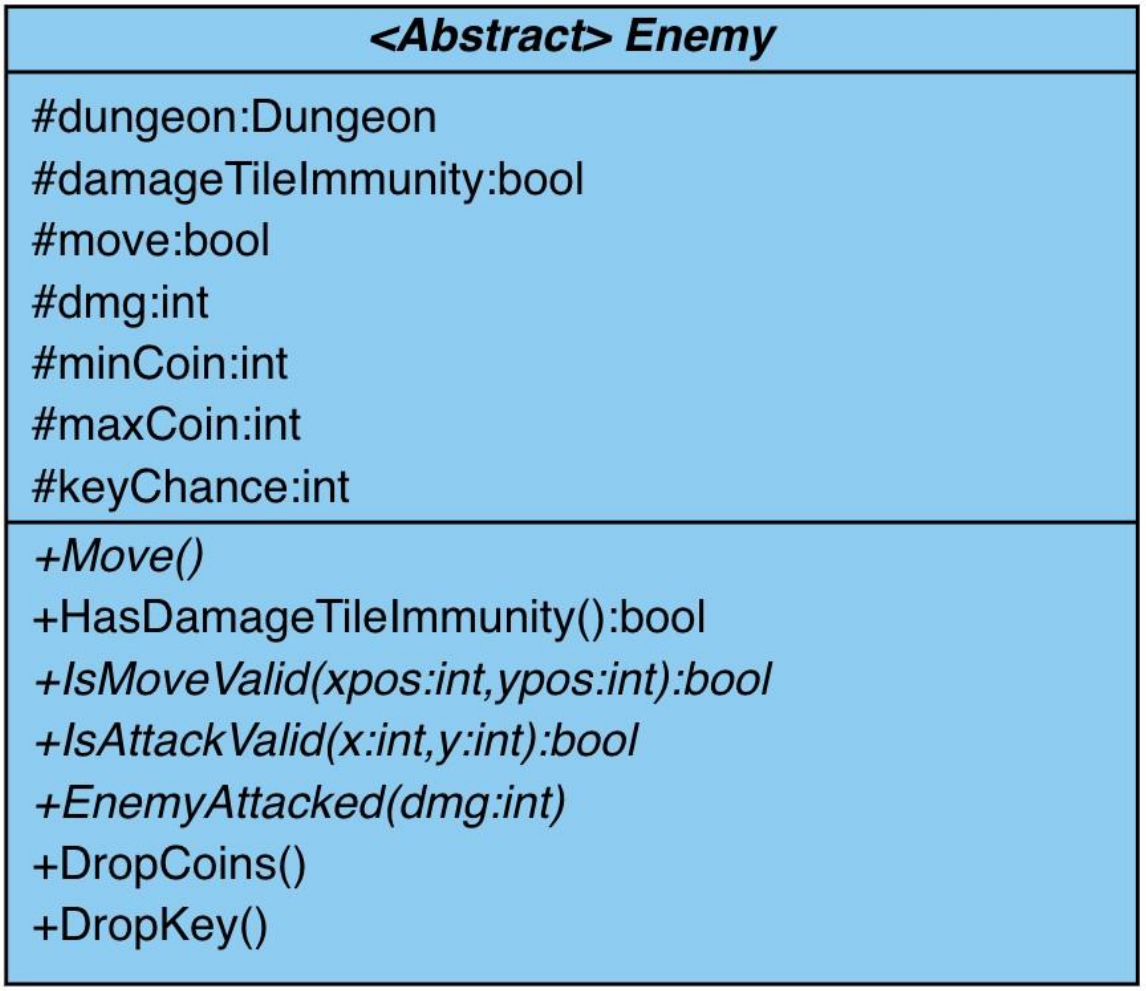
2.3.5 Enemies

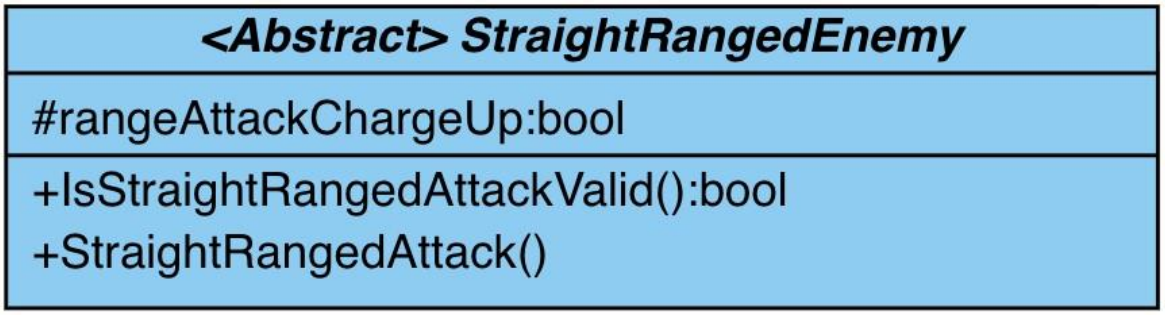
Different types of enemies move differently, attack differently, drop different amounts of coins, do different amounts of damage and have different amounts of health.
As difficulty increases, more types of enemies appear, and enemies that have previously appeared might have their damage or health increased.

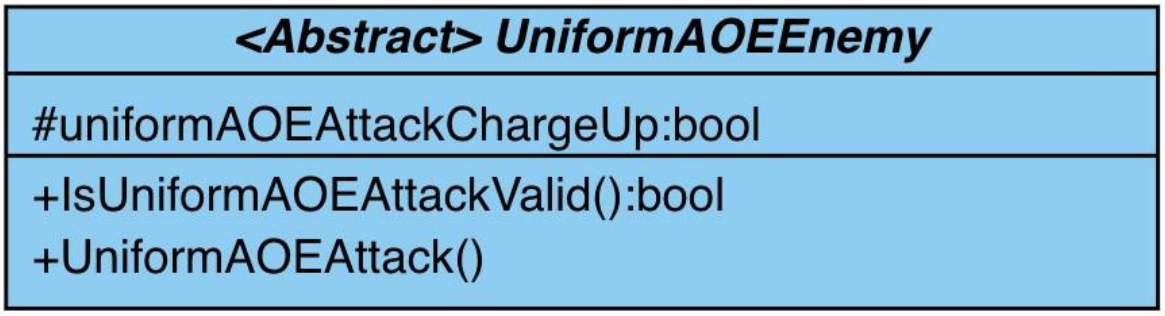
Enemy class diagram



Enemy class definitions







In the class diagram above, inside the Enemy class, the method Move is abstract, since every enemy moves differently thus they will all implement move differently. Differently, the

methods IsMoveValid, IsAttackValid and EnemyAttacked have the same implementation for some enemies and different implementations for a select few, thus they are virtual.

Adding enemies to the dungeon

Each room in the dungeon has three attributes that are important to adding enemies:

- The boolean variable ‘enemies’ which indicates whether or not a room will contain enemies
- The integer variable ‘minEnemies’ which indicates the minimum number of enemies a room can contain
- The integer variable ‘maxEnemies’ which indicates the maximum number of enemies a room can contain

The boolean variable is set to true for random rooms, and false for chest rooms and the starting room.

In order to determine the value of minEnemies and maxEnemies, the following algorithm is used, in pseudocode:

```
minEnemies = 1
IF (height-2)*(width-2) SMALLER THAN 16
        maxEnemies = 1
ELSEIF (height-2)*(width-2) SMALLER THAN 25
        maxEnemies = 2
ELSEIF (height-2)*(width-2) SMALLER THAN 36
        maxEnemies = 3
ELSEIF (height-2)*(width-2) SMALLER THAN 49
minEnemies = 2        maxEnemies = 4
ELSEIF (height-2)*(width-2) SMALLER THAN 64
minEnemies = 3        maxEnemies = 5
ELSEIF (height-2)*(width-2) SMALLER THAN 81
minEnemies = 4        maxEnemies = 6 ELSE
minEnemies = 4        maxEnemies = 6
ENDIF
```

I wanted to make it so that the possible number of enemies in a room is proportional to the number of floor tiles in the room, hence why (height-2)*(width-2) is used, which calculates the number of floor tiles in a room.

After this, the AddEnemies subroutine is called. This subroutine goes over each room in the dungeon and does the following:

1. Check if the room has enemies by using the room’s HasEnemies() subroutine
2. If the room has enemies, then get the number of enemies in the room using the room’s GetNumEnemies() subroutine
3. Call the AddEnemy subroutine for as many number of enemies there are in the room

The AddEnemy subroutine generates an enemy, and then calls the AddEnemyToEntityLayer subroutine to add the enemy to the dungeon. The enemy generated depends on the difficulty

- for certain ranges of difficulty, a certain group of enemies are randomly chosen from in order to generate an enemy.
The groups of enemies for ranges of difficulties are shown:

A difficulty <= 1.5:
- a VerticalEnemy (HP 1, DMG 1)
- A HorizontalEnemy (HP 1, DMG 1)
- A RandomEnemy (HP 1, DMG 1)

A difficulty <= 2:
- a VerticalEnemy (HP 2, DMG 1)
- A HorizontalEnemy (HP 2, DMG 1)
- A RandomEnemy (HP 1, DMG 1)
- A TrackingEnemy (HP 1, DMG 1)

A difficulty <= 3:
- a VerticalEnemy (HP 2, DMG 1)
- A HorizontalEnemy (HP 2, DMG 1)
- A RandomEnemy (HP 2, DMG 1) ● A TrackingEnemy (HP 2, DMG 1)
- A StaticStraightRangedEnemy (HP 1, DMG 1)
- A StaticUniformAOEEnemy (HP 1, DMG 1)

A difficulty <= 4:
- a VerticalEnemy (HP difficulty, DMG (difficulty/2))
- A HorizontalEnemy (HP difficulty, DMG (difficulty/2))
- A RandomEnemy (HP difficulty, DMG (difficulty/2)) ● A TrackingEnemy (HP difficulty, DMG (difficulty/2))
- A StaticStraightRangedEnemy (HP difficulty-1, DMG (difficulty/2)-0.5)
- A StaticUniformAOEEnemy(HP difficulty-1, DMG (difficulty/2)-0.5)
- A TrailDamageTileTrackingEnemy (HP 1, DMG 1, MOVE DELAY 5)

A difficulty <= 5:
- a VerticalEnemy (HP difficulty, DMG (difficulty/2))
- A HorizontalEnemy (HP difficulty, DMG (difficulty/2))
- A RandomEnemy (HP difficulty, DMG (difficulty/2)) ● A TrackingEnemy (HP difficulty, DMG (difficulty/2))
- A StaticStraightRangedEnemy (HP difficulty-1, DMG (difficulty/2)-0.5)
- A StaticUniformAOEEnemy(HP difficulty-1, DMG (difficulty/2)-0.5)
- A TrailDamageTileTrackingEnemy (HP 1, DMG 1, MOVE DELAY 3)

A difficulty >5:
- a VerticalEnemy (HP difficulty, DMG (difficulty/2))
- A HorizontalEnemy (HP difficulty, DMG (difficulty/2))
- A RandomEnemy (HP difficulty, DMG (difficulty/2)) ● A TrackingEnemy (HP difficulty, DMG (difficulty/2))
- A StaticStraightRangedEnemy (HP difficulty-1, DMG (difficulty/2)-0.5)
- A StaticUniformAOEEnemy(HP difficulty-1, DMG (difficulty/2)-0.5)
- A TrailDamageTileTrackingEnemy (HP difficulty-3, DMG (difficulty/2)-1.5, MOVE DELAY 2)
- A TrackingStraightRangedEnemy (HP difficulty-4, DMG (difficulty/2)-1.5)
- A TrackingUniformAOEEnemy (HP difficulty-4, DMG (difficulty/2)-1.5)

After selecting a random enemy from the group corresponding to the correct difficulty, the AddEnemyToEntityLayer subroutine is called.

The subroutine generates a random position for the enemy within their room, and then checks if the tile where the enemy will spawn has collision or if there is an entity already there. If neither of these conditions are true, then the enemy is placed and the subroutine ends. If either or both of the conditions are true, then the process repeats, and a new position is generated. This repeats until the maximum number of attempts is reached, or if the enemy is successfully added.

Enemy movement and validation

Enemy movement validation is identical to player movement validation, except from the fact that they cannot break walls, open chests or open doors. Instead of having conditions for each type of tile, like a player does, they instead have a single condition ‘if the tile has collision, then return false’.

All enemies move every other turn, in order to provide the player with more opportunities to strategise and predict what enemies will do.

Horizontal enemies move left and right, and their direction is determined by their boolean variable ‘right’, which is randomly assigned true or false when the enemy is generated. Every turn that they move, they check the value of right. If it is true, then the enemy checks the cell at the position (x+1,y) is valid, and if it is, then it moves to said position. If the IsMoveValid subroutine returns false, then it checks if it's because the player is at the position that it's trying to move to. If so, then the enemy stays at the same position but attacks the player. Otherwise, the enemy changes its right variable to false.
The same occurs if the enemy’s right variable is false, except the enemy would check the cell at the position (x-1,y) and would change right to true.

Vertical enemies have the exact same behaviour as horizontal enemies, except they have a boolean variable ‘down’ instead of ‘right’.

Random enemies randomly decide between changing their x or y position, and then randomly decide between changing their x or y position by 1 or -1.

Tracking enemies use BFS to find the shortest path and to decide the next position to move to.

The static enemies don’t move.

Smart tracking enemies and bosses use a form of Dijkstra’s to find the shortest path and to decide the next position to move to.

Enemy pathfinding using BFS and Dijkstra's

In order to use BFS and Dijkstra's, I needed to convert the tile layer of my dungeon into a graph. To do this, I created a subroutine inside the Dungeon class called MazeToAdjacencyList, which works as follows:

1. Create a dictionary where they keys are the Coord structure and the values are a list of Coord structures
2. Loop over every cell in the tile layer
3. If the cell isn't null and doesn't have collision, then add the coordinate of the cell to the dictionary as a key
4. Check if each of the cell's neighbours (cells that are directly connected above, below, to the left and to the right) aren't null and don't have collision
5. If a neighbouring cell satisfies these conditions, then add it's coordinate to the list of Coord structure where the key is the original cell

To store my graph, I decided to use an adjacency list instead of an adjacency matrix since I didn't need to quickly check the presence of particular neighbours to a cell, and I would have to store the coordinates of many cells which didn't have any valid neighbours, which would be a waste of memory. I also decided to only add the coordinates of cells that weren't null and didn't have collision since these are the only cells that entities can move on, hence there's no need to store information about cells that are incapable of being moved across for an algorithm to find the shortest path in my case.

Since dijkstra's is used on weighted graphs, I needed to create a weight map which would represent the weights of any coordinates in the graph. This is done by the GenerateWeightMap subroutine inside the Dungeon class, which works as follows:

1. Create a 2D array of integers that has the same dimensions as the dungeon
2. Loop over every cell in the tile layer
3. If the cell is null, assign a very high number to the array at that location
4. If the cell has collision, assign a very high number to the array at that location
5. If the cell contains a floor, assign 1 to the array at that location
6. If the cell contains a damage tile, assign 100 to the array at that location 7. Otherwise, assign 1 to the array at that location

A very high number indicates that the cell cannot be moved across, and the weight 100 is used for the damage tile so that the enemy tries to avoid moving onto it and taking damage.

To implement BFS, I created a queue class, and to implement Dijkstra's, I created a priority queue class that inherits from the queue class.

A pseudocode example of the implementation of BFS and Dijkstra's (separately) can be seen below.

```
FUNC BreadthFirstSearch(Dungeon dungeon, Coord destination, Coord source)
        Coord currentNode
```

```
        Tile[,] tileLayer = dungeon.GetTileLayer()
int dunHeight = dungeon.GetDunHeight()     int
dunWidth = dungeon.GetDunWidth()
        Dictionary<Coord, List<Coord>> coordAdjacencyList
= dungeon.GetCoordAdjacencyList()          bool found =
false
        List<Coord> shortestPath = new
List<Coord>()          Queue queue = new
Queue(1000)  bool[,] visited = new
bool[dunHeight,dunWidth]
        Coord[,] parent = new Coord[dunHeight,dunWidth]
        FOR i = 0 TO dunHeight - 1
                FOR j = 0 TO dunWidth - 1
                        IF tileLayer[i,j] NOT EQUAL TO null AND tileLayer[i,j].DoesCollide()
                                visited[i,j] = true
                        ENDIF
                ENDFOR
        ENDFOR
        parent[source.y,source.x] = new Coord(-1,-1)
queue.Enqueue(source)
        visited[source.y,source.x] = true
        WHILE NOT found AND NOT queue.IsEmpty()
currentNode = queue.Dequeue()
                FOREACH Coord neighbour IN coordAdjacencyList[currentNode]
                IF NOT visited[neighbour.y,neighbour.x]
                                queue.Enqueue(neighbour)
                                visited[neighbour.y,neighbour.x] = true
parent[neighbour.y,neighbour.x] = currentNode
                                IF neighbour.y EQUAL TO destination.y AND neighbour.x
EQUAL TO destination.x
                                found = true
                                ENDIF
                        ENDIF
                ENDFOREACH
ENDWHILE
        currentNode = destination
bool sourceReached = false
        IF found
                WHILE NOT sourceReached
shortestPath.Add(currentNode)
                        currentNode = parent[currentNode.y,currentNode.x]
                        IF currentNode.x EQUAL TO source.x AND currentNode.y EQUAL TO
source.y
                                sourceReached = true
                        ENDIF
                ENDWHILE
                shortestPath.Reverse()
```

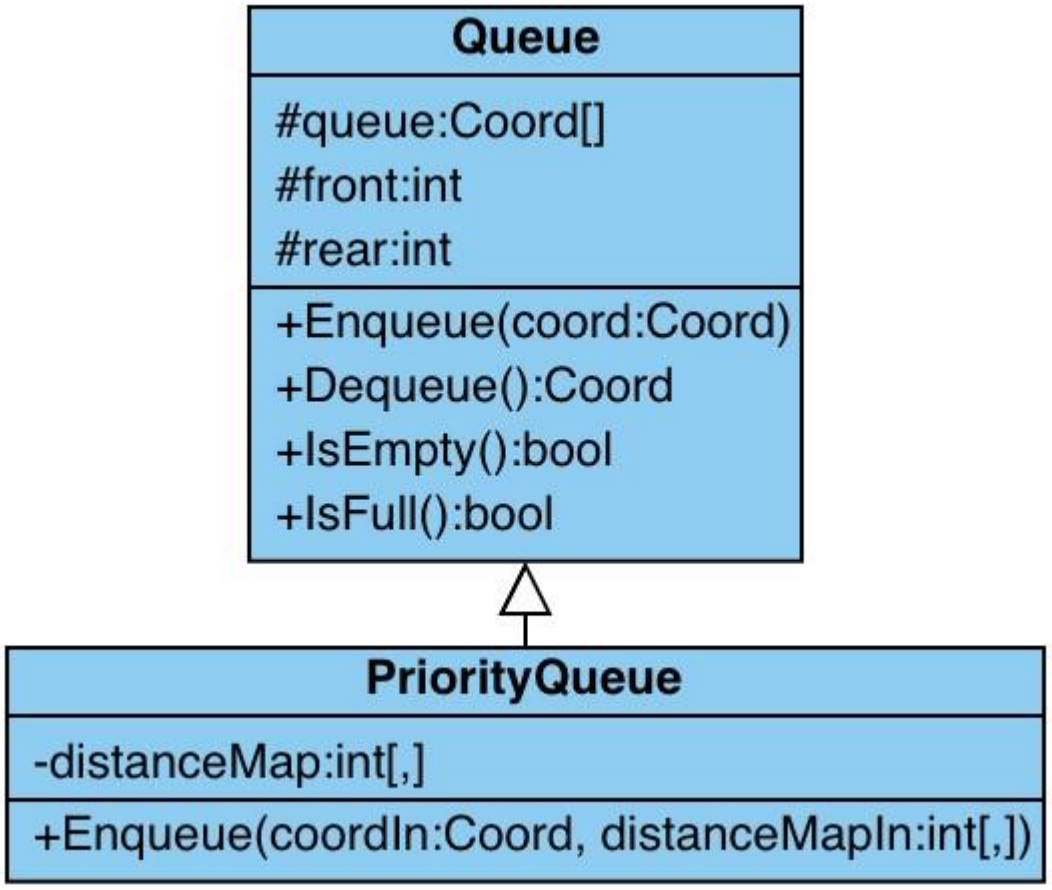
```
        return shortestPath
ENDFUNC

FUNC Dijkstras(Dungeon dungeon, Coord destination, Coord source)
        Coord currentNode
        Tile[,] tileLayer = dungeon.GetTileLayer()
int dunHeight = dungeon.GetDunHeight()     int
dunWidth = dungeon.GetDunWidth()
        Dictionary<Coord, List<Coord>> coordAdjacencyList
= dungeon.GetCoordAdjacencyList()          bool found =
false
        List<Coord> shortestPath = new List<Coord>()
PriorityQueue queue = new PriorityQueue(5000)
bool[,] visited = new bool[dunHeight,dunWidth]
Coord[,] parent = new Coord[dunHeight,dunWidth]   int[,]
weightMap = dungeon.GetWeightMap()       int[,]
distance = new int[dunHeight,dunWidth]
        FOR i = 0 TO dunHeight - 1
                FOR j = 0 TO dunWidth - 1
distance[i,j] = 1000000
                ENDFOR
        ENDFOR
        weightMap[source.y,source.x]=0
parent[source.y,source.x] = new Coord(-1,-1)
queue.Enqueue(source,distance)
visited[source.y,source.x] = true       WHILE NOT
queue.IsEmpty()                 currentNode =
queue.Dequeue()
                FOREACH Coord neighbour IN coordAdjacencyList[currentNode]
                        IF NOT visited[neighbour.y,neighbour.x]
                                IF
weightMap[neighbour.y,neighbour.x]+distance[currentNode.y,currentNode.x] LESS THAN
distance[neighbour.y,neighbour.x]
                                        distance[neighbour.y,neighbour.x] =
weightMap[neighbour.y,neighbour.x] +distance[currentNode.y,currentNode.x]
                                        parent[neighbour.y,neighbour.x]=currentNode
                                ELSE
                                        visited[neighbour.y,neighbour.x]=true
                                ENDIF
                                queue.Enqueue(neighbour,distance)
                                IF neighbour.y EQUAL TO destination.y AND neighbour.x
EQUAL TO destination.x
                                found = true
                                ENDIF
                        ENDIF
                ENDFOREACH
ENDWHILE
        currentNode = destination
```

```
        bool sourceReached = false
        IF found
                WHILE NOT sourceReached
shortestPath.Add(currentNode)
                        currentNode = parent[currentNode.y,currentNode.x]
                        IF currentNode.x EQUAL TO source.x AND currentNode.y EQUAL TO
source.y
                                sourceReached = true
                        ENDIF
                ENDWHILE
                shortestPath.Reverse()
return shortestPath
ENDFUNC
```

The 2D array of the Coord structure 'parent' is used to reconstruct the shortest path from the source to the destination. Reconstructing the path works by going back through the parent of each of the cells from the destination to the source, adding the coordinates of each cell to the shortestPath list, and then reversing the list once the source has been reached.

Below is a class diagram and definition for the queue and priority queue classes. The distanceMap variable for the priorityQueue class is used to determine the order of coordinates in the queue, where coordinates with lower weights are closer to the front of the queue and coordinates with higher weights are further to the back of the queue.



Dropping coins and keys

Whenever an enemy is killed, they will drop coins and they have a chance of dropping a key. The attributes of the Enemy class which are involved with this are minCoin, maxCoin and keyChance. minCoin and maxCoin determine the minimum and maximum number of coins that an enemy can drop when killed. keyChance is the percentage chance that an enemy will drop a key when killed. When an enemy is killed, the DropCoin subroutine is called, which adds a random number of coins between minCoin and maxCoin to the cell where the enemy is killed in the item layer. Then, the DropKey subroutine is called, which generates a random number from 0 to 99. If the number generated is smaller than the value of keyChance, then a key is added to the cell where the enemy is killed in the item layer.

Direct attacks

Direct attacks occur when an enemy attempts to move onto a tile that the player occupies. The enemy calls the Attack subroutine, which uses the player's ChangeHP subroutine and changes the player's HP to the player's current HP minused by the damage of the enemy.

Ranged attacks and validation

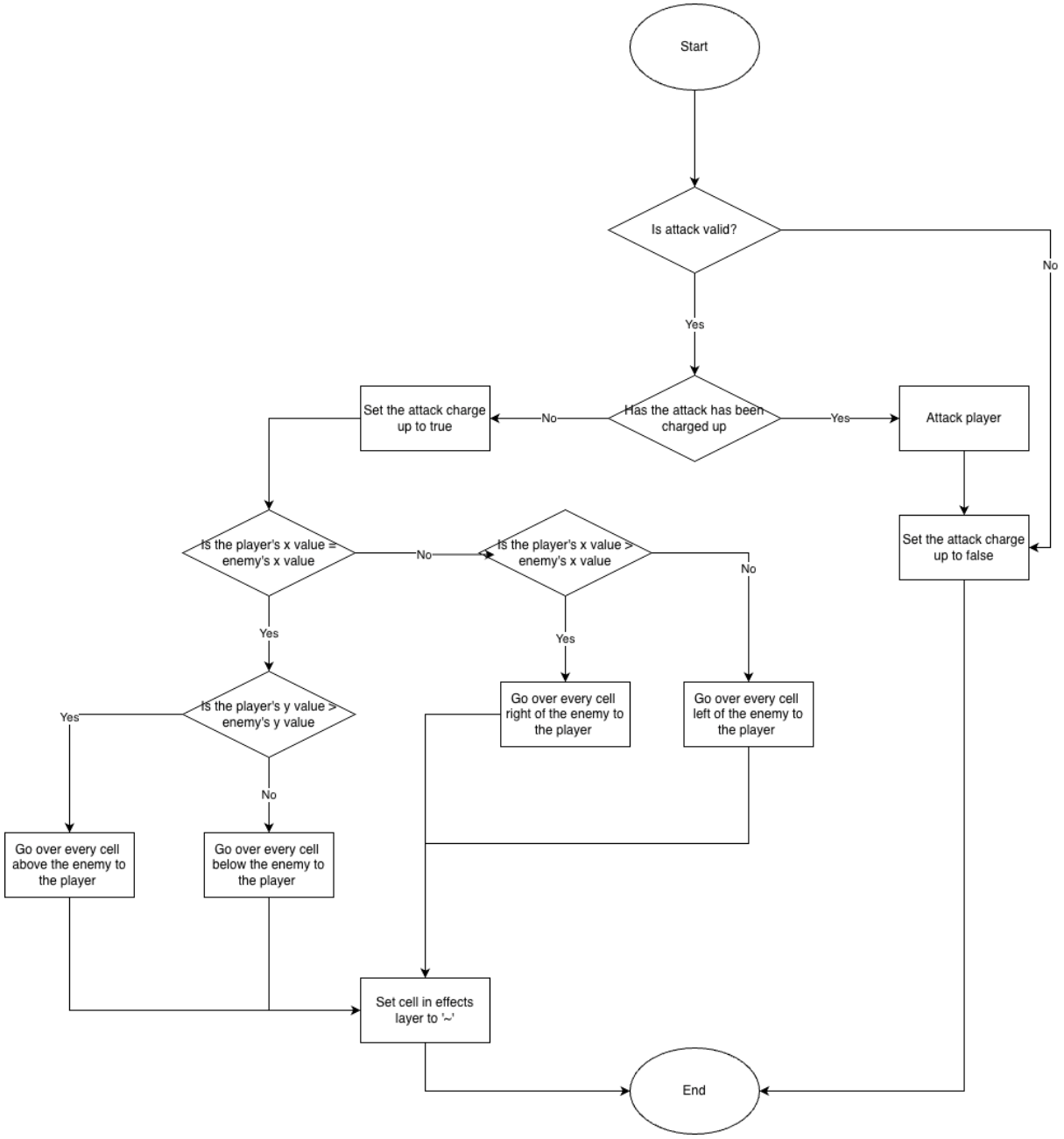
To validate a ranged attack, it is first checked whether or not the enemy and player are in the same row and not neighbouring, in the same column and not neighbouring, or neither. If it's neither, then false is returned by the IsStraightRangedAttackValid() subroutine, since the attack can't possibly be valid if the player and enemy aren't in the same column or row - the ranged attack wouldn't be straight otherwise.

If the player and enemy are in the same column (share the same x value) and aren't neighbouring (player's y value is not the enemy's y value ± 1), then check which side of the enemy the player is on (whether the player's y value is greater or less than the enemy's y value). If the player is above the enemy, then check every cell from the cell directly above the enemy to the cell where the player is, checking that every cell is valid. If any cell is invalid, then false is returned and the attack is invalid.
Otherwise, if the player is below the enemy, then do the same thing, but instead check the cells below instead of above.

Otherwise, if the player and enemy are in the same row (share the same y value) and aren't neighbouring (player's x value is not the enemy's x value ± 1), then do the same thing, but go over the cells either to the left or right of the enemy depending on which side the player is on.

The validation of each cell that is iterated over is carried out by the IsAttackValid subroutine, which applies the conditions:
IF cell is not within bounds of the dungeon OR tile at cell is null OR tile at cell has collision OR (there is an entity in the cell AND it isn't the player) THEN return
    false
OTHERWISE
    return true

Below shows a flow chart of the StraightRangedAttack subroutine.

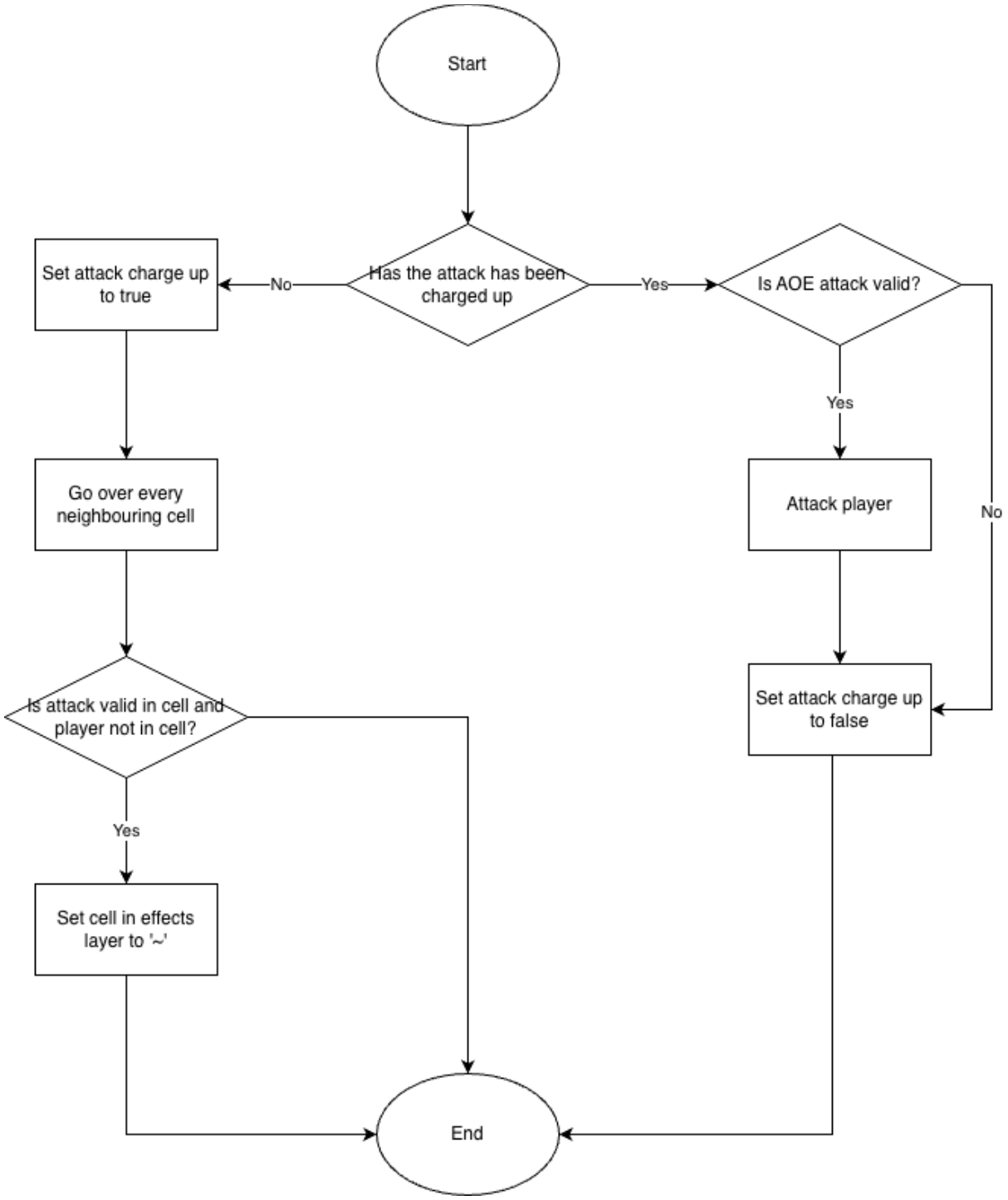


If the subroutine is called and the attack hasn't been charged up, instead of attacking, a trail of '~' will appear between the enemy and the player in order to indicate that the enemy will perform a ranged attack on the next turn.

Area of effect attacks and validation

In order to validate an area of effect attack, the cells directly bordering the enemy (this time, including cells that are diagonally neighbours) are checked for the presence of the player. If any of the cells contain the player, then the IsUniformAOEAttackValid subroutine returns true, otherwise it returns false.

Below shows a flow chart of the UniformAOEAttack subroutine.



Same with the validation, by ‘go over every neighbouring cell’ it also means to go over cells that are diagonally neighbours to the enemy.
The IsAttackValid subroutine is used for the ‘Is attack valid’ condition for figuring out where to set cells to ‘~’.

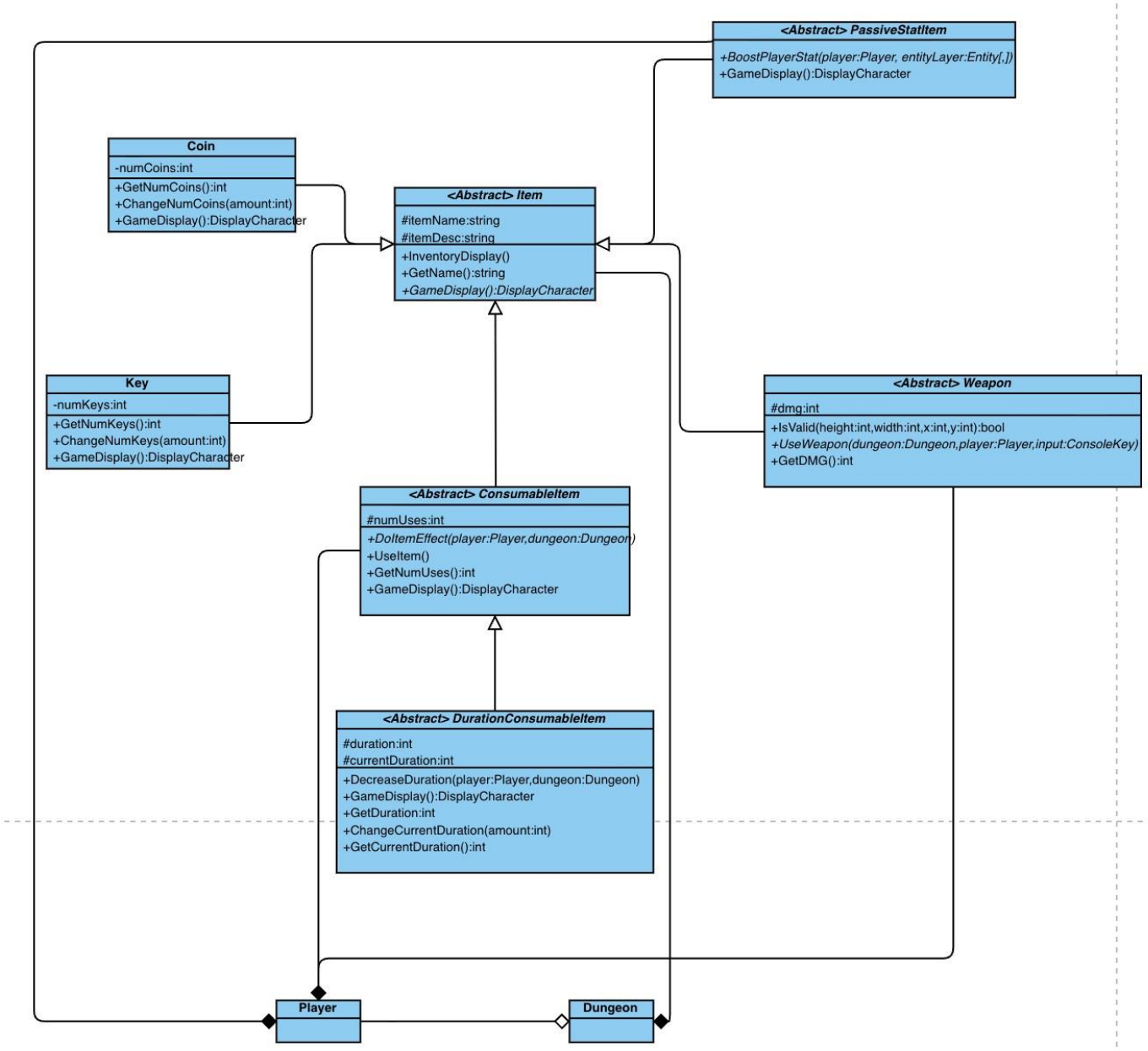
2.3.6 Items

Items can be picked up by the player, some of which are usable or give a status boost. The different types of items are:

- Coins - used to track the player’s score

- Keys - used to open chests
- Consumable items - can be used, providing the player with some sort of effect or utility
- Passive stat items - boost the player's stats

Class diagram



Dropping items from chests

When a chest is opened by the player, the DropItem subroutine is called. It works by generating a random number between 0 and 99. Then, to determine what group of items to pick from, it checks what region the number is within. The regions and their corresponding groups of items are shown below.

- From 0 to 49: common items
- From 50 to 79: uncommon items

- From 80 to 94: rare items
- From 95 to 99: epic items

After the group of items is determined, an item from the group is randomly selected and returned. As expected, the items in each group become better and more powerful for the smaller ranges of numbers, since they have a lower chance of being dropped.

Using consumable items

First, the player presses the 'E' key, indicating the player is attempting to use their consumable item.
Then, if the player has a consumable item and the consumable item is a subclass of the DurationConsumableItem class and the number of uses of the item is greater than 0 and the current duration of the item is 0, then the ChangeCurrentDuration subroutine is called for the player's consumable item, with the GetDuration subroutine of the item's class as the parameter. The DoItemEffect for the item is then called, which performs an effect unique to each item. The UseItem subroutine is then called, which decreases the usage count of the item. Then, it is checked if the item's usage count is 0 and the item's current duration is 0, if so, then the player's consumable item is changed to null in order to remove it from the player since it has been used up.
If the consumable item isn't a subclass of the DurationConsumableItem class, then the other conditions are followed as they would if it were a subclass, however it isn't checked if the current duration is 0 and the current duration isn't changed to the item's duration, since the item is a regular consumable item that doesn't have a duration.

It is checked for duration consumable items that their current duration is 0 before they are used, so that they can't be used again before the duration of a previous use has been seen through.

For duration consumable items, every turn they have their duration decreased by calling their DecreaseDuration subroutine. Within this subroutine, it is checked if their current duration is greater than 0. If it is, then its current duration is decreased by 1 and its DoItemEffect subroutine is called. This is so that items with durations have their effect carried out every turn that the item is active.

Consumable items

- Heal potions - Come in three sizes: small, medium and large. They each increase the player's current HP by 1, 2 and 3 respectively.
- Strength potion - Increase the player's damage by 1 for a certain amount of time
- Regeneration potion - Increase the player's current HP by 1 every certain number of turns
- Active/passive ability duration item - Increases the duration of the player's active/passive ability by 1
- Active/passive ability cooldown item - Decreases the cooldown of the player's active passive ability by 1

Passive items

- Full heart item - Increases the player's current and maximum HP by a certain amount

- Damage boost item - Increases the player's damage by a certain amount

Weapons

- Sword - attacks an enemy in a single neighbouring cell of a particular direction
- Longsword - attacks an enemy within 2 neighbouring cells of a particular direction
- Broadsword - attacks any enemies in a neighbouring cell of a particular direction, and in the cells either side of said cell
- Mace - attacks any enemies in neighbouring cells

When a weapon is used by pressing any of the WASD keys, the UseWeapon subroutine is called. One of the parameters of the UseWeapon subroutine is the key pressed by the player, which is then used to determine the direction of the attack. The AttackInDirection subroutine is called for each of the WASD keys, each having the same parameters passed in except from the xInc and yInc parameters, which are unique for each of the WASD keys, shown as follows:

- W: xInc = 0, yInc = -1
- A: xInc = -1, yInc = 0
- S: xInc = 0, yInc = 1
- D: xInc = 1, yInc = 0

The implementation of the UseWeapon subroutine is different for each weapon, however they each use the xInc and yInc parameters to the same effect.

For the original sword class, it is checked that the cell at the coordinates [x+xInc,y+yInc] is valid (where x and y are the coordinates of the player), and that there is an enemy at the same coordinates. If so, then the enemy is attacked.

For the longsword class, it is first checked that the cell at the coordinates [x+xInc,y+yInc] is valid and that there is an enemy at the same coordinates. If so, then the enemy is attacked. If there isn't an enemy at the same coordinates and there isn't a tile with collision or a null tile at the coordinates, then it is checked that the cell at the coordinates [x+xInc*2,y+yInc*2] is valid and that there is an enemy at the same coordinates. If so, then the enemy is attacked. It is checked that there is no collision in the first cell before attacking in the second cell so that the player cannot attack through walls.

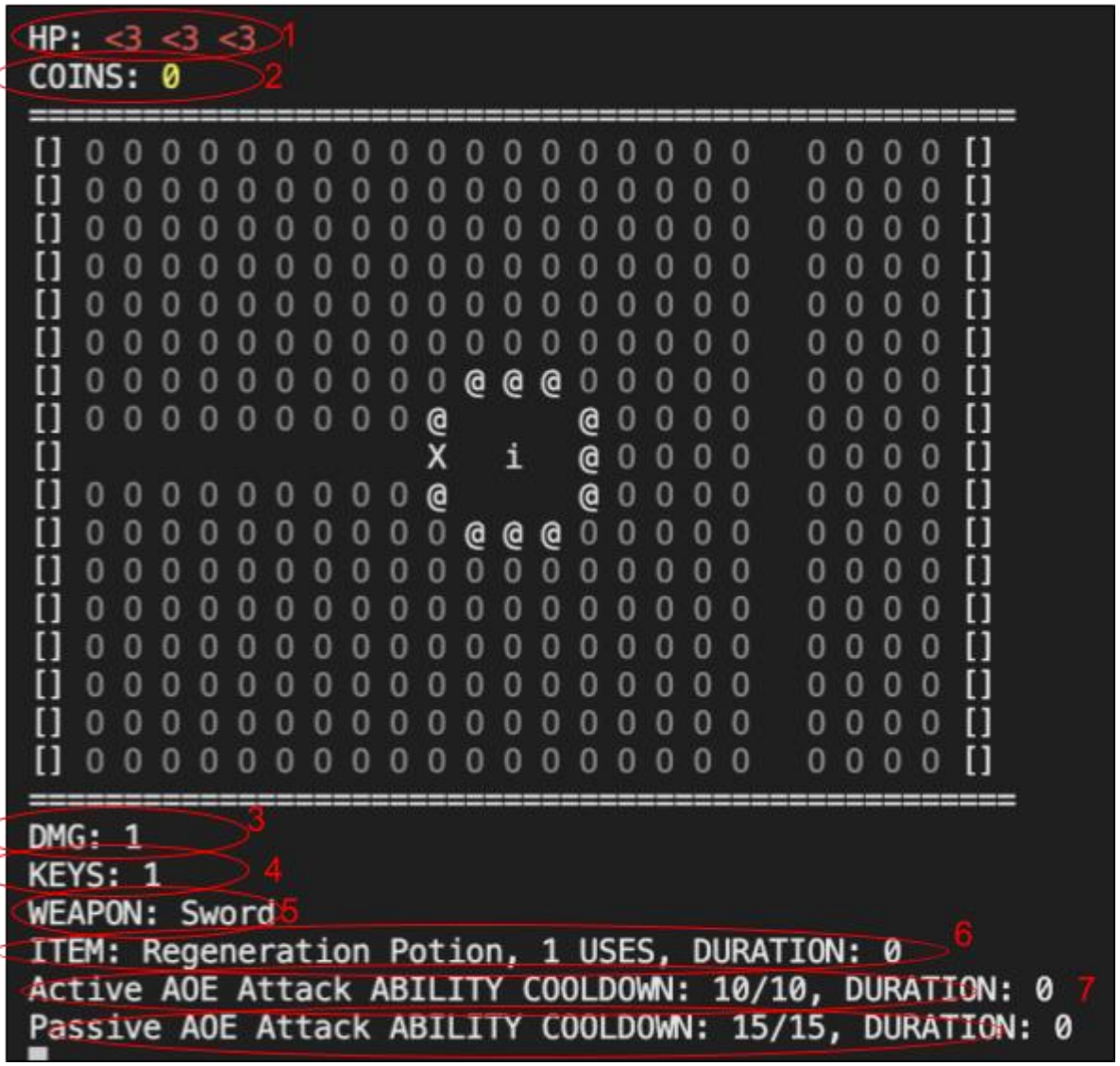
For the broadsword class, it is first checked that the cell at the coordinates [x+xInc,y+yInc] is valid and that there is an enemy at the same coordinates. If so, then the enemy is attacked. It is then checked if xInc is 0. If so, then the cells at coordinates [x-1,y+yInc] and [x+1,y+yInc] are checked to be valid and any enemies are attacked in said cells. If, instead, the yInc is 0, then the same occurs but instead for the cells at coordinates [x+xInc,y-1] and [x+xInc,y+1].

For the mace class, each of the cells directly neighbouring the player are validated and any enemies in cells are attacked (excluding cells diagonal to the player). The mace attacks are independent of direction, thus don't make use of the xInc and yInc parameters.

3 Testing

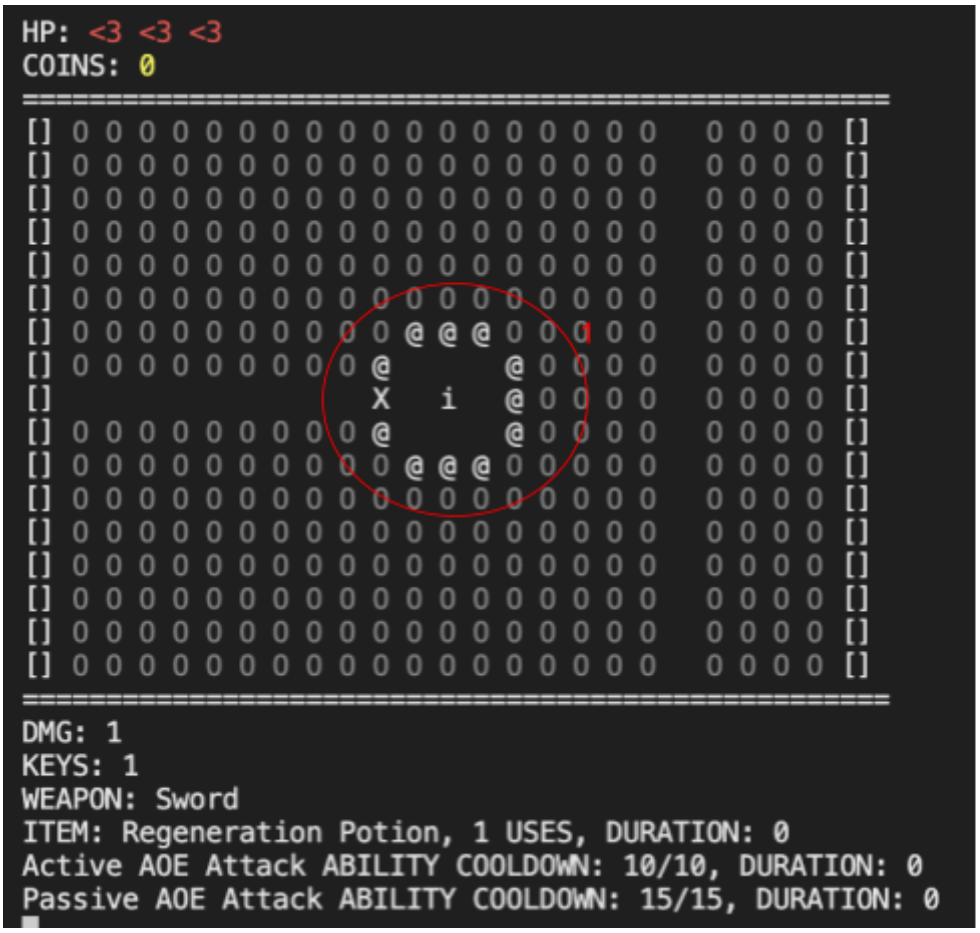
Screenshots:

A:

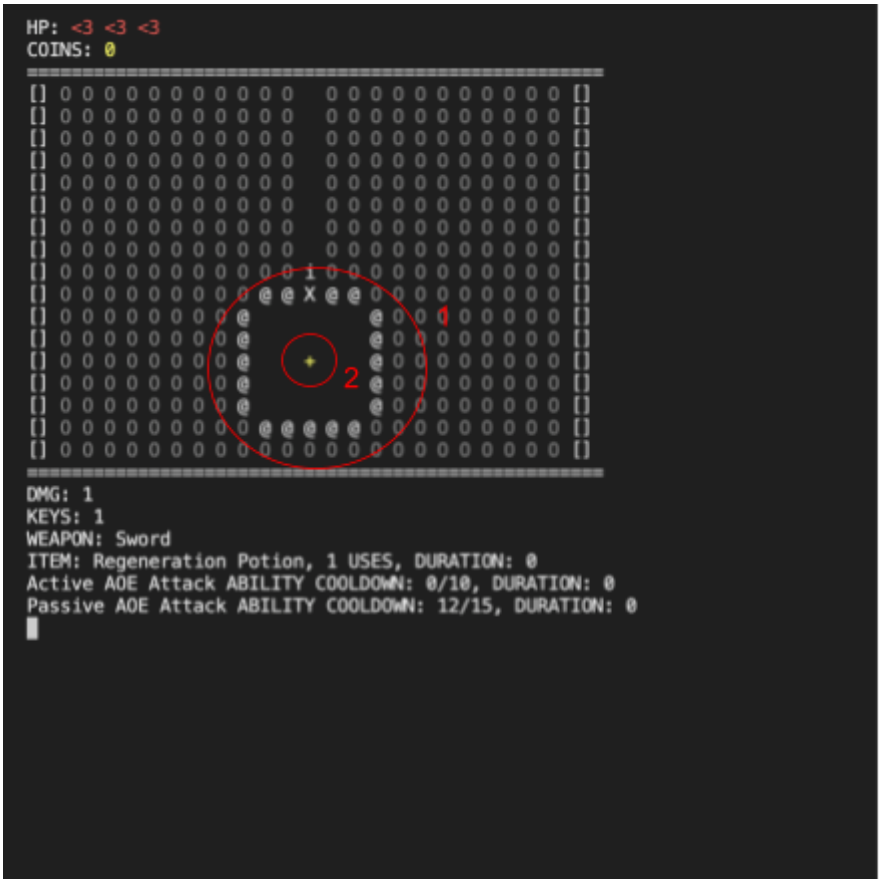


A1. Health display
A2. Money display
A3. Damage display
A4. Key display
A5. Weapon display
A6. Item display A7.
Ability display

B:



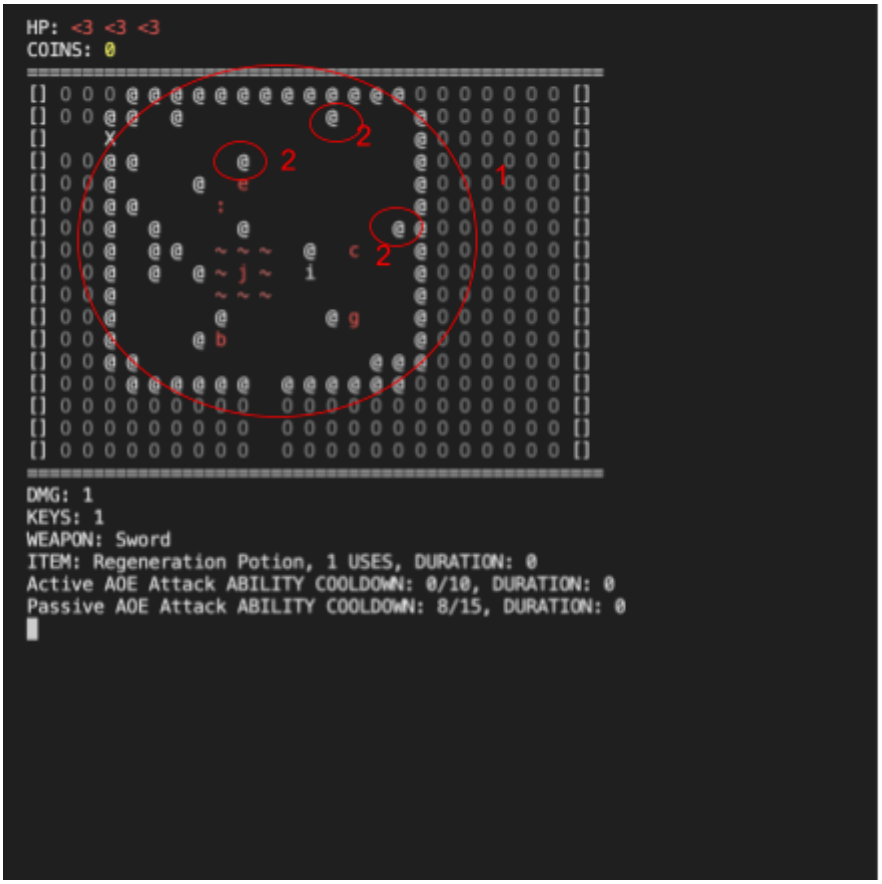
B1. Starting room C:



C1. Chest room

C2. Chest tile

D:


HP: <3 <3 <3
COINS: 0
DMG: 1
KEYS: 1
WEAPON: Sword
ITEM: Regeneration Potion, 1 USES, DURATION: 0
Active AOE Attack ABILITY COOLDOWN: 0/10, DURATION: 0
Passive AOE Attack ABILITY COOLDOWN: 8/15, DURATION: 0


HP: <3 <3 <3
COINS: 0
DMG: 1
KEYS: 1
WEAPON: Sword
ITEM: Regeneration Potion, 1 USES, DURATION: 0
Active Heal ABILITY COOLDOWN: 0/30, DURATION: 0
Passive Heal ABILITY COOLDOWN: 6/30, DURATION: 0



D1. Random room

D2. Randomly placed wall tiles inside random room

Test number	Description	Expected outcome	Pass/Fail	Objective number	Timestamp
1	Player navigates the menu using up and down arrow keys, and presses enter when on the 'Exit' option	Option selector can move up and down and the program will end	Pass	1.1.1	00:00
2	Player changes difficulty	The current difficulty will change to the difficulty that the selector is on when enter is pressed	Pass	1.1.2	00:09

3	Player starts game	The game will start	Pass	1.1.3	00:23

4	Player opens tutorial	Tutorial will be displayed when tutorial option is selected	Pass	1.1.4	00:30
5	Player exits game	Program will end	Pass	1.1.5	00:00
6	Player HUD	*Refer to objective 1.2*	Pass	A1, 1.2.1 A2,1.2.5 A3, 1.2.7 A4, 1.2.6 A5, 1.2.2 A6, 1.2.3 A7, 1.2.4	Screenshot A
7	Player moves up, down, left and right using arrow keys	Player moves up, down, left and then right	Pass	2.1.1	00:39
8	Player moves, then activates ability using q key, and then moves until ability ends	Player's ability cooldown decreases each time the player moves. When the ability activates, player damage increases by 1, and the duration changes to the ability's max duration. When the player moves, the duration decreases until it reaches 0, then the player's damage goes back to 1.	Pass	2.1.2	00:48

9	Player activates item using e key	Player's HP increases by 1 and the uses count decreases by 1 each time the player activates item	Pass	2.1.3	01:01

10	Player attacks up, down, left and then right	Enemy in each direction will die, dropping coins	Pass	2.1.4 2.2.3.4	01:07
11	Dungeon is generated multiple times	The dungeon will be different each time	Pass	2.2.0	01:17
12	Random room generated	Room will have random size and random walls placed within	Pass	2.2.1.1	Screenshots D
13	Chest room generated	Room will have constant size and chest tile in the center	Pass	2.2.1.2	Screenshot C
14	Starting room generated	Room will have constant size	Pass	2.2.1.3	Screenshot B
15	Boss room generated	Room will have constant size and have a boss in	Pass	2.2.1.4	01:59

16	Player moves around, turning tiles with collision into floors, and moving onto tiles which don’t have collision	If the player tries to move toward a wall, then it turns to a floor, if the player tries to move toward a floor, they move toward it, if the player moves toward a door, it opens	Pass	2.2.2.1 2.2.2.2	02:13
17	Enemy moves over damage tile, then player moves over damage tile	Enemy takes damage, then player takes damage	Pass	2.2.2.3	02:28
18	Player moves toward chest tile	Chest tile drops an item	Pass	2.2.2.4	02:48

19	Random enemy moves around	Random enemy moves around randomly	Pass	2.2.3.1.1 2.2.3.2	02:57
20	Tracking enemy moves around	Tracking enemy moves towards player	Pass	2.2.3.1.2 2.2.3.2	03:06
21	Static enemy doesn't move	Static enemy doesn’t move	Pass	2.2.3.1.3 2.2.3.2	03:35
22	Horizontal enemy moves, vertical enemy moves	Horizontal enemy moves only left and right, vertical enemy moves only up and down	Pass	2.2.3.1.4 2.2.3.2	03:43

23	Smart tracking enemy moves around	Smart tracking enemy moves towards player but tries to avoid damage tiles	Pass	2.2.3.1.5 2.2.3.2	03:57
24	Enemy performs a ranged attack on the player	Player will take damage	Pass	2.2.3.3.1	04:36
25	Enemy performs an attack on the player by moving directly into player	Player will take damage	Pass	2.2.3.3.2	04:50
26	Damage tile enemy moves towards player	Damage tiles will be placed in the enemy's path	Pass	2.2.3.3.3	05:05
27	Enemy performs an area of effect attack	Player will take damage	Pass	2.2.3.3.4	05:17
28	Player defeats boss and goes to the next level	The player will move onto the exit tile, and a new dungeon will be generated	Pass	2.2.4.1	05:25
29	Dungeon is generated 2 times, once at a low difficulty and once at a high difficulty	Dungeon will become bigger, room sizes will increase, number of enemies will increase, the number of types of enemies will increase	Pass	2.2.4.2	05:48

30	Player attacks enemies using sword, then longsword, then broadsword and then mace	Enemies die	Pass	2.3.1	06:31
31	Player picks up longsword	Player drops sword	Pass	2.3.1.1	07:21
32	Player uses healing potion	Player's HP increases by 1 after each use, uses count decreases	Pass	2.3.2 2.3.2.2	01:01
33	Player uses strength potion	Player's DMG increases by 1, and then decreases by 1 when duration ends	Pass	2.3.2 2.3.2.3	07:35
34	Player picks up strength potion	Player drops healing potion	Pass	2.3.2.1	07:45
35	Player picks up passive HP boost item	Player's HP and max HP increase by 1	Pass	2.3.3	08:01
36	Player picks up passive DMG boost item	Player's DMG increases by 1	Pass	2.3.3 2.3.3.1	08:01

4 Evaluation

4.1 Overall effectiveness of system

My program was created to solve the problems behind beginners shying away from the roguelike genre due to how complex, unaffordable and intimidating new roguelike games are. I have made my program simple, free and full of the features that you'd find in most roguelikes in order to provide the user with a taste for the genre that isn't overwhelming and

without needing to make a financial investment. I have added a wide array of enemies, to provide the player with different attack patterns to memorise and adapt to, a wide array of items, to provide the player with different opportunities to defeat enemies and different abilities to give the player different playstyles. I have also implemented a procedurally generated dungeon system, and this alongside the wide variety of items and enemies at the player's disposal provides a replayable experience. The use of options to change difficulty makes the game more friendly to beginners, since they can make the game easier if they so choose, and it also makes it friendly to more experienced players, since they could do the opposite.

4.2 Evaluation of objectives

1.1) I was able to implement all of the menu features I set out to create - the options to start the game, change the difficulty, read a tutorial and exit the game. The tutorial gives a quick overview of the main aspects of the game, allowing the player to be brought up to speed swiftly. The menu is also navigated by using the up and down arrow keys, and choices are selected by using the enter key. To improve upon what I've done, the menu could be made to look more appealing, since it is very simple and purely functional.

1.2) I successfully added a health bar to the top of the screen, which uses red hearts to indicate the player's current HP and dark gray hearts to indicate HP that the player has lost. It still works for a very large HP, displaying a lot of hearts at the top of the display, but this could be inconvenient since it might block the view of the dungeon display (though it is very improbable that the player would get this many hearts).
The weapon, item, ability, coin and key displays were all implemented as planned, and clearly display what the player has. A small issue is that they are all displayed in the same colour, font, and are all displayed in one clump at the bottom, which can make it difficult to initially distinguish between them. To improve, they could have unique displays, possibly with different colours and formatting.

2.1) All of the controls I planned to add were implemented, and they are all shown in the tutorial section of the main menu.

2.2.0) I feel like the dungeon generation algorithm I have used creates nicely randomly laid out dungeons where each floor feels somewhat unique but somewhat similar.

2.2.1) Each of the rooms I set out to add have been successfully added. I hoped to add more room types, however I didn't get round to this. Due to the class system I created with the Room class and Tile class, alongside the subroutines to create random dungeon floors within the Dungeon class, I would easily be able to add different types of rooms or tiles if I had more time.

2.2.2) I also successfully added all of the tiles in my objectives. Initially, the player wasn't able to move through any tiles with collision, but then I made it so that walls could be interacted with to be turned into floors to give the player more freedom with their movement. Doors and chests remain the same tile when interacted with - their collision is merely switched to false. I decided to build on top of the damage tile idea - I added a boolean variable to the damage tile class called IsEnemyDamageTile, and if it is set to be true, then it

will deal damage to all entities, and if it is false then it only deals damage to enemies. This allows the player to use their own damage tiles to their advantage.

2.2.3) I felt I did a good job with the enemies in my game - I created many different types of enemies, which each had unique behaviour, either through their movement or how they attacked. They each have a unique letter associated with them, which allows the player to differentiate between each type of enemy and react accordingly. The class system I created for enemies would allow me to easily add any new enemies with different combinations of existing behaviours if I had more time, or add new types of enemies with new unique features. I didn't initially plan to add effects for ranged or area of effect attacks, but I realised that it would make attacks very unpredictable for the player. I feel as if the boss is a bit underwhelming - a bit too easy to defeat. If I had more time, I would hope to add a higher skill roof to the boss by adding different abilities for it to use rather than just increase its HP and damage. I successfully added all of the objectives for enemies.

2.2.4) Defeating the boss is compulsory to reaching the next level. Difficulty increases as the player progresses through levels by increasing the number of types of enemies, the HP and damage of enemies, the size of rooms (and hence the number of enemies in each room) and the number of rooms. I felt somewhat satisfied with the levelling in my game. Difficulty does increase as the game progresses. However, I feel like most of the difficulty in the game comes from each dungeon floor rather than the boss, which seems a little counterintuitive. As I mentioned previously, I would hope to improve the boss so that it was more difficult, and its difficulty would scale better as progression occurs.

2.3) I was able to add different types of items to the game - I am satisfied with this. For the time I was given, I'm happy with how many items I added that the player can use. All of the objectives with item functionality were successful: only one consumable item and weapon can be held at once, many passive items can be held at once, some consumable items have durations and consumable items have a usage count which decreases after each use. If I had more time, I definitely would want to add more items to the game - for a roguelike, my game doesn't have enough. The use of my item layer being a 2D array of lists allowed many items to be in a single cell, which I didn't initially account for - I only planned to have a single item held per cell, but realized that this would lead to problems if an enemy died and tried to drop coins on a cell where there was already an item.

4.3 End user feedback

I handed my brother my laptop with my program open on it and allowed him to play through the game, pointing out features that he might've missed. I asked him to pay extra attention to how easy the game was to navigate and get a handle on, since the main purpose of my game was to be beginner friendly. I finally asked him to give me some feedback, which is shown below.

"The first impression I got of the game was that it was simple - this was through the display. Using letters for the display made each piece of the game more distinct and distinguishable from the other, since certain combinations of letters and colours were used to represent certain things, like how red letters are enemies, yellow dollar signs are coins and white @s are walls. It made it much easier to understand what was what. However, after a while, the game display gets a bit boring due to its minimalist

nature. It makes the game feel less gripping. I found it harder to stay interested because the display wasn't very interesting. "

I feel satisfied with this feedback. I intended to make the display very simple, which was accommodating of the time I was given and aligned with the problem I was trying to solve - the intimidation caused by the complexity of modern roguelikes. I expected the upside of this to be an easier-to-understand and more digestible experience, which is evident from this feedback, and the downside to be a lack of engagement. If I had more time, then I could still stick to the simple aesthetic yet add more depth to the display to make it more interesting and engrossing for the player.

"I liked that the dungeon was randomly generated. It makes the experience change each time you play, and the wide variety of weapons, items and abilities encourage you to play in different ways. I thought there could be more abilities, and maybe to have the option to select abilities instead of having them randomly given to you. Also, I thought that the passive and active heal abilities were a bit overpowered, since it's quite hard to come by ways to heal. I like the different weapons that you can use, and each one is different enough from the others to make you interact with enemies differently, which is fun. What I think could be cool is if there were ranged weapons though in order to provide more variety. "

I'm happy that it makes the user feel like each run is different, since replayability is a massive part of roguelike games. I hoped adding variety to each of my features would provide the user with this experience, and it seems to have succeeded to a certain extent. As expected, there could always be more variety, especially since I wasn't able to add a great lot due to time constraints. I could've made healing a bit more balanced by either adding more ways to receive healing, or by increasing the cooldown of the healing abilities to nerf them. Adding ranged weapons is a good idea, and I definitely considered it. I just ran out of time, and I didn't think it would add much more value to my program initially.

4.4 System improvements

My program successfully completed all of the objectives I set. However, in order for it to provide a more thorough and deep taste of the roguelike genre, I would need to increase the variety of my game - since roguelikes are all about variety. I implemented all of the core features of roguelikes, I would just need to build upon these features to provide a more well rounded experience, since the game feels a bit flat as is. Due to the way I have made my program through using OOP, and as a consequence of creating the classes for all of the core features of a roguelike, it wouldn't be a significant feat to add more items to the item class, add new types of enemy to the enemy class, add new tiles to the tile class, etc.

As mentioned in the end user feedback, a lot more could be done with the way my game is displayed. I was happy that it made the game easier to understand since I wanted it to be beginner friendly, but it made it less sustainable for longer playing sessions. Instead of having my game in the console, to improve I could've possibly used a game engine like unity to make use of sprites and better UI. This would make the game more visually appealing and prevent the player from becoming disinterested. I seemed to push too far left toward the

simple end of the visually complex spectrum, and I could've instead met more in the middle. I chose to prioritise the algorithms behind the game, such as the random dungeon generation and the tracking for the enemies rather than the display.

Balancing is a bit of an issue. As my brother mentioned, some abilities are much more favourable than others - the healing abilities are overpowered. This could be solved by increasing the cooldown of such abilities. I found that the strength potion was underpowered, so it could have its potency increased or duration increased.

Difficulty could be improved in some areas - as I mentioned before, the boss can be 'cheesed', since it's easy to avoid attacks when it only has a few attacks and you can easily predict what it's going to do. The boss could have more abilities added, such as summoning enemies in the boss room or having a wider random area of effect attack (which I planned to implement, which is why there is already a UNIFORM area of effect attack).

I think providing information through the display could be better, like having an inventory system to show what passive stat items the player is currently holding (since they could hold multiple) and to give descriptions of items that the player is holding. I think it would be helpful to know the health and damage of enemies explicitly, without needing to kill them or take damage from them to figure out each respectively. It's happened where I would be playing and then be shocked when I'd get obliterated in one shot by a ranged enemy across a room, expecting for it to only take a portion of my health.

In conclusion, my program is a successful solution for my problem - though it is a bit bare-bones. With a bit of expanding and polishing, it would be greatly improved, and would represent the roguelike genre much better.

5 Technical Solution

Directory of technical skills

Code section	Description	Page(s)
Constructor in Dungeon class	A high level overview of creating the dungeon and the generate tile layer subroutine	72-73
DisplayDungeon subroutine in Dungeon class	Creating the dungeon display for the player	79-81
AddRooms subroutine in Dungeon class	Shows the algorithm to add rooms to the dungeon	81-82
Corridor subroutines (Adding and validation) in Dungeon class	Shows the algorithms to add corridors to the dungeon	83-87

Adding enemy subroutines in Dungeon class	Shows the algorithms to add enemies to the dungeon	88-93
Tile class	Inheritance, subtype polymorphism, use of abstract classes, use of access modifiers	103-107
Dijkstras method in LivingEntityClass	Implements Dijkstra's algorithm	109-110
Breadth first search method in LivingEntityClass	Implements breadth first search algorithm	111-112
Enemies classes	Inheritance, subtype polymorphism, use of abstract classes, use of access modifiers, shows the algorithms behind making ranged attacks and AOE attacks, shows the algorithms behind enemy movement	121-156
Item classes	Inheritance, subtype polymorphism, use of abstract classes, use of access modifiers and shows the algorithms behind using items, decreasing their usage counters, activating their effects and decreasing their durations	157-164
Queue and priority queue classes	Implements the queue and priority queue data structures, also uses inheritance	165-166
Weapon classes	Shows the algorithms behind attacking and attacking validation	167-173
Ability classes	Shows the algorithms behind activating abilities and decreasing their cooldowns and duration	174-185

Code

```
namespace goodbyeworld

{

   internal class Program

   {


       public static Random rand = new Random();

       static void Main(string[] args)

       {

           Menu();


       }

       static bool GetMenuChoice(ref int pointer,int max)

       {
```

```
            //Allow user to interact with the menu, moving pointer if up/down arrow
keys are used or
            //running the appropriate process if the user presses the enter key by
returning true to indicate
            //a process must be carried out
            ConsoleKey menuInput = Console.ReadKey().Key;
            if (menuInput == ConsoleKey.UpArrow && pointer > 0)
            {
                pointer--;
            }
            else if (menuInput == ConsoleKey.DownArrow && pointer < max)
            {
                pointer++;
            }
            else if (menuInput == ConsoleKey.Enter)
            {
                return true;
            }
            return false;

        }
        static void Menu()
        {
            //Displays the menu to the user
            double difficulty = 1.5;
            int pointer = 0;
            bool exit = false;
```

```
            string[] choices = { "Start Game", "Change Difficulty", "Tutorial",
"Exit Game" };
            while (!exit)
            {
                Console.Clear();
                System.Console.WriteLine("ROGUELIKE GAME");
                for (int i = 0; i < 4; i++)
                {
                    if (i == pointer)
                    {
                        System.Console.WriteLine(">" + choices[i] + "<");
```

```
                }
                else
                {
                    System.Console.WriteLine(choices[i]);
                }
            }
            if (GetMenuChoice(ref pointer,choices.Count()-1))
            {
                switch (pointer)
                {
                    case 0:
                        PlayGame(difficulty);
                        break;
                    case 1:
                        ChangeDifficulty(ref difficulty);
                        break;
                    case 2:
                        Tutorial();
                        break;
                    case 3:
                        Console.Clear();
                        exit=true;
                        break;
                    default:
                        break;
                }
            }
        }

    }
    static void PlayGame(double difficultyIn)
    {
        double difficulty = difficultyIn;
```

```
        Player player = new Player(-1, -1, 3,0);
        Dungeon dungeon = new Dungeon(ref player, difficulty);
        bool dungeonDefeated = true;
        int level = 1;
        //keep generating dungeons if the player manages to exit without dying
```

```
            while (dungeonDefeated)
            {
                //player attempts to exit a dungeon
                dungeonDefeated = dungeon.Play();
                if (dungeonDefeated) //if player manages to exit dungeon, then
generate a boss room
                {
                    dungeon = new Dungeon(ref player, difficulty);
                    dungeon.GenerateBossRoom();
                    dungeonDefeated = dungeon.Play();
                    if (dungeonDefeated) //if player manages to exit boss room, then
increase difficulty and generate a new dungeon
                    {
                        difficulty = difficulty * 1.25;
                        dungeon = new Dungeon(ref player, difficulty);
                        level++;
                    }
                }
            }

            Console.Clear();
            System.Console.WriteLine($"YOU DIED... YOU REACHED LEVEL {level}");
            System.Console.WriteLine($"You got a score of {player.GetNumCoins()}");
            System.Console.WriteLine("(Press ANY KEY to EXIT)");
            Console.ReadKey();
            Console.Clear();
        }
        static void ChangeDifficulty(ref double difficulty)
        {
            int pointer = 0;
            string[] choices = { "Easy", "Normal", "Hard", "Impossible","EXIT" };
            bool exit = false;
            string currentDifficulty = "Normal";

            while (!exit)
            {
                Console.Clear();
                System.Console.WriteLine("CHOOSE DIFFICULTY");
```

```
                System.Console.WriteLine($"Current difficulty:
{currentDifficulty}");
                for (int i = 0; i < 5; i++)
                {
                    if (i == pointer)
                    {
                        System.Console.WriteLine(">" + choices[i] + "<");
                    }
                    else
                    {
                        System.Console.WriteLine(choices[i]);
                    }
                }
                if (GetMenuChoice(ref pointer,choices.Count()-1))
                {
                    switch (pointer)
                    {
                        case 0:
                            difficulty = 1;
                            currentDifficulty = "Easy";
                            break;
                        case 1:
                            difficulty = 1.5;
                            currentDifficulty = "Normal";
                            break;
                        case 2:
                            difficulty = 2.5;
                            currentDifficulty = "Hard";
                            break;
                        case 3:
                            difficulty = 4;
                            currentDifficulty = "Impossible";
                            break;
                        case 4:
                            exit=true;
                            break;
                        default:
                            break;
                    }
```

```
            }
        }
    }
    static void Tutorial()
    {
        Console.Clear();
        System.Console.WriteLine("Welcome to my roguelike game!\n\n");
        System.Console.WriteLine("Controls:");
        System.Console.WriteLine("Move: ARROW KEYS");
        System.Console.WriteLine("Attack: WASD");
        System.Console.WriteLine("Use item: E");
        System.Console.WriteLine("Use ability: Q\n\n");

        System.Console.Write($"ENEMIES are lowercase letters, e.g 'a', 'b' or 'c'");
        System.Console.WriteLine("Each TYPE of enemy has its own DISTINCT way of MOVING around, figuring out each-\n-of their movement patterns is essential to defeating them\n\n");
        System.Console.WriteLine("Enemies drop $ and have a chance of dropping a key");

        System.Console.WriteLine("CHESTS are represented by '+'");
        System.Console.WriteLine("They are opened by moving toward them");
        System.Console.WriteLine("You must have a key to open a chest");
        System.Console.WriteLine("They will drop an item\n\n");

        System.Console.WriteLine("To exit a dungeon, you must find the EXIT TILE");
        System.Console.WriteLine("The exit tile is a ':', but enemies can stand over the tile which-\n-might hide it\n\n");

        System.Console.WriteLine("You are able to BREAK WALLS by moving toward them\n\n");

        System.Console.WriteLine("Try to get as many $ as you can before you meet your inevitable demise!");
        System.Console.WriteLine("GOOD LUCK!\n\n");
```

```
            System.Console.WriteLine("(Press ANY KEY to EXIT)");
            Console.ReadKey();




        }

    }
}
```

```
namespace goodbyeworld
{
    public struct DisplayCharacter(string characterIn,ConsoleColor colourIn)
    {
        public string character = characterIn;
        public ConsoleColor colour = colourIn;
    }
    public class Dungeon
    {
        private static Random rand = new Random();
        private Dictionary<Coord, List<Coord>> coordAdjacencyList;
        private List<Room> rooms;
        private List<Enemy> enemies;
        private int dunHeight;
        private int dunWidth;
        private int numChestRooms;

        private int maxNumRandomRooms;
        private int minRandomRoomHeight;
        private int maxRandomRoomHeight;
        private int minRandomRoomWidth;
        private int maxRandomRoomWidth;
        private Tile[,] tileLayer;
        private Entity[,] entityLayer;
        private List<Item>[,] itemLayer;
        private DisplayCharacter[,] effectsLayer;
```

```
        private int[,] weightMap;
        private Player player;
        private double difficulty;
        private const int cameraHeight =17;
        private const int cameraWidth = 23;
        private const int startingRoomHeight = 5;
        private const int startingRoomWidth = 5;
        private bool exitedDungeon = false;
        private Boss boss;

        public Dungeon(ref Player playerIn, double difficultyIn)
        {
            player = playerIn;
            difficulty = difficultyIn;
            rooms = new List<Room>();
            enemies = new List<Enemy>();
            coordAdjacencyList = new Dictionary<Coord, List<Coord>>();
```

```
            //calculate the width and height of the dungeon
            dunWidth = (int)(45 * difficulty);
            if (dunWidth > 150)
            {
                dunWidth = 150;
            }
            dunHeight = (int)(45 * difficulty);
            if (dunHeight > 150)
            {
                dunHeight = 150;
            }


            tileLayer = new Tile[dunHeight, dunWidth];
            entityLayer = new Entity[dunHeight, dunWidth];
            effectsLayer = new DisplayCharacter[dunHeight, dunWidth];
            itemLayer = new List<Item>[dunHeight, dunWidth];
            GenerateTileLayer(); //add rooms and corridors to dungeon
            AddEnemies(); //add enemies to dungeon
```

```
            //add player to dungeon
            player.x = rooms[0].x + rooms[0].width / 2;
            player.y = rooms[0].y + rooms[0].height / 2;
            entityLayer[player.y, player.x] = player;
        }
        public void GenerateTileLayer()
        {
            int maxAttempts = 100;
            int numAttempts = 0;
            bool roomsValid = false;
            bool corridorsValid = false;
            numChestRooms = 2;
            //calculate the number of random rooms
            maxNumRandomRooms = Convert.ToInt32(3 * difficulty);
            if (maxNumRandomRooms > 8)
            {
                maxNumRandomRooms = 8;
            }
            //calculate the minimum and maximum height and width of random rooms
            minRandomRoomHeight = Convert.ToInt32(4 * (difficulty + 1) / 2);
            if (minRandomRoomHeight > 7)
            {
                minRandomRoomHeight = 7;
```

```
            }
            maxRandomRoomHeight = Convert.ToInt32(7 * (difficulty + 1) / 2);
            if (maxRandomRoomHeight > 15)
            {
                maxRandomRoomHeight = 15;
            }
            minRandomRoomWidth = Convert.ToInt32(4 * (difficulty + 1) / 2);
            if (minRandomRoomWidth > 7)
            {
                minRandomRoomWidth = 7;
            }
            maxRandomRoomWidth = Convert.ToInt32(7 * (difficulty + 1) / 2);
            if (maxRandomRoomWidth > 15)
            {
                maxRandomRoomWidth = 15;
```

```
            }
            //Attempt to add rooms and corridors to dungeon
            while ((!roomsValid || !corridorsValid) && numAttempts < maxAttempts)
//repeat until dungeon is generated or max attempts is reached
            {
                rooms.Clear();
                tileLayer = new Tile[dunHeight, dunWidth];
                AddDungeonBorder();
                roomsValid = AddRooms();
                corridorsValid = AddCorridors();
                numAttempts++;
            }
            if (numAttempts == maxAttempts) //if max attempts is reached, dungeon
hasn't generated so throw an exception
            {
                throw new Exception("Dungeon failed to generate");
            }
        }
        public void AddDungeonBorder() //adds a border to the dungeon which the
player cannot break
        {
            for(int i = 0; i < dunHeight; i++)
            {
                for(int j = 0; j < dunWidth; j++)
                {
                    if (i == 0 || i == dunHeight - 1 || j == 0 || j == dunWidth - 1)
                    {
                        tileLayer[i, j] = new DungeonBorder();
                    }
```

```
                }
            }
        }
        public void ExitDungeon(){
            exitedDungeon=true;
        }
        public bool Play()
        {
```

```
            while (player.IsAlive() && !exitedDungeon) //while the player is alive
and the player hasnt exited the dungeon, repeat
            {
                Console.Clear();

                TopDisplay();
                DisplayDungeon();
                BottomDisplay();

                effectsLayer = new DisplayCharacter[dunHeight, dunWidth];

                PlayerMove();

                EnemiesMove();

                coordAdjacencyList.Clear();
            }
            if (exitedDungeon)
            {
                return true;
            }
            else
            {
                Console.Clear();
                TopDisplay();
                Console.BackgroundColor=ConsoleColor.DarkRed;
                DisplayDungeon();
                Console.ResetColor();
                BottomDisplay();
                Thread.Sleep(3000);
            }
            return false;
        }
        public void PlayerMove()
        {
```

```
            player.Input(this); //allow player to make a move
            player.DecreasePlayerCooldownsAndDurations(this);
            if (tileLayer[player.y, player.x].GetType() ==
```

```
typeof(DamageTile)&&((DamageTile)tileLayer[player.y,
player.x]).IsEnemyDamageTile())
            {
                ((DamageTile)tileLayer[player.y, player.x]).DoDamage(player,
entityLayer);
                effectsLayer[player.y,player.x]=new
DisplayCharacter("*",ConsoleColor.DarkRed);
            }
        }
        public void EnemiesMove()
        {
            List<Enemy> temp = new List<Enemy>();
            MazeToAdjacencyList();
            GenerateWeightMap();
            foreach (Enemy enemy in enemies) //make all enemies in dungeon move
            {
                enemy.Move(); //enemy moves
                //if the enemy moves onto a damage tile
                if (tileLayer[enemy.y, enemy.x].GetType() == typeof(DamageTile))
                {
                    //if the enemy has damage tile immunity and the damage tile is
from the player or if the enemy
                    //doesnt have immunity, then do damage to the enemy

if((enemy.HasDamageTileImmunity()&&!((DamageTile)tileLayer[enemy.y,
enemy.x]).IsEnemyDamageTile())||(!enemy.HasDamageTileImmunity()))
                    {
                        ((DamageTile)tileLayer[enemy.y, enemy.x]).DoDamage(enemy,
entityLayer);
                        if(((DamageTile)tileLayer[enemy.y,
enemy.x]).IsEnemyDamageTile())
                        {
                            effectsLayer[enemy.y,enemy.x]=new
DisplayCharacter("*",ConsoleColor.DarkRed);
                        }
                        else
                        {
                            effectsLayer[enemy.y,enemy.x]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
```

```
                    }
                }
```

```
            }


            if (enemy.GetHP() > 0)
            {
                temp.Add(enemy);
            }
        }
        //clear enemies
        enemies.Clear();
        //add all enemies from temp list(enemies that are alive) to enemies list
        foreach(Enemy enemy in temp)
        {
            enemies.Add(enemy);
        }



    }
    public void TopDisplay()
    {
        Console.ForegroundColor = ConsoleColor.Gray;

        //display red hearts for hp the player currently has
        Console.Write("HP: ");
        Console.ForegroundColor = ConsoleColor.Red;
        for (int i = 0; i < player.GetHP(); i++)
        {
            Console.Write("<3 ");
        }
        //display dark grey hearts for hp the player has lost
        Console.ForegroundColor = ConsoleColor.DarkGray;
        for (int i = 0; i < player.GetMaxHP() - player.GetHP(); i++)
        {
            Console.Write("<3 ");
        }
        System.Console.WriteLine();
```

```
            Console.ForegroundColor = ConsoleColor.Gray;
            Console.Write("COINS: ");
            Console.ForegroundColor = ConsoleColor.Yellow;
            Console.Write(player.GetNumCoins());
            System.Console.WriteLine();

            Console.ForegroundColor = ConsoleColor.Gray;
```

```
        }
        public void BottomDisplay()
        {
            System.Console.WriteLine($"DMG:
{player.GetDMG()+player.GetWeapon().GetDMG()}");
            System.Console.WriteLine($"KEYS: {player.GetNumKeys()}");
            System.Console.WriteLine($"WEAPON: {player.GetWeapon().GetName()}");
            if (player.HasConsumableItem())
            {
                System.Console.Write($"ITEM: {player.GetConsumableItem().GetName()},
{player.GetConsumableItem().GetNumUses()} USES");
                if
(player.GetConsumableItem().GetType().IsSubclassOf(typeof(DurationConsumableItem)))
                {
                    DurationConsumableItem item =
(DurationConsumableItem)player.GetConsumableItem();
                    System.Console.Write($", DURATION:
{item.GetCurrentDuration()}\n");
                }
                else
                {
                    System.Console.WriteLine();
                }
            }
            else
            {
                System.Console.WriteLine("ITEM: NONE");
            }
            if (player.HasActiveAbility())
            {
```

```
                player.GetActiveAbility().Display();
            }
            if (player.HasPassiveAbility())
            {
                player.GetPassiveAbility().Display();
            }
        }
        public void GenerateBossRoom()
        {
            dunHeight = 99;
            dunWidth = 99;
            tileLayer = new Tile[dunHeight, dunWidth];
            entityLayer = new Entity[dunHeight, dunWidth];
            itemLayer = new List<Item>[dunHeight, dunWidth];
```

```
            effectsLayer = new DisplayCharacter[dunHeight, dunWidth];
            enemies = new List<Enemy>();
            rooms.Clear();

            int bossRoomWidth = 29;
            int bossRoomHeight = 29;
            int bossRoomX = ((dunWidth-1)/2)-((bossRoomWidth-1)/2);
            int bossRoomY = ((dunHeight - 1) / 2) - ((bossRoomHeight - 1) / 2);

            AddDungeonBorder();

            Room startingRoom = new Room(dunWidth / 2 - 5 / 2, dunHeight - 5 - 10,
startingRoomHeight, startingRoomWidth);
            BossRoom bossRoom = new
BossRoom(bossRoomX,bossRoomY,bossRoomHeight,bossRoomWidth,difficulty);
            bossRoom.AddRoom(tileLayer);
            startingRoom.AddRoom(tileLayer);
            rooms.Add(startingRoom);
            rooms.Add(bossRoom);

            AddVerticalCorridor(startingRoom.y, bossRoomY + bossRoomHeight - 1,
bossRoomX + ((bossRoomWidth - 1) / 2));

            player.x = rooms[0].x+rooms[0].width/2;
```

```
            player.y = rooms[0].y+rooms[0].height/2;
            entityLayer[player.y,player.x] = player;



            boss = new Boss(-1, -1, 4, 1, this);
            AddEnemyToEntityLayer(rooms[1], boss);



        }
        public void DisplayDungeon()
        {
            List<DisplayCharacter> displayOutput = new List<DisplayCharacter>();
            int cameraTop;
            int cameraBottom;
            int cameraLeft;
            int cameraRight;
            //if the top of the camera is within the dungeon
            if (player.y - (cameraHeight - 1) / 2 >= 0)
            {
                //if the bottom of the camera is within the dungeon
```

```
                if (player.y + (cameraHeight - 1) / 2 <= dunHeight - 1)
                {
                    cameraTop = player.y - (cameraHeight - 1) / 2;
                    cameraBottom = player.y + (cameraHeight - 1) / 2;
                }
                //if the bottom of the camera isnt within the dungeon
                else
                {
                    cameraTop = dunHeight - cameraHeight;
                    cameraBottom = dunHeight - 1;
                }
            }
            //if the top of the camera isnt within the dungeon
            else
            {
                cameraTop = 0;
                cameraBottom = cameraHeight - 1;
            }
```

```
            //if the left of the camera is within the dungeon
            if (player.x - (cameraWidth - 1) / 2 >= 0)
            {
                //if the right of the camera is within the dungeon
                if (player.x + (cameraWidth - 1) / 2 <= dunWidth - 1)
                {
                    cameraLeft = player.x - (cameraWidth - 1) / 2;
                    cameraRight = player.x + (cameraWidth - 1) / 2;
                }
                //if the right of the camera isnt within the dungeon
                else
                {
                    cameraLeft = dunWidth - cameraWidth;
                    cameraRight = dunWidth - 1;
                }
            }
            //if the top of the camera isnt within the dungeon
            else
            {
                cameraLeft = 0;
                cameraRight = cameraWidth - 1;
            }
            for (int i = 0; i < cameraWidth+3; i++)
            {
                displayOutput.Add(new DisplayCharacter("==", ConsoleColor.White));
            }
```

```
            displayOutput.Add(new DisplayCharacter("\n", ConsoleColor.Black));
            for (int i = cameraTop; i <= cameraBottom; i++)
            {
                for (int j = cameraLeft; j <= cameraRight; j++)
                {
                    if (j == cameraLeft)
                    {
                        displayOutput.Add(new DisplayCharacter("[] ",
ConsoleColor.White));
                    }
                    if (effectsLayer[i, j].character!=null)
                    {
```

```
                        DisplayCharacter displayCharacter = effectsLayer[i,j];
                        displayCharacter.character=displayCharacter.character+" ";
                        displayOutput.Add(displayCharacter);
                    }
                    else if (entityLayer[i, j] != null)
                    {
                        DisplayCharacter displayCharacter = entityLayer[i,
j].Display();
                        displayCharacter.character=displayCharacter.character+" ";
                        displayOutput.Add(displayCharacter);
                    }
                    else if (itemLayer[i, j] != null)
                    {
                        DisplayCharacter displayCharacter = itemLayer[i,
j][0].GameDisplay();
                        displayCharacter.character=displayCharacter.character+" ";
                        displayOutput.Add(displayCharacter);
                    }
                    else
                    {
                        DisplayCharacter displayCharacter = new DisplayCharacter();
                        if (tileLayer[i, j] == null)
                        {
                            displayCharacter = new
DisplayCharacter("O",ConsoleColor.Black);
                        }
                        else
                        {
                            displayCharacter = tileLayer[i, j].Display();
                        }
                        displayCharacter.character=displayCharacter.character+" ";
                        displayOutput.Add(displayCharacter);
```

```
                    }
                    if (j == cameraRight)
                    {
                        displayOutput.Add(new DisplayCharacter("[] ",
ConsoleColor.White));
                    }
```

```
            }
            displayOutput.Add(new DisplayCharacter("\n", ConsoleColor.Black));
        }
        for (int i = 0; i < cameraWidth+3; i++)
        {
            displayOutput.Add(new DisplayCharacter("==", ConsoleColor.White));
        }
        displayOutput.Add(new DisplayCharacter("\n", ConsoleColor.Black));
        foreach (DisplayCharacter character in displayOutput)
        {
            Console.ForegroundColor = character.colour;
            Console.Write(character.character);
        }
        Console.ForegroundColor = ConsoleColor.Gray;
    }
    public bool AddRooms()
    {
        int currentNumChestRooms = 0;
        int currentNumRandomRooms = 0;
        int maxAttempts = (numChestRooms + maxNumRandomRooms) * 10;
        int numAttempts = 0;
        int exitRoom=rand.Next(0,maxNumRandomRooms);
        Room room;

        Room startingRoom = new Room(dunWidth / 2 - 5 / 2, dunHeight / 2 - 5 /
2, 5, 5);
        rooms.Add(startingRoom);
        startingRoom.AddRoom(tileLayer);

        while (currentNumChestRooms < numChestRooms&& numAttempts < maxAttempts)
        {
            room = new ChestRoom(rand.Next(1, dunWidth - 7 - 1), rand.Next(1,
dunWidth - 7 - 1));
            numAttempts++;
            if (room.IsRoomValid(tileLayer))
            {
                currentNumChestRooms++;
                rooms.Add(room);
```

```
                    room.AddRoom(tileLayer);
                }
            }
            if (currentNumChestRooms != numChestRooms)
            {
                return false;
            }
            while (currentNumRandomRooms < maxNumRandomRooms && numAttempts <
maxAttempts)
            {
                if(currentNumRandomRooms==exitRoom)
                {
                    room = new RandomRoom(dunWidth, dunHeight, minRandomRoomHeight,
maxRandomRoomHeight, minRandomRoomWidth, maxRandomRoomWidth,true);
                }
                else
                {
                    room = new RandomRoom(dunWidth, dunHeight, minRandomRoomHeight,
maxRandomRoomHeight, minRandomRoomWidth, maxRandomRoomWidth,false);
                }
                numAttempts++;
                if (room.IsRoomValid(tileLayer))
                {
                    rooms.Add(room);
                    currentNumRandomRooms++;
                    room.AddRoom(tileLayer);
                }

            }
            if (currentNumRandomRooms==maxNumRandomRooms)
            {
                return true;
            }
            return false;
        }
        public bool AddCorridors()
        {
            Room room1 = new Room();
            Room room2 = new Room();
```

```
            int maxAttempts = (rooms.Count - 1) * 10;
            int numAttempts = 0;
            int numCorridors = 0;
            int currentRoom1 = numChestRooms;
            int currentRoom2;
```

```
            bool corridorValid = false;
            while (numCorridors < rooms.Count - 1 && numAttempts < maxAttempts)
            {
                corridorValid = false;
                room1 = rooms[currentRoom1];

                currentRoom2 = (currentRoom1 + 1) % rooms.Count;
                while (!corridorValid && numAttempts < maxAttempts)
                {

                    if (currentRoom2 == currentRoom1)
                    {
                        return false;
                    }
                    room2 = rooms[currentRoom2];
                    if (IsHorizontalCorridorValid( room1.x + (room1.width - 1) / 2,
room2.x + (room2.width - 1) / 2, room1.y + (room1.height - 1) / 2) &&
IsVerticalCorridorValid(room2.y + (room2.height - 1) / 2, room1.y + (room1.height -
1) / 2, room2.x + (room2.width - 1) / 2))
                    {
                        corridorValid = true;
                        AddHorizontalCorridor(room1.x + (room1.width - 1) / 2,
room2.x + (room2.width - 1) / 2, room1.y + (room1.height - 1) / 2);
                        AddVerticalCorridor(room2.y + (room2.height - 1) / 2,
room1.y + (room1.height - 1) / 2, room2.x + (room2.width - 1) / 2);
                    }
                    else if (IsHorizontalCorridorValid( room2.x + (room2.width - 1)
/ 2, room1.x + (room1.width - 1) / 2, room2.y + (room2.height - 1) / 2) &&
IsVerticalCorridorValid(room1.y + (room1.height - 1) / 2, room2.y + (room2.height -
1) / 2, room1.x + (room1.width - 1) / 2))
                    {
                        corridorValid = true;
                        AddHorizontalCorridor(room2.x + (room2.width - 1) / 2,
```

```
room1.x + (room1.width - 1) / 2, room2.y + (room2.height - 1) / 2);
                    AddVerticalCorridor(room1.y + (room1.height - 1) / 2,
room2.y + (room2.height - 1) / 2, room1.x + (room1.width - 1) / 2);
                }
                else
                {
                    currentRoom2 = (currentRoom2 + 1) % rooms.Count;
                }
                numAttempts++;
            }
            currentRoom1 = (currentRoom1 + 1) % rooms.Count; ;
```

```
            numCorridors++;
        }
        if (numCorridors == rooms.Count - 1)
        {
            return true;
        }
        return false;
    }
    public bool IsHorizontalCorridorValid(int x1, int x2, int y)
    {
        int consecutiveWalls = 0;
        if (x1 < x2)
        {
            for (int i = x1; i <= x2; i++)
            {
                if (tileLayer[y, i] == null)
                {
                    consecutiveWalls = 0;
                }

                else if (tileLayer[y, i].GetType() == (new Wall()).GetType())
                {
                    consecutiveWalls++;
                }

                if (consecutiveWalls > 1)
```

```
                {
                    return false;
                }
            }
        }
        else if (x2 < x1)
        {
            for (int i = x1; i >= x2; i--)
            {
                if (tileLayer[y, i] == null)
                {
                    consecutiveWalls = 0;
                }

                else if (tileLayer[y, i].GetType() == (new Wall()).GetType())
                {
                    consecutiveWalls++;
                }

                if (consecutiveWalls > 1)
                {
                    return false;
                }
            }
        }
        return true;
    }
    public bool IsVerticalCorridorValid( int y1, int y2, int x)
    {
        int consecutiveWalls = 0;
        if (y1 < y2)
        {
            for (int i = y1; i <= y2; i++)
            {
                if (tileLayer[i, x] == null)
                {
                    consecutiveWalls = 0;
                }
```

```
                else if (tileLayer[i, x].GetType() == (new Wall()).GetType())
                {
                    consecutiveWalls++;
                }

                if (consecutiveWalls > 1)
                {
                    return false;
                }
            }
        }
        else if (y2 < y1)
        {
            for (int i = y1; i >= y2; i--)
            {
                if (tileLayer[i, x] == null)
                {
                    consecutiveWalls = 0;
                }

                else if (tileLayer[i, x].GetType() == (new Wall()).GetType())
                {
                    consecutiveWalls++;
                }

                if (consecutiveWalls > 1)
                {
                    return false;
                }
            }
        }
        return true;
    }
    public void AddHorizontalCorridor(int x1, int x2, int y)
    {
        if (x1 < x2)
        {
```

```
            for (int i = x1; i <= x2; i++)
            {
                if (tileLayer[y, i] == null)
                {
                    tileLayer[y, i] = new CorridorFloor();
                }
                else if (tileLayer[y, i].GetType() == (new Wall()).GetType())
                {
                    tileLayer[y, i] = new Door(i, y);
                }
            }
        }
        else if (x2 < x1)
        {
            for (int i = x1; i >= x2; i--)
            {
                if (tileLayer[y, i] == null)
                {
                    tileLayer[y, i] = new CorridorFloor();
                }
                else if (tileLayer[y, i].GetType() == (new Wall()).GetType())
                {
                    tileLayer[y, i] = new Door(i, y);
                }
            }
        }

    }
    public void AddVerticalCorridor(int y1, int y2, int x)
```

```
    {
        if (y1 < y2)
        {
            for (int i = y1; i <= y2; i++)
            {
                if (tileLayer[i, x] == null)
                {
                    tileLayer[i, x] = new CorridorFloor();
                }
```

```
                else if (tileLayer[i, x].GetType() == (new Wall()).GetType())
                {
                    tileLayer[i, x] = new Door(i, x);
                }
            }
        }
        else if (y2 < y1)
        {
            for (int i = y1; i >= y2; i--)
            {
                if (tileLayer[i, x] == null)
                {
                    tileLayer[i, x] = new CorridorFloor();
                }
                else if (tileLayer[i, x].GetType() == (new Wall()).GetType())
                {
                    tileLayer[i, x] = new Door(i, x);
                }
            }
        }

    }
    public void AddEnemies()
    {
        int numEnemiesInRoom;
        enemies = new List<Enemy>();
        foreach(Room room in rooms)
        {
            if(room.HasEnemies())
            {numEnemiesInRoom = room.GetNumEnemies();
            for(int j = 0;j<numEnemiesInRoom;j++)
            {
                AddEnemy(room);
            }}
        }
```

```
    }
    public void AddEnemy(Room room)
    {
```

```
            Enemy enemy = new RandomEnemy(-1,-1,-1,-1,this);
            int random;
            if (difficulty <= 1.5)
            {

                random = rand.Next(0, 3);
                switch (random)
                {
                    case 0:
                        enemy = new VerticalEnemy(-1, -1, 1, 1, this);
                        break;
                    case 1:
                        enemy = new HorizontalEnemy(-1, -1, 1, 1, this);
                        break;
                    case 2:
                        enemy = new RandomEnemy(-1, -1, 1, 1, this);
                        break;
                }
            }
            else if (difficulty <= 2)
            {
                random = rand.Next(0, 4);
                switch (random)
                {
                    case 0:
                        enemy = new VerticalEnemy(-1, -1, 2, 1, this);
                        break;
                    case 1:
                        enemy = new HorizontalEnemy(-1, -1, 2, 1, this);
                        break;
                    case 2:
                        enemy = new RandomEnemy(-1, -1, 1, 1, this);
                        break;
                    case 3:
                        enemy = new TrackingEnemy(-1, -1, 1, 1, this);
                        break;
                }
            }
            else if (difficulty <= 3)
```

```
            {
                random = rand.Next(0, 6);
```

```
                switch (random)
                {
                    case 0:
                        enemy = new VerticalEnemy(-1, -1, 2, 1, this);
                        break;

                    case 1:
                        enemy = new HorizontalEnemy(-1, -1, 2, 1, this);
                        break;

                    case 2:
                        enemy = new RandomEnemy(-1, -1, 2, 1, this);
                        break;

                    case 3:
                        enemy = new TrackingEnemy(-1, -1, 2, 1, this);
                        break;

                    case 4:
                        enemy = new StaticStraightRangedEnemy(-1, -1, 1, 1, this);
                        break;
                    case 5:
                        enemy = new StaticUniformAOEEnemy(-1, -1, 1, 1, this);
                        break;

                }
            }
            else if (difficulty <= 4)
            {
                random = rand.Next(0, 7);
                switch (random)
                {
                    case 0:
                        enemy = new VerticalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), Convert.ToInt32(difficulty / 2), this);
                        break;
```

```
                    case 1:
                        enemy = new HorizontalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 2:
```

```
                        enemy = new RandomEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 3:
                        enemy = new TrackingEnemy(-1, -1, Convert.ToInt32(1 *
difficulty) - 1, Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 4:
                        enemy = new StaticStraightRangedEnemy(-1, -1,
Convert.ToInt32(1 * difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5),
this);
                        break;
                    case 5:
                        enemy = new StaticUniformAOEEnemy(-1, -1, Convert.ToInt32(1
* difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5), this);
                        break;
                    case 6:
                        enemy = new TrailDamageTileTrackingEnemy(-1, -1, 1, 1, this,
5);
                        break;

                }
            }
            else if (difficulty <= 5)
            {
                random = rand.Next(0, 7);
                switch (random)
                {
                    case 0:
```

```
                    enemy = new VerticalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                    break;

                case 1:
                    enemy = new HorizontalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                    break;

                case 2:
                    enemy = new RandomEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                    break;
```

```
                case 3:
                    enemy = new SmartTrackingEnemy(-1, -1, Convert.ToInt32(1 *
difficulty) - 1, Convert.ToInt32(difficulty / 2), this);
                    break;
                case 4:
                    enemy = new StaticStraightRangedEnemy(-1, -1,
Convert.ToInt32(1 * difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5),
this);
                    break;
                case 5:
                    enemy = new StaticUniformAOEEnemy(-1, -1, Convert.ToInt32(1
* difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5), this);
                    break;
                case 6:
                    enemy = new TrailDamageTileTrackingEnemy(-1, -1, 1, 1, this,
5);
                    break;

            }
        }
        else
        {
            random = rand.Next(0, 9);
            switch (random)
            {
```

```
                    case 0:
                        enemy = new VerticalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 1:
                        enemy = new HorizontalEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 2:
                        enemy = new RandomEnemy(-1, -1, Convert.ToInt32(1 *
difficulty), 1 + Convert.ToInt32(difficulty / 2), this);
                        break;

                    case 3:
                        enemy = new SmartTrackingEnemy(-1, -1, Convert.ToInt32(1 *
difficulty) - 1, Convert.ToInt32(difficulty / 2), this);
                        break;
```

```
                    case 4:
                        enemy = new StaticStraightRangedEnemy(-1, -1,
Convert.ToInt32(1 * difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5),
this);
                        break;
                    case 5:
                        enemy = new StaticUniformAOEEnemy(-1, -1, Convert.ToInt32(1
* difficulty) - 1, Convert.ToInt32((difficulty / 2) - 0.5), this);
                        break;
                    case 6:
                        enemy = new TrailDamageTileTrackingEnemy(-1, -1,
Convert.ToInt32(1 * difficulty) - 3, Convert.ToInt32((difficulty / 2) - 0.5), this,
3);
                        break;
                    case 7:
                        enemy=new TrackingUniformAOEEnemy(-1,-1,Convert.ToInt32(1 *
difficulty) - 4,Convert.ToInt32((difficulty / 2) - 1.5),this);
                        break;
```

```
                    case 8:
                        enemy=new
TrackingStraightRangedEnemy(-1,-1,Convert.ToInt32(1 * difficulty) -
4,Convert.ToInt32((difficulty / 2) - 1.5),this);
                        break;

                }
            }
            AddEnemyToEntityLayer(room, enemy);

        }
        public bool AddEnemyToEntityLayer(Room room, Enemy enemy)
        {
            int enemyx;
            int enemyy;
            int maxAttempts = 1000;
            int numAttempts = 0;
            while (numAttempts < maxAttempts)
            {
                enemyx = rand.Next(room.x + 1, room.x + room.width - 1);
                enemyy = rand.Next(room.y + 1, room.y + room.height - 1);
                if (!tileLayer[enemyy, enemyx].DoesCollide() && entityLayer[enemyy,
enemyx] == null)
                {
                    enemy.x = enemyx;
```

```
                    enemy.y = enemyy;
                    entityLayer[enemyy, enemyx] = enemy;
                    enemies.Add(enemy);
                    return true;
                }
            }
            return false;

        }
        public void MazeToAdjacencyList()
        {

            Coord currentCoord;
```

```
            coordAdjacencyList = new Dictionary<Coord, List<Coord>>();
            for (int i = 0; i < dunHeight; i++)
            {
                for (int j = 0; j < dunWidth; j++)
                {

                    currentCoord = new Coord(j, i);

                    if (tileLayer[i, j] != null && !tileLayer[i, j].DoesCollide())
                    {
                        coordAdjacencyList.Add(currentCoord, new List<Coord>());

                        if (tileLayer[i + 1, j] != null && !tileLayer[i + 1,
j].DoesCollide())
                        {
                            coordAdjacencyList[currentCoord].Add(new Coord(j, i +
1));

                        }
                        if (tileLayer[i - 1, j] != null && !tileLayer[i - 1,
j].DoesCollide())
                        {
                            coordAdjacencyList[currentCoord].Add(new Coord(j, i -
1));

                        }
                        if (tileLayer[i, j + 1] != null && !tileLayer[i, j +
1].DoesCollide())
                        {
                            coordAdjacencyList[currentCoord].Add(new Coord(j + 1,
i));
```

```
                        }
                        if (tileLayer[i, j - 1] != null && !tileLayer[i, j -
1].DoesCollide())
                        {
                            coordAdjacencyList[currentCoord].Add(new Coord(j - 1,
i));
```

```
                    }
                }



            }
        }

    }
    public void GenerateWeightMap()
    {
        weightMap = new int[dunHeight,dunWidth];
        for(int i = 0; i < dunHeight; i++)
        {
            for(int j = 0; j < dunWidth; j++)
            {
                if (tileLayer[i, j] == null)
                {
                    weightMap[i,j]=1000000;
                }
                else if (tileLayer[i, j].DoesCollide())
                {
                    weightMap[i,j]=1000000;
                }
                else if (tileLayer[i, j].GetType() == typeof(Floor))
                {
                    weightMap[i,j]=1;
                }
                else if (tileLayer[i, j].GetType() == typeof(DamageTile))
                {
                    weightMap[i,j]=100;
                }
                else
                {
                    weightMap[i,j]=1;
                }
            }
        }
```

```
    }
    public Player GetPlayer()
    {
        return player;
    }
    public Dictionary<Coord,List<Coord>> GetCoordAdjacencyList()
    {
        return coordAdjacencyList;
    }
    public List<Item>[,] GetItemLayer()
    {
        return itemLayer;
    }
    public Tile[,] GetTileLayer()
    {
        return tileLayer;
    }
    public Entity[,] GetEntityLayer()
    {
        return entityLayer;
    }
    public DisplayCharacter[,] GetEffectsLayer()
    {
        return effectsLayer;
    }
    public int[,] GetWeightMap()
    {
        return weightMap;
    }
    public List<Enemy> GetEnemies(){
        return enemies;
    }
    public int GetDunWidth()
    {
        return dunWidth;
    }
    public int GetDunHeight()
    {
        return dunHeight;
```

```
        }
        public double GetDifficulty()
        {
            return difficulty;
```

```
        }
    }
}
namespace goodbyeworld
{

    public class Room
    {
        public static Random rand = new Random();
        public int x;
        public int y;
        public int height;
        public int width;
        protected bool enemies;
        protected int minEnemies;
        protected int maxEnemies;
        public Room(int xin, int yin, int heightin, int widthin)
        {
            x = xin;
            y = yin;
            height = heightin;
            width = widthin;
            enemies = false;
            minEnemies = 1;
            if ((height - 2) * (width - 2) < 16)
            {
                maxEnemies = 1;
            }
            else if ((height - 2) * (width - 2) < 25)
            {
                maxEnemies = 2;
            }
            else if ((height - 2) * (width - 2) < 36)
            {
```

```
                maxEnemies = 3;
            }
            else if ((height - 2) * (width - 2) < 49)
            {
                minEnemies = 2;
                maxEnemies = 4;
            }
            else if ((height - 2) * (width - 2) < 64)
            {
                minEnemies = 3;
```

```
                maxEnemies = 5;
            }
            else if ((height - 2) * (width - 2) < 81)
            {
                minEnemies = 4;
                maxEnemies = 6;
            }
            else
            {
                minEnemies = 4;
                maxEnemies = 6;
            }
        }
        public Room() { }

        public virtual void AddRoom(Tile[,] dungeon)
        {
            for (int i = y; i < y + height; i++)
            {
                for (int j = x; j < x + width; j++)
                {
                    if ((i == y || i == y + height - 1) && (j == x || j == x + width
- 1))
                    {
                        continue;
                    }
                    else if (i == y || i == y + height - 1)
                    {
```

```
                    dungeon[i, j] = new Wall();
                }
                else if (j == x || j == x + width - 1)
                {
                    dungeon[i, j] = new Wall();
                }
                else
                {
                    dungeon[i, j] = new Floor();
                }

            }
        }
    }
    public virtual bool IsRoomValid(Tile[,] dungeon)
    {
```

```
        for (int i = y - 1; i < y + height + 1; i++)
        {
            for (int j = x - 1; j < x + width + 1; j++)
            {
                if (dungeon[i, j] != null)
                {
                    return false;
                }
            }
        }

        return true;
    }
    public int GetNumEnemies()
    {
        return rand.Next(minEnemies, maxEnemies + 1);
    }
    public bool HasEnemies()
    {
        return enemies;
    }
}
```

```
public class ChestRoom : Room
{
    public ChestRoom() { }
    public ChestRoom(int xin, int yin):base(xin,yin,7,7)
    {
        enemies = false;
    }
    public override void AddRoom(Tile[,] dungeon)
    {
        for (int i = y; i < y + height; i++)
        {
            for (int j = x; j < x + width; j++)
            {
                if ((i == y || i == y + height - 1) && (j == x || j == x + width
- 1))
                {
                    continue;
                }
                else if (i == y || i == y + height - 1)
                {
                    dungeon[i, j] = new Wall();
                }
```

```
                else if (j == x || j == x + width - 1)
                {
                    dungeon[i, j] = new Wall();
                }
                else if (i == y + (height-1)/2 && j == x + (width-1) / 2)
                {
                    dungeon[i, j] = new Chest(j, i);
                }
                else
                {
                    dungeon[i, j] = new Floor();
                }

            }
        }
    }
```

```
    }
    public class RandomRoom : Room
    {
        protected bool exitRoom;
        protected bool hasRandomWalls = false;
        public RandomRoom() { }
        public RandomRoom(int maxx, int maxy, int minHeight, int maxHeight, int
minWidth, int maxWidth, bool isExitRoom) : base()
        {
            height = rand.Next(minHeight, maxHeight + 1);
            width = rand.Next(minWidth, maxWidth + 1);
            x = rand.Next(1, maxx - width - 1);
            y = rand.Next(1, maxy - height - 1);
            enemies = true;
            exitRoom = isExitRoom;
            minEnemies = 1;
            if (rand.Next(0, 2) == 0)
            {
                hasRandomWalls = true;
            }
            if ((height - 1) * (width - 1) < 16)
            {
                maxEnemies = 1;
            }
            else if ((height - 1) * (width - 1) < 25)
            {
                maxEnemies = 2;
            }
```

```
            else if ((height - 1) * (width - 1) < 36)
            {
                maxEnemies = 3;
            }
            else if ((height - 1) * (width - 1) < 49)
            {
                minEnemies = 2;
                maxEnemies = 4;
            }
            else if ((height - 1) * (width - 1) < 64)
```

```
            {
                minEnemies = 3;
                maxEnemies = 5;
            }
            else if ((height - 1) * (width - 1) < 81)
            {
                minEnemies = 4;
                maxEnemies = 6;
            }
            else
            {
                minEnemies = 4;
                maxEnemies = 6;
            }

        }
        public override void AddRoom(Tile[,] dungeon)
        {
            for (int i = y; i < y + height; i++)
            {
                for (int j = x; j < x + width; j++)
                {
                    if ((i == y || i == y + height - 1) && (j == x || j == x + width
- 1))
                    {
                        continue;
                    }
                    else if (i == y || i == y + height - 1)
                    {
                        dungeon[i, j] = new Wall();
                    }
                    else if (j == x || j == x + width - 1)
                    {
                        dungeon[i, j] = new Wall();
```

```
                    }
                    else
                    {
                        if (rand.Next(0, (height - 2) * (width - 2)) < (height - 2)
```

```
* (width - 2) / 7)//1/7 chance of a floor being a wall instead
                        {
                            dungeon[i, j] = new Wall();
                        }
                        else
                        {
                            dungeon[i, j] = new Floor();
                        }
                    }

                }
            }
            if(exitRoom)
            {dungeon[rand.Next(y+1,y+height-1),rand.Next(x+1,x+width-1)]=new
ExitTile();}
        }
    }
    public class BossRoom : Room
    {
        protected double difficulty;
        public BossRoom(int xin, int yin,int heightIn, int widthIn,double
difficultyIn):base(xin,yin,heightIn,widthIn)
        {
            enemies = false;
            difficulty=difficultyIn;
        }
        public override void AddRoom(Tile[,] dungeon)
        {
            for (int i = y; i < y + height; i++)
            {
                for (int j = x; j < x + width; j++)
                {
                    if ((i == y || i == y + height - 1) && (j == x || j == x + width
- 1))
                    {
                        continue;
                    }
                    else if (i == y || i == y + height - 1)
                    {
```

```
                    dungeon[i, j] = new Wall();
```

```
                }
                else if (j == x || j == x + width - 1)
                {
                    dungeon[i, j] = new Wall();
                }
                else
                {
                    int chance = rand.Next(0,50);
                    if (difficulty > 4)
                    {
                        if (chance < 44)
                        {
                            dungeon[i, j] = new Floor();
                        }
                        else if(chance<48)
                        {
                            dungeon[i, j] = new Wall();
                        }
                        else
                        {
                            dungeon[i,j] = new DamageTile(1,true);
                        }
                    }
                    else
                    {
                        if (chance < 48)
                        {
                            dungeon[i, j] = new Floor();
                        }
                        else
                        {
                            dungeon[i, j] = new Wall();
                        }
                    }
                }

            }
        }
```

```
        }
    }
}


namespace goodbyeworld
{
```

```
    public abstract class Tile
    {
        protected bool collision;
        public Tile()
        {


        }
        public virtual bool DoesCollide()
        {
            if (collision)
            {
                return true;
            }
            return false;
        }
        public virtual DisplayCharacter Display()
        {


            return new DisplayCharacter(" ",ConsoleColor.Black);
        }
    }
    public class DungeonBorder : Tile
    {
        public DungeonBorder()
        {
            collision = true;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("@",ConsoleColor.Black);
        }
    }
```

```
public class Floor : Tile
{
    public Floor() : base()
    {
        collision = false;
    }
    public override DisplayCharacter Display()
    {
        return new DisplayCharacter(" ",ConsoleColor.Black);
    }
}
public class CorridorFloor : Tile
```

```
{
    public CorridorFloor()
    {
        collision = false;
    }
    public override DisplayCharacter Display()
    {
        return new DisplayCharacter(" ",ConsoleColor.Black);
    }
}
public class Wall : Tile
{
    public Wall()
    {
        collision = true;
    }
    public override DisplayCharacter Display()
    {
        return new DisplayCharacter("@",ConsoleColor.Gray);
    }
}
public class Door : Tile
{
    int x;
    int y;
    public Door() : base()
```

```
        {
            collision = true;
        }
        public Door(int xin, int yin) : base()
        {
            x = xin;
            y = yin;
            collision = true;
        }
        public void Interact()
        {
            collision = false;
        }
        public override DisplayCharacter Display()
        {
            if (collision)
            { return new DisplayCharacter("X",ConsoleColor.Gray); }
            return new DisplayCharacter("X",ConsoleColor.DarkGray);
```

```
        }
    }
    public class Chest : Tile
    {
        Random rand = new Random();
        private int x;
        private int y;
        public Chest() : base()
        {
            collision = true;
        }
        public Chest(int xin, int yin) : base()
        {
            x = xin;
            y = yin;
            collision = true;
        }
        public void OpenChest(Player player, List<Item>[,] itemLayer)
        {
            if (player.GetNumKeys() > 0)
```

```
            {
                player.ChangeNumKeys(player.GetNumKeys() - 1);
                //itemLayer[y, x].Add(DropItem());
                if (itemLayer[y, x] == null)
                {
                    itemLayer[y, x] = new List<Item>();
                }
                itemLayer[y, x].Add(DropItem());
                collision = false;
            }



        }
        public Item DropItem()
        {
            int chance = rand.Next(0, 100);
            if (chance < 50)
            {
                List<Item> commonItems = new List<Item> { new SmallHealPotion(1),
new StrengthPotion(1, 10),new PassiveAbilityCooldownItem(1),new
ActiveAbilityCooldownItem(1) };
                return commonItems[rand.Next(0, commonItems.Count)];
            }
            else if (chance < 80)
```

```
            {
                List<Item> uncommonItems = new List<Item> {new Longsword(1), new
Broadsword(1),new MediumHealPotion(1),new SmallHealPotion(2), new StrengthPotion(2,
15), new DMGBoostItem(1),new FullHeartItem(1),new PassiveAbilityCooldownItem(2),new
ActiveAbilityCooldownItem(2) };
                return uncommonItems[rand.Next(0, uncommonItems.Count)];
            }
            else if (chance < 95)
            {
                List<Item> rareItems = new List<Item> {new SmallHealPotion(3), new
MediumHealPotion(2),new LargeHealPotion(1), new StrengthPotion(3, 20), new
Broadsword(2),new Longsword(2),new DMGBoostItem(2),new FullHeartItem(2),new
PassiveAbilityDurationItem(1),new ActiveAbilityDurationItem(1),new
PassiveAbilityCooldownItem(4),new ActiveAbilityCooldownItem(4),new Mace(1) };
```

```
                return rareItems[rand.Next(0, rareItems.Count)];
            }
            else if(chance<100)
            {
                List<Item> epicItems = new List<Item> { new Mace(3), new
Longsword(3), new FullHeartItem(3),new LargeHealPotion(3),new DMGBoostItem(3),new
FullHeartItem(4),new PassiveAbilityDurationItem(3),new ActiveAbilityDurationItem(3)
};
                return epicItems[rand.Next(0, epicItems.Count)];
            }
            return new Coin(1);
        }
        public override DisplayCharacter Display()
        {
            if(collision)
            {return new DisplayCharacter("+",ConsoleColor.Yellow);}
            return new DisplayCharacter("+",ConsoleColor.DarkGray);


        }


    }
    public class ExitTile : Tile
    {
        public ExitTile()
        {
            collision = false;
        }
        public void Enter(ref bool isFightingBoss)
        {
            isFightingBoss = true;
```

```
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter(":", ConsoleColor.Red);
        }
    }
    public class DamageTile : Tile
    {
```

```
        protected bool isEnemyDamageTile;
        protected int dmg;
        public DamageTile(int dmgIn,bool isEnemyDamageTileIn)
        {
            dmg = dmgIn;
            collision = false;
            isEnemyDamageTile=isEnemyDamageTileIn;
        }
        public void DoDamage(LivingEntity entity, Entity[,] entityLayer)
        {
            entity.ChangeHP(entity.GetHP() - dmg, entityLayer);
        }
        public override DisplayCharacter Display()
        {
            if(isEnemyDamageTile)
            {
                return new DisplayCharacter("#", ConsoleColor.DarkRed);
            }
            else
            {
                return new DisplayCharacter("#", ConsoleColor.DarkBlue);
            }
        }
        public bool IsEnemyDamageTile()
        {
            return isEnemyDamageTile;
        }
    }
}

namespace goodbyeworld
{
    public abstract class Entity
    {
        public static Random rand = new Random();
        public int x;
```

```
        public int y;
        public Entity(int xin, int yin)
        {
```

```
            x = xin;
            y = yin;
        }
        public Entity(){}
        public abstract DisplayCharacter Display();



    }
    public abstract class LivingEntity : Entity
    {
        protected bool damageTileImmunity = false;
        protected int maxHP;
        protected int hp;
        public LivingEntity() : base() { }
        public LivingEntity(int xin, int yin, int hpin) : base(xin, yin)
        {
            maxHP = hpin;
            hp = maxHP;


        }
        public List<Coord> Dijkstras(Dungeon dungeon,Coord destination, Coord
source)
        {
            Coord currentNode;
            bool found = false;
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int dunHeight = dungeon.GetDunHeight();
            int dunWidth = dungeon.GetDunWidth();
            Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
            int[,] weightMap = dungeon.GetWeightMap();
            List<Coord> shortestPath = new List<Coord>();
            PriorityQueue queue = new PriorityQueue(5000);
            bool[,] visited = new bool[dunHeight, dunWidth];
            Coord[,] parent = new Coord[dunHeight, dunWidth];
            int[,] distance = new int[dunHeight,dunWidth];
            for (int i = 0; i < dunHeight; i++)
            {
                for (int j = 0; j < dunWidth; j++)
```

```
            {
                distance[i,j] = 1000000;
```

```
            }
        }
        weightMap[source.y,source.x]=0;
        parent[source.y, source.x] = new Coord(-1,-1);
        distance[source.y,source.x]=0;
        visited[source.y, source.x] = true;
        queue.Enqueue(source,distance);
        while (!queue.IsEmpty())
        {
            currentNode = queue.Dequeue();
            foreach (Coord neighbour in coordAdjacencyList[currentNode]) //check
each neighbour of current node and add them to queue if valid
            {
                if (!visited[neighbour.y, neighbour.x]) //if it hasnt been
visited
                {
                    if
(weightMap[neighbour.y,neighbour.x]+distance[currentNode.y,currentNode.x]<distance[
neighbour.y,neighbour.x]) //if the weight at the neighbour+the distance of the
current node is smaller than the neighbour's current distance
                    {

distance[neighbour.y,neighbour.x]=weightMap[neighbour.y,neighbour.x]+distance[curre
ntNode.y,currentNode.x]; //change the distance
                        parent[neighbour.y,neighbour.x]=currentNode;// change
the parent
                    }

                    else //if the weight at the neighbour+the distance of the
current node is greater than or equal to the neighbour's current distance
                    {
                        visited[neighbour.y,neighbour.x]=true; //set the
neighbour as visited, its distance wont be lower
                    }
```

```
                    queue.Enqueue(neighbour,distance);
                }
                if (neighbour.y == destination.y && neighbour.x ==
destination.x)
                {
                    found = true;
                }
            }
```

```
        }
        currentNode = destination;
        bool sourceReached = false;
        if(found)
        {
            while (!sourceReached)
            {
                shortestPath.Add(currentNode);
                currentNode = parent[currentNode.y, currentNode.x];
                if (currentNode.x == source.x && currentNode.y == source.y)
                {
                    sourceReached = true;
                }
            }
        }
        shortestPath.Reverse();

        return shortestPath;
    }
    public List<Coord> BreadthFirstSearch(Dungeon dungeon,Coord destination,
Coord source)
    {
        Coord currentNode;
        Tile[,] tileLayer = dungeon.GetTileLayer();
        int dunHeight = dungeon.GetDunHeight();
        int dunWidth = dungeon.GetDunWidth();
        Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
        bool found = false;
```

```
            List<Coord> shortestPath = new List<Coord>();
            Queue queue = new Queue(1000);
            bool[,] visited = new bool[dunHeight, dunWidth]; //create bool grids to
show which nodes of maze have been visited
            Coord[,] parent = new Coord[dunHeight, dunWidth];
            parent[source.y, source.x] = new Coord(-1, -1);
            queue.Enqueue(source);
            visited[source.y, source.x] = true;
            while (!found && !queue.IsEmpty()) //while the destination hasn't been
found and the queue still has a node in, repeat
            {
                currentNode = queue.Dequeue(); //look at the next node in the queue
                foreach (Coord neighbour in coordAdjacencyList[currentNode]) //check
each neighbour of current node and add them to queue if valid
```

```
                {
                    if (!visited[neighbour.y, neighbour.x])
                    {
                        queue.Enqueue(neighbour);
                        visited[neighbour.y, neighbour.x] = true;
                        parent[neighbour.y, neighbour.x] = currentNode;
                        if (neighbour.y == destination.y && neighbour.x ==
destination.x)
                        {

                            found = true;
                        }
                    }

                }



            }
            currentNode = destination;
            bool sourceReached = false;
            if (found)
            {
                while (!sourceReached)
                {
```

```
                shortestPath.Add(currentNode);
                currentNode = parent[currentNode.y, currentNode.x];
                if (currentNode.x == source.x && currentNode.y == source.y)
                {
                    sourceReached = true;
                }
            }
            shortestPath.Reverse();
        }
        return shortestPath;

    }
    public bool IsAlive()
    {
        if (hp > 0)
        {
            return true;
        }
        return false;
    }
```

```
    public int GetHP()
    {
        return hp;
    }
    public int GetMaxHP()
    {
        return maxHP;
    }
    public void ChangeMaxHP(int amount)
    {
        maxHP = amount;
    }
    public void ChangeHP(int amount,Entity[,] entityLayer)
    {
        if (amount >= maxHP)
        {
            hp = maxHP;
        }
```

```
            else if(amount<0)
            {
                hp = 0;
            }
            else
            {
                hp=amount;
            }
            if(!IsAlive()){
                entityLayer[y,x]=null;
            }
        }

    }
    public class Player : LivingEntity
    {
        protected int dmg;
        protected List<PassiveStatItem> passiveStatItems= new
List<PassiveStatItem>();
        protected ConsumableItem consumableItem;
        protected Weapon weapon = new Sword(1);
        protected PassiveAbility passiveAbility;
        protected ActiveAbility activeAbility;

        protected int numCoins = 0;
        protected int numKeys = 1;
```

```
        public Player(int xin, int yin, int hpin, int dmgin) : base(xin, yin, hpin)
        {
            dmg = dmgin;
            GivePlayerRandomAbilities();
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("i",ConsoleColor.White);
        }
        public void GivePlayerRandomAbilities()
        {
```

```
            List<PassiveAbility> passiveAbilities = new List<PassiveAbility> {new
PassiveAOEAttack(15,1,0),new PassiveHeal(30,1,0),new
PassiveTemporaryDMGBoost(20,1,5),new PassiveDamageTileTrail(15,1,0) };
            List<ActiveAbility> activeAbilities = new List<ActiveAbility> {new
ActiveAOEAttack(10,1,0),new ActiveHeal(30,1,0),new
ActiveTemporaryDMGBoost(20,1,5),new ActiveDamageTileTrail(25,1,0) };
            passiveAbility = passiveAbilities[rand.Next(0, passiveAbilities.Count)];
            activeAbility = activeAbilities[rand.Next(0, activeAbilities.Count)];
        }
        public void Input(Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            ConsoleKey input;
            input = Console.ReadKey().Key;
            if (input == ConsoleKey.UpArrow)
            {
                if (IsMoveValid(dungeon, x, y - 1))
                {
                    entityLayer[y, x] = null;
                    y -= 1;
                    entityLayer[y, x] = this;
                }
            }
            else if (input == ConsoleKey.DownArrow)
            {
                if (IsMoveValid(dungeon, x, y + 1))
                {
                    entityLayer[y, x] = null;
                    y += 1;
                    entityLayer[y, x] = this;
                }
            }
```

```
            else if (input == ConsoleKey.LeftArrow)
            {
                if (IsMoveValid(dungeon, x - 1, y))
                {
                    entityLayer[y, x] = null;
                    x -= 1;
```

```
                    entityLayer[y, x] = this;


                }
            }
            else if (input == ConsoleKey.RightArrow)
            {
                if (IsMoveValid(dungeon, x + 1, y))
                {
                    entityLayer[y, x] = null;
                    x += 1;
                    entityLayer[y, x] = this;
                }
            }
            else if (input == ConsoleKey.W || input == ConsoleKey.A || input ==
ConsoleKey.S || input == ConsoleKey.D)
            {
                weapon.UseWeapon(dungeon, this, input);
            }
            else if (input == ConsoleKey.E)
            {
                if (consumableItem != null)
                {
                    if
(consumableItem.GetType().IsSubclassOf(typeof(DurationConsumableItem)))
                    {
                        DurationConsumableItem item =
(DurationConsumableItem)consumableItem;
                        if (item.GetNumUses() > 0)
                        {
                            if (item.GetCurrentDuration() == 0)
                            {
                                item.ChangeCurrentDuration(item.GetDuration());
                                item.DoItemEffect(this, dungeon);
                                item.UseItem();
                                if (item.GetNumUses() <= 0 &&
item.GetCurrentDuration() <= 0)
                                {
                                    consumableItem = null;
```

```
                            }
                        }
                    }
                    else
                    {
                        consumableItem = null;
                    }
                }
                else
                {
                    if (consumableItem.GetNumUses() > 0)
                    {
                        consumableItem.DoItemEffect(this, dungeon);
                        consumableItem.UseItem();
                        if (consumableItem.GetNumUses() <= 0)
                        {
                            consumableItem = null;
                        }
                    }
                    else
                    {
                        consumableItem = null;
                    }
                }

            }
        }
        else if (input == ConsoleKey.Q)
        {
            if (activeAbility != null && activeAbility.CanUse())
            {
                if (activeAbility.GetDuration() > 0)
                {

activeAbility.ChangeCurrentDuration(activeAbility.GetDuration());
                }
                else
                {
                    activeAbility.UseAbility(dungeon, this);
```

```
                }
            }
        }
    }
    public void DecreasePlayerCooldownsAndDurations(Dungeon dungeon)
```

```
    {
        if (activeAbility!=null)
        {
            activeAbility.DecreaseCooldown(dungeon, this);
        }
        if (passiveAbility!=null)
        {
            passiveAbility.DecreaseCooldown(dungeon, this);
        }
        if (consumableItem!=null &&
consumableItem.GetType().IsSubclassOf(typeof(DurationConsumableItem)))
        {
            DurationConsumableItem item =
(DurationConsumableItem)consumableItem;
            item.DecreaseDuration(this, dungeon);
            if (item.GetCurrentDuration() <= 0 && item.GetNumUses() <= 0)
            {
                consumableItem = null;
            }
        }
    }
    public bool IsMoveValid(Dungeon dungeon,int x, int y)
    {
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        List<Item>[,] itemLayer = dungeon.GetItemLayer();
        List<Enemy> enemies = dungeon.GetEnemies();
        int height = dungeon.GetDunHeight();
        int width = dungeon.GetDunWidth();
        if (x < 0 || x > width - 1 || y < 0 || y > height - 1)
        {
            return false;
        }
```

```
            else if (tileLayer[y, x] == null)
            {
                tileLayer[y, x] = new Floor();
                return false;
            }
            else if (tileLayer[y, x].GetType() == new Wall().GetType())
            {
                tileLayer[y, x] = new Floor();
                return false;
            }
```

```
            else if (tileLayer[y, x].GetType() == new Door().GetType() &&
tileLayer[y, x].DoesCollide())
            {
                ((Door)tileLayer[y, x]).Interact();
                return false;

            }
            else if (tileLayer[y, x].GetType() == new Chest().GetType()&&
tileLayer[y, x].DoesCollide())
            {
                ((Chest)tileLayer[y, x]).OpenChest(this, itemLayer);
                return false;

            }
            else if (tileLayer[y, x].GetType() == typeof(DungeonBorder))
            {
                return false;
            }
            else if (entityLayer[y, x] != null)
            {
                return false;
            }
            else if (tileLayer[y, x].GetType() == new ExitTile().GetType())
            {
                dungeon.ExitDungeon();
            }
            if (itemLayer[y, x] != null && itemLayer[y, x].Count > 0)
            {
```

```
            PickUpItem(dungeon, itemLayer[y, x], x, y);
        }
        return true;
    }
    public void PickUpItem(Dungeon dungeon, List<Item> items, int x, int y)
    {
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        List<Item>[,] itemLayer = dungeon.GetItemLayer();
        List<Item> newItems = new List<Item>();
        foreach (Item item in items)
        {

            if (item.GetType() == new Coin().GetType())
            {
                Coin coin = (Coin)item;
                numCoins += coin.GetNumCoins();
```

```
            }
            else if (item.GetType() == new Key().GetType())
            {
                Key key = (Key)item;
                numKeys += key.GetNumKeys();

            }
            else if (item.GetType().IsSubclassOf(typeof(Weapon)))
            {
                Weapon temp = weapon;
                weapon = (Weapon)item;
                newItems.Add(temp);
            }
            else if (item.GetType().IsSubclassOf(typeof(ConsumableItem)))
            {
                if (consumableItem == null)
                {
                    consumableItem = (ConsumableItem)item;
                }
                else
                {
```

```
                    ConsumableItem temp = consumableItem;
                    consumableItem = (ConsumableItem)item;
                    newItems.Add(temp);
                }
            }
            else if (item.GetType().IsSubclassOf(typeof(PassiveStatItem)))
            {
                PassiveStatItem temp = (PassiveStatItem)item;
                temp.BoostPlayerStat(this, entityLayer);
                passiveStatItems.Add(temp);
            }

        }
        if (newItems.Count <= 0)
        { itemLayer[y, x] = null; }
        else
        {
            itemLayer[y, x].Clear();
            foreach (Item item in newItems)
            {
                itemLayer[y, x].Add(item);
            }
```

```
        }



    }
    public int EnemyAttack(Enemy enemy, Dungeon dungeon)
    {
        enemy.EnemyAttacked(dmg);
        return dmg;
    }
    public int GetDMG()
    {
        return dmg;
    }
    public void ChangeDMG(int amount)
    {
        dmg = amount;
```

```
    }
    public void DisplayInventory()
    {


    }
    public int GetNumCoins()
    {
        return numCoins;
    }
    public int GetNumKeys()
    {
        return numKeys;
    }
    public Weapon GetWeapon()
    {
        return weapon;
    }
    public void ChangeNumCoins(int amount)
    {
        numCoins = amount;
    }
    public void ChangeNumKeys(int amount)
    {
        numKeys = amount;
    }
    public ActiveAbility GetActiveAbility()
    {
        return activeAbility;
```

```
    }
    public PassiveAbility GetPassiveAbility()
    {
        return passiveAbility;
    }
    public ConsumableItem GetConsumableItem()
    {
        return consumableItem;
    }
    public bool HasConsumableItem()
```

```
        {
            if (consumableItem == null)
            {
                return false;
            }
            return true;
        }
        public bool HasPassiveAbility()
        {
            if (passiveAbility == null)
            {
                return false;
            }
            return true;
        }
        public bool HasActiveAbility()
        {
            if (activeAbility == null)
            {
                return false;
            }
            return true;
        }
    }
}

namespace goodbyeworld
{
    public abstract class Enemy : LivingEntity
    {
        protected Dungeon dungeon;
        protected bool damageTileImmunity = false;

        protected bool move;
```

```
        protected int dmg;
        protected int minCoin;
        protected int maxCoin;
        protected int keyChance;
```

```
        public Enemy() : base() { }

        public Enemy(int xin, int yin, int hpin, int dmgin,Dungeon dungeonIn) :
base(xin, yin, hpin)
        {
            dungeon = dungeonIn;
            hp = hpin;
            dmg = dmgin;
            keyChance=1;
            minCoin=1;
            maxCoin=1;
        }
        public abstract void Move();
        public bool HasDamageTileImmunity()
        {
            return damageTileImmunity;
        }




        public virtual bool IsMoveValid(int xpos, int ypos)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            //if the tile theyre moving to is outside the bounds of the array,
return false
            if (xpos < 0 || xpos >= width || ypos < 0 || ypos >= height)
            {
                return false;
            }
            //if the tile is null, return false
            else if (tileLayer[ypos, xpos] == null)
            {
                return false;
            }
            //if the tile isnt null, and the tile has collision, return false
```

```
            else if (tileLayer[ypos, xpos].DoesCollide())
            {
                return false;
            }
            //if there is an entity on the tile theyre moving to, return false
            else if (entityLayer[ypos, xpos] != null)
            {
                return false;
            }
            return true;
        }
        public virtual bool IsAttackValid(int x, int y)
        {
            int dunHeight = dungeon.GetDunHeight();
            int dunWidth = dungeon.GetDunWidth();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Player player = dungeon.GetPlayer();
            //if the tile is out of bounds, return false
            if (x >= dunWidth)
            {
                return false;
            }
            else if (y >= dunHeight)
            {
                return false;
            }
            else if (x < 0)
            {
                return false;
            }
            else if (y < 0)
            {
                return false;
            }
            //if the tile is null, return false
            else if (tileLayer[y, x] == null)
            {
```

```
                return false;
            }
            //if the tile has collision, return false
            else if (tileLayer[y, x].DoesCollide())
            {
                return false;
            }
            //if there is an entity and its not a player, return false
            else if (entityLayer[y, x] != null && entityLayer[y, x] != player)
            {
                return false;
            }
            return true;
        }
        public virtual void EnemyAttacked(int dmg)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            List<Item>[,] itemLayer = dungeon.GetItemLayer();
            List<Enemy> enemies = dungeon.GetEnemies();
            hp -= dmg;

            if (!IsAlive())
            {
                DropCoins( );
                DropKey();
                //remove the enemy from the enemies list and remove them from the
entity layer
                enemies.Remove(this);
                entityLayer[y, x] = null;
            }
        }
        public void DropCoins()
        {
            List<Item>[,] itemLayer = dungeon.GetItemLayer();
            if (itemLayer[y, x] == null)
            {
                itemLayer[y, x] = new List<Item>();
```

```
            }
            itemLayer[y, x].Add(new Coin(rand.Next(minCoin, maxCoin + 1)));
        }
        public void DropKey( )
        {
            List<Item>[,] itemLayer = dungeon.GetItemLayer();
            if (rand.Next(0, 100) < keyChance)
            {
                if (itemLayer[y, x] == null)
                {
                    itemLayer[y, x] = new List<Item>();
```

```
                }
                itemLayer[y, x].Add(new Key(1));
            }

        }
        public int Attack()
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Player player = dungeon.GetPlayer();
            player.ChangeHP(player.GetHP()-dmg,  entityLayer);
            return dmg;
        }

    }

    public abstract class StraightRangedEnemy : Enemy
    {
        protected bool rangeAttackChargeUp = false;
        public StraightRangedEnemy(int xin, int yin, int hpin, int dmgin,Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin,dungeonIn)
        {
        }
        public bool IsStraightRangedAttackValid()
        {
            Tile[,] tileLayer = dungeon.GetTileLayer();
            Player player = dungeon.GetPlayer();
```

```
        if (player.x == x&&player.y!=y-1&&player.y!=y+1)
        {
            if (player.y > y)
            {
                for (int i = y + 1; i <= player.y; i++)
                {
                    if (!IsAttackValid(x,i))
                    {
                        return false;
                    }
                }
                return true;
            }
            else
            {
                for (int i = y - 1; i >= player.y; i--)
                {
                    if (!IsAttackValid(x, i))
```

```
                    {
                        return false;
                    }
                }
                return true;
            }
        }
        else if (player.y == y&&player.x!=x-1&&player.x!=x+1)
        {
            if (player.x > x)
            {
                for (int i = x + 1; i <= player.x; i++)
                {
                    if (!IsAttackValid(i, y))
                    {
                        return false;
                    }
                }
                return true;
            }
```

```
            else
            {
                for (int i = x - 1; i >= player.x; i--)
                {
                    if (!IsAttackValid(i, y))
                    {
                        return false;
                    }
                }
                return true;
            }
        }
        return false;
    }
    public void StraightRangedAttack()
    {
        Player player = dungeon.GetPlayer();
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
        if (IsStraightRangedAttackValid())
        {
            if (rangeAttackChargeUp)
            {
```

```
                player.ChangeHP(player.GetHP() - dmg, entityLayer);
                effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                rangeAttackChargeUp = false;
            }
            else
            {
                rangeAttackChargeUp = true;
                if (player.x == x)
                {
                    if (player.y > y)
                    {
                        for (int i = y + 1; i <= player.y - 1; i++)
                        {
```

```
                            effectsLayer[i, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
                    }
                    else
                    {
                        for (int i = y - 1; i >= player.y + 1; i--)
                        {
                            effectsLayer[i, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
                    }
                }
                else if (player.y == y)
                {
                    if (player.x > x)
                    {
                        for (int i = x + 1; i <= player.x - 1; i++)
                        {
                            effectsLayer[y, i] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
                    }
                    else
                    {
                        for (int i = x - 1; i >= player.x + 1; i--)
                        {
                            effectsLayer[y, i] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
```

```
                    }
                }

            }
        }
        else
        {
            rangeAttackChargeUp = false;
```

```
            }
        }


    }
    public abstract class UniformAOEEnemy : Enemy
    {
        protected bool UniformAOEAttackChargeUp = false;
        public UniformAOEEnemy(int xin, int yin, int hpin, int dmgin,Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin,dungeonIn)
        {
        }
        public void UniformAOEAttack()
        {
            Player player = dungeon.GetPlayer();
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (UniformAOEAttackChargeUp)
            {
                if (IsUniformAOEAttackValid())
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                UniformAOEAttackChargeUp = false;
            }
            else
            {
                UniformAOEAttackChargeUp = true;
                if (IsAttackValid(x + 1, y) && (player.x != x + 1 || player.y != y))
                {
                    effectsLayer[y, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                if (IsAttackValid(x - 1, y) && (player.x != x - 1 || player.y != y))
                {
```

```
                    effectsLayer[y, x-1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
```

```
                }
                 if (IsAttackValid(x - 1, y + 1) && (player.x != x - 1 || player.y
!= y + 1))
                {
                    effectsLayer[y+1, x-1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x + 1, y + 1) && (player.x != x + 1 || player.y
!= y + 1))
                {
                    effectsLayer[y+1, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x - 1, y - 1) && (player.x != x - 1 || player.y
!= y - 1))
                {
                    effectsLayer[y-1, x-1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x + 1, y - 1) && (player.x != x + 1 || player.y
!= y - 1))
                {
                    effectsLayer[y-1, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x, y - 1) && (player.x != x || player.y != y -
1))
                {
                    effectsLayer[y-1, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x, y+1) && (player.x != x||player.y!=y+1))
                {
                    effectsLayer[y+1, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
            }
        }
        public bool IsUniformAOEAttackValid()
```

```
    {
        Player player = dungeon.GetPlayer();
        if (x + 1 == player.x && y == player.y)
        {
            return true;
        }
        else if (x - 1 == player.x && y == player.y)
        {
            return true;
        }
        else if (x + 1 == player.x && y + 1 == player.y)
        {
            return true;
        }
        else if (x - 1 == player.x && y + 1 == player.y)
        {
            return true;
        }
        else if (x + 1 == player.x && y - 1 == player.y)
        {
            return true;
        }
        else if (x - 1 == player.x && y - 1 == player.y)
        {
            return true;
        }
        else if (x == player.x && y + 1 == player.y)
        {
            return true;
        }
        else if (x == player.x && y - 1 == player.y)
        {
            return true;
        }
        return false;
    }

}
```

```
}
namespace goodbyeworld
{
   public class DummyEnemy : Enemy
   {
       public DummyEnemy(int xin, int yin,int hpin, int dmgin,Dungeon dungeonIn) :
base(xin, yin, hpin, dmgin, dungeonIn)
       {
```

```
       }
       public override void Move()
       {
       }
       public override DisplayCharacter Display()
       {
           return new DisplayCharacter("x",ConsoleColor.Red);
       }
   }
   public class HorizontalEnemy : Enemy
   {
       protected bool right;
       public HorizontalEnemy(int xin, int yin, int hpin, int dmgin, Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
       {
           if (rand.Next(0, 2) == 0)
           {
               right = true;
               move = false;
           }
           else
           {
               right = false;
               move = true;

           }
           double difficulty = dungeon.GetDifficulty();
           minCoin = Convert.ToInt32(3 * difficulty);
           maxCoin = Convert.ToInt32(6 * difficulty);
```

```
            keyChance = 3;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("a",ConsoleColor.Red);
        }
        public override void Move()
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (move)
```

```
            {
                if (right)
                {
                    if (IsMoveValid(x + 1, y))
                    {
                        entityLayer[y, x] = null;
                        x++;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[y, x + 1] == player)
                    {
                        Attack();
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                    }
                    else
                    {
                        right = false;
                    }
                }
                else
                {
                    if (IsMoveValid(x - 1, y))
```

```
                    {
                        entityLayer[y, x] = null;
                        x--;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[y, x - 1] == player)
                    {
                        Attack();
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                    }
                    else
                    {
                        right = true;
                    }
                }
                move = false;
            }
            else
            {
                move = true;
```

```
            }
        }


    }
    public class VerticalEnemy : Enemy
    {
        protected bool down;
        public VerticalEnemy(int xin, int yin, int hpin, int dmgin, Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
        {
            if (rand.Next(0, 2) == 0)
            {
                down = true;
                move = false;
            }
            else
            {
```

```
                down = false;
                move = true;
            }
            double difficulty = dungeon.GetDifficulty();
            minCoin = Convert.ToInt32(3 * difficulty);
            maxCoin = Convert.ToInt32(6 * difficulty);
            keyChance = 3;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("b",ConsoleColor.Red);
        }
        public override void Move()
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (move)
            {
                if (down)
                {
                    if (IsMoveValid(x, y + 1))
                    {
                        entityLayer[y, x] = null;
```

```
                        y++;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[y + 1, x] == player)
                    {
                        Attack();
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                    }
                    else
                    {
```

```
                        down = false;
                    }
                }
                else
                {
                    if (IsMoveValid(x, y - 1))
                    {
                        entityLayer[y, x] = null;
                        y--;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[y - 1, x] == player)
                    {
                        Attack();
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                    }
                    else
                    {
                        down = true;
                    }
                }
                move = false;
            }
            else
            {
                move = true;
            }
        }
    }
    public class RandomEnemy : Enemy
    {
```

```
        public RandomEnemy(int xin, int yin, int hpin, int dmgin, Dungeon dungeonIn)
: base(xin, yin, hpin, dmgin, dungeonIn)
        {
            if (rand.Next(0, 2) == 0)
            {
                move = false;
```

```
        }
        else
        {
            move = true;
        }
        double difficulty = dungeon.GetDifficulty();
        minCoin = Convert.ToInt32(3 * difficulty);
        maxCoin = Convert.ToInt32(6 * difficulty);
        keyChance = 5;
    }
    public override DisplayCharacter Display()
    {
        return new DisplayCharacter("c",ConsoleColor.Red);
    }
    public override void Move()
    {
        DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
        Player player = dungeon.GetPlayer();
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        int height = dungeon.GetDunHeight();
        int width = dungeon.GetDunWidth();
        int[] moves = new int[] { -1, 1 };
        int xpos = 0;
        int ypos = 0;
        if (move)
        {
            if (rand.Next(0, 2) == 0)
            {
                xpos = x + moves[rand.Next(0, 2)];
                ypos = y;
            }
            else
            {
                ypos = y + moves[rand.Next(0, 2)];
                xpos = x;
            }
            if (IsMoveValid(xpos, ypos))
```

```
                {
                    entityLayer[y, x] = null;

                    entityLayer[ypos, xpos] = this;
                    y = ypos;
                    x = xpos;

                }
                else if (entityLayer[ypos, xpos] == player)
                {
                    Attack();
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }

                move = false;
            }
            else
            {
                move = true;
            }
        }

    }
    public class TrackingEnemy : Enemy
    {
        public TrackingEnemy(int xin, int yin, int hpin, int dmgin, Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
        {
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
            {
                move = true;
            }
            minCoin = Convert.ToInt32(8 * difficulty);
```

```
            maxCoin = Convert.ToInt32(12 * difficulty);
            keyChance = 7;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("d", ConsoleColor.Red);
        }
        public override void Move()
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
            if (move)
            {
                List<Coord> nextCoords = BreadthFirstSearch(dungeon, new
Coord(player.x, player.y), new Coord(x, y));
                if (nextCoords.Count > 0)
                {
                    Coord nextCoord = nextCoords[0];
                    if (IsMoveValid(nextCoord.x, nextCoord.y))
                    {
                        entityLayer[y, x] = null;
                        x = nextCoord.x;
                        y = nextCoord.y;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[nextCoord.y, nextCoord.x] == player)
                    {
                        Attack();
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                    }
                }
                move = false;
            }
            else
```

```
            {
                move = true;
            }

        }
    }
    public class StaticStraightRangedEnemy : StraightRangedEnemy
    {
        public StaticStraightRangedEnemy(int xin, int yin, int hpin, int dmgin,
Dungeon dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
        {
```

```
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
            {
                move = true;
            }
            minCoin = Convert.ToInt32(8 * difficulty);
            maxCoin = Convert.ToInt32(12 * difficulty);
            keyChance = 15;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("e",ConsoleColor.Red);
        }
        public override void Move()
        {
            if (rangeAttackChargeUp)
            {
                StraightRangedAttack();
            }
            else
            {
                if (move)
                {
```

```
                    if(IsStraightRangedAttackValid())
                    {
                        StraightRangedAttack();
                    }
                    move = false;
                }
                else
                {
                    move = true;
                }
            }
        }
    }
    public class StaticUniformAOEEnemy : UniformAOEEnemy
    {
        public StaticUniformAOEEnemy(int xin, int yin, int hpin, int dmgin, Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
```

```
        {
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
            {
                move = true;
            }
            minCoin = Convert.ToInt32(8 * difficulty);
            maxCoin = Convert.ToInt32(12 * difficulty);
            keyChance = 15;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("f", ConsoleColor.Red);
        }
        public override void Move()
        {
            if (UniformAOEAttackChargeUp)
```

```
            {
                UniformAOEAttack();
            }
            else
            {
                if (move)
                {
                    UniformAOEAttack();
                    move = false;
                }
                else
                {
                    move = true;
                }
            }
        }
    }
    public class TrailDamageTileTrackingEnemy : Enemy
    {
        protected int moveCounter = 1;
        protected int moveDelay;
        public TrailDamageTileTrackingEnemy(int xin, int yin, int hpin, int dmgin,
Dungeon dungeonIn,int moveDelayIn) : base(xin, yin, hpin, dmgin, dungeonIn)
```

```
        {
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
            {
                move = true;
            }
            minCoin = Convert.ToInt32(8 * difficulty);
            maxCoin = Convert.ToInt32(12 * difficulty);
            keyChance = 7;
            moveDelay = moveDelayIn;
            damageTileImmunity = true;
```

```
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("g", ConsoleColor.Red);
        }
        public override void Move()
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            moveCounter++;
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (move&&moveCounter%moveDelay==0)
            {
                moveCounter = 0;
                List<Coord> nextCoords = BreadthFirstSearch(dungeon, new
Coord(player.x, player.y), new Coord(x, y));
                if (nextCoords.Count > 0)
                {
                    Coord nextCoord = nextCoords[0];
                    if (IsMoveValid(nextCoord.x, nextCoord.y))
                    {
                        entityLayer[y, x] = null;
                        x = nextCoord.x;
                        y = nextCoord.y;
```

```
                        entityLayer[y, x] = this;
                        if(rand.Next(0,2)==0)
                        {
                            tileLayer[y, x] = new DamageTile(1,true);
                        }
                    }
                    else if (entityLayer[nextCoord.y, nextCoord.x] == player)
                    {
                        player.ChangeHP(player.GetHP() - dmg, entityLayer);
```

```
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
            }
            move = false;
        }
        else
        {
            move = true;
        }

    }
}
public class SmartTrackingEnemy : Enemy
{
    public SmartTrackingEnemy(int xin, int yin, int hpin, int dmgin, Dungeon
dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
    {
        double difficulty = dungeon.GetDifficulty();
        if (rand.Next(0, 2) == 0)
        {
            move = false;
        }
        else
        {
            move = true;
        }
        minCoin = Convert.ToInt32(8 * difficulty);
        maxCoin = Convert.ToInt32(12 * difficulty);
        keyChance = 7;
    }
    public override DisplayCharacter Display()
    {
        return new DisplayCharacter("h", ConsoleColor.Red);
    }
```

```
    public override void Move()
    {
        DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
```

```
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (move)
            {
                List<Coord> nextCoords = Dijkstras(dungeon, new Coord(player.x,
player.y), new Coord(x, y));
                if (nextCoords.Count > 0)
                {
                    Coord nextCoord = nextCoords[0];
                    if (IsMoveValid(nextCoord.x, nextCoord.y))
                    {
                        entityLayer[y, x] = null;
                        x = nextCoord.x;
                        y = nextCoord.y;
                        entityLayer[y, x] = this;
                    }
                    else if (entityLayer[nextCoord.y, nextCoord.x] == player)
                    {
                        player.ChangeHP(player.GetHP() - dmg, entityLayer);
                        effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);

                    }
                }
                move = false;
            }
            else
            {
                move = true;
            }

        }
    }
    public class TrackingUniformAOEEnemy: UniformAOEEnemy
```

```
    {
```

```
        public TrackingUniformAOEEnemy(int xin, int yin, int hpin, int dmgin,
Dungeon dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
        {
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
            {
                move = true;
            }
            minCoin = Convert.ToInt32(10 * difficulty);
            maxCoin = Convert.ToInt32(14 * difficulty);
            keyChance = 20;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("j", ConsoleColor.Red);
        }
        public override void Move()
        {

            if (UniformAOEAttackChargeUp)
            {
                UniformAOEAttack();
            }
            else
            {
                if (move)
                {
                    Player player = dungeon.GetPlayer();
                    Entity[,] entityLayer = dungeon.GetEntityLayer();
                    List<Coord> nextCoords = BreadthFirstSearch(dungeon, new
Coord(player.x, player.y), new Coord(x, y));
                    if (nextCoords.Count > 0)
                    {
```

```
                        Coord nextCoord = nextCoords[0];
                        if (IsMoveValid(nextCoord.x, nextCoord.y))
                        {
                            entityLayer[y, x] = null;
                            x = nextCoord.x;
                            y = nextCoord.y;
                            entityLayer[y, x] = this;
```

```
                        }
                        else if (entityLayer[nextCoord.y, nextCoord.x] == player)
                        {
                            DisplayCharacter[,] effectsLayer =
dungeon.GetEffectsLayer();
                            Attack();
                            effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                        }
                    }
                    UniformAOEAttack();
                    move = false;
                }
                else
                {
                    move = true;
                }
            }
        }
    }
    public class TrackingStraightRangedEnemy : StraightRangedEnemy
    {
        public TrackingStraightRangedEnemy(int xin, int yin, int hpin, int dmgin,
Dungeon dungeonIn) : base(xin, yin, hpin, dmgin, dungeonIn)
        {
            double difficulty = dungeon.GetDifficulty();
            if (rand.Next(0, 2) == 0)
            {
                move = false;
            }
            else
```

```
            {
                move = true;
            }
            minCoin = Convert.ToInt32(10 * difficulty);
            maxCoin = Convert.ToInt32(14 * difficulty);
            keyChance = 20;
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("k",ConsoleColor.Red);
        }
        public override void Move()
        {
```

```
            if (rangeAttackChargeUp)
            {
                StraightRangedAttack();
            }
            else
            {
                if (move)
                {
                    if(IsStraightRangedAttackValid())
                    {
                        StraightRangedAttack();
                    }
                    else
                    {
                        Player player = dungeon.GetPlayer();
                        Entity[,] entityLayer = dungeon.GetEntityLayer();
                        List<Coord> nextCoords = BreadthFirstSearch(dungeon, new
Coord(player.x, player.y), new Coord(x, y));
                        if (nextCoords.Count > 0)
                        {
                            Coord nextCoord = nextCoords[0];
                            if (IsMoveValid(nextCoord.x, nextCoord.y))
                            {
                                entityLayer[y, x] = null;
                                x = nextCoord.x;
```

```
                            y = nextCoord.y;
                            entityLayer[y, x] = this;
                        }
                        else if (entityLayer[nextCoord.y, nextCoord.x] ==
player)
                        {
                            DisplayCharacter[,] effectsLayer =
dungeon.GetEffectsLayer();
                            Attack();
                            effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                        }
                    }
                }
                move = false;
            }
            else
            {
                move = true;
```

```
            }
        }
    }
}
public class Boss:Enemy{

    protected bool rangeAttackChargeUp = false;
    protected bool UniformAOEAttackChargeUp = false;
    protected double difficulty;
    public Boss(int xin, int yin, int hpin, int dmgin, Dungeon dungeonIn) :
base(xin, yin, hpin, dmgin, dungeonIn)
    {
        damageTileImmunity=true;
        difficulty = dungeon.GetDifficulty();
        if (rand.Next(0, 2) == 0)
        {
            move = false;
        }
        else
```

```
            {
                move = true;
            }
            minCoin = Convert.ToInt32(20 * difficulty);
            maxCoin = Convert.ToInt32(25 * difficulty);
            keyChance = 100;
            hp = Convert.ToInt32(hpin * difficulty);
            dmg = Convert.ToInt32(dmgin * ((1 + difficulty) / 2));
        }
        public override DisplayCharacter Display()
        {
            return new DisplayCharacter("z",ConsoleColor.Red);
        }
        public override void Move()
        {

            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            Dictionary<Coord, List<Coord>> coordAdjacencyList =
dungeon.GetCoordAdjacencyList();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (rangeAttackChargeUp)
            {
```

```
                StraightRangedAttack();
            }
            if (UniformAOEAttackChargeUp)
            {
                UniformAOEAttack();
            }
            if (move)
            {
                if (IsStraightRangedAttackValid() && rand.Next(0, 2) ==
0&&difficulty>=2.5)
                {
                    StraightRangedAttack();
                }
```

```
                else if (IsUniformAOEAttackValid() && rand.Next(0, 2) ==
0&&difficulty>=2)
                {
                    UniformAOEAttack();
                }
                else
                {
                    List<Coord> nextCoords = Dijkstras(dungeon, new Coord(player.x,
player.y), new Coord(x, y));
                    if (nextCoords.Count > 0)
                    {
                        Coord nextCoord = nextCoords[0];
                        if (IsMoveValid(nextCoord.x, nextCoord.y))
                        {
                            entityLayer[y, x] = null;
                            x = nextCoord.x;
                            y = nextCoord.y;
                            entityLayer[y, x] = this;
                        }
                        else if (entityLayer[nextCoord.y, nextCoord.x] == player)
                        {
                            DisplayCharacter[,] effectsLayer =
dungeon.GetEffectsLayer();
                            Attack();
                            effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                        }

                    }
                    move = false;
                }
```

```
            }
            else
            {
                move = true;

            }
```

```
        }
        public void UniformAOEAttack()
        {
            Player player = dungeon.GetPlayer();
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (UniformAOEAttackChargeUp)
            {
                if (x + 1 == player.x && y == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x - 1 == player.x && y == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x + 1 == player.x && y + 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x - 1 == player.x && y + 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x + 1 == player.x && y - 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
```

```
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x - 1 == player.x && y - 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x == player.x && y + 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                else if (x == player.x && y - 1 == player.y)
                {
                    player.ChangeHP(player.GetHP() - dmg, entityLayer);
                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                }
                UniformAOEAttackChargeUp = false;
            }
            else
            {
                UniformAOEAttackChargeUp = true;
                if (IsAttackValid(x + 1, y) && (player.x != x + 1 || player.y != y))
                {
                    effectsLayer[y, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                if (IsAttackValid(x - 1, y) && (player.x != x - 1 || player.y != y))
                {
                    effectsLayer[y, x-1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x - 1, y + 1) && (player.x != x - 1 || player.y
!= y + 1))
                {
                    effectsLayer[y+1, x-1] = new
```

```
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x + 1, y + 1) && (player.x != x + 1 || player.y
!= y + 1))
                {
                    effectsLayer[y+1, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x - 1, y - 1) && (player.x != x - 1 || player.y
!= y - 1))
                {
                    effectsLayer[y-1, x-1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x + 1, y - 1) && (player.x != x + 1 || player.y
!= y - 1))
                {
                    effectsLayer[y-1, x+1] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x, y - 1) && (player.x != x || player.y != y -
1))
                {
                    effectsLayer[y-1, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
                 if (IsAttackValid(x, y+1) && (player.x != x||player.y!=y+1))
                {
                    effectsLayer[y+1, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                }
            }
        }
        public bool IsUniformAOEAttackValid()
        {
            Player player = dungeon.GetPlayer();
            if (x + 1 == player.x && y == player.y)
            {
```

```
            return true;
        }
        else if (x - 1 == player.x && y == player.y)
        {
            return true;
        }
        else if (x + 1 == player.x && y + 1 == player.y)
        {
            return true;
        }
```

```
        else if (x - 1 == player.x && y + 1 == player.y)
        {
            return true;
        }
        else if (x + 1 == player.x && y - 1 == player.y)
        {
            return true;
        }
        else if (x - 1 == player.x && y - 1 == player.y)
        {
            return true;
        }
        else if (x == player.x && y + 1 == player.y)
        {
            return true;
        }
        else if (x == player.x && y - 1 == player.y)
        {
            return true;
        }
        return false;
    }
    public bool IsStraightRangedAttackValid()
    {
        Tile[,] tileLayer = dungeon.GetTileLayer();
        Player player = dungeon.GetPlayer();
        if (player.x == x&&player.y!=y-1&&player.y!=y+1)
        {
```

```
            if (player.y > y)
            {
                for (int i = y + 1; i <= player.y; i++)
                {
                    if (!IsAttackValid(x,i))
                    {
                        return false;
                    }
                }
                return true;
            }
            else
            {
                for (int i = y - 1; i >= player.y; i--)
                {
                    if (!IsAttackValid(x, i))
```

```
                    {
                        return false;
                    }
                }
                return true;
            }
        }
        else if (player.y == y&&player.x!=x-1&&player.x!=x+1)
        {
            if (player.x > x)
            {
                for (int i = x + 1; i <= player.x; i++)
                {
                    if (!IsAttackValid(i, y))
                    {
                        return false;
                    }
                }
                return true;
            }
            else
            {
```

```
                    for (int i = x - 1; i >= player.x; i--)
                    {
                        if (!IsAttackValid(i, y))
                        {
                            return false;
                        }
                    }
                    return true;
                }
            }
            return false;
        }
        public void StraightRangedAttack()
        {
            Player player = dungeon.GetPlayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            if (player.x == x || player.y == y)
            {
                if (rangeAttackChargeUp)
                {
```

```
                    if (player.x == x)
                    {
                        if (player.y > y)
                        {
                            for (int i = y+1; i <= player.y; i++)
                            {
                                if (!IsAttackValid(x,i))
                                {
                                    break;
                                }
                                else if (i == player.y)
                                {
                                    player.ChangeHP(player.GetHP() - dmg,
entityLayer);
                                    effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
```

```
                            }
                        }
                    }
                    else
                    {
                        for (int i = y-1; i >= player.y; i--)
                        {
                            if (!IsAttackValid(x,i))
                            {
                                break;
                            }
                            else if (i == player.y)
                            {
                                player.ChangeHP(player.GetHP() - dmg,
entityLayer);
                                effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                            }
                        }
                    }
                }
                else if (player.y == y)
                {
                    if (player.x > x)
                    {
                        for (int i = x+1; i <= player.x; i++)
                        {
                            if (!IsAttackValid(i,y))
```

```
                            {
                                break;
                            }
                            else if (i == player.x)
                            {
                                player.ChangeHP(player.GetHP() - dmg,
entityLayer);
                                effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                            }
```

```
                        }
                    }
                    else
                    {
                        for (int i = x-1; i >= player.x; i--)
                        {
                            if (!IsAttackValid(i,y))
                            {
                                break;
                            }
                            else if (i == player.x)
                            {
                                player.ChangeHP(player.GetHP() - dmg,
entityLayer);
                                effectsLayer[player.y, player.x] = new
DisplayCharacter("*",ConsoleColor.DarkRed);
                            }
                        }
                    }
                }
                rangeAttackChargeUp = false;
            }
            else
            {
                rangeAttackChargeUp = true;
                if (player.x == x)
                {
                    if (player.y > y)
                    {
                        for (int i = y + 1; i <= player.y - 1; i++)
                        {
                            if (!IsAttackValid(x,i))
                            {
                                break;
```

```
                            }
                            effectsLayer[i, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
```

```
                    }
                    else
                    {
                        for (int i = y - 1; i >= player.y + 1; i--)
                        {
                            if (!IsAttackValid(x,i))
                            {
                                break;
                            }
                            effectsLayer[i, x] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
                    }
                }
                else if (player.y == y)
                {
                    if (player.x > x)
                    {
                        for (int i = x + 1; i <= player.x - 1; i++)
                        {
                            if (!IsAttackValid(i,y))
                            {
                                break;
                            }
                            effectsLayer[y, i] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
                    }
                    else
                    {
                        for (int i = x - 1; i >= player.x + 1; i--)
                        {
                            if (!IsAttackValid(i,y))
                            {
                                break;
                            }
                            effectsLayer[y, i] = new
DisplayCharacter("~",ConsoleColor.DarkRed);
                        }
```

```
                    }
```

```
                }


            }
        }
        else
        {
            rangeAttackChargeUp = false;
        }
    }
    public override void EnemyAttacked(int dmg)
    {
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        List<Item>[,] itemLayer = dungeon.GetItemLayer();
        List<Enemy> enemies = dungeon.GetEnemies();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        Player player = dungeon.GetPlayer();
        hp -= dmg;
        bool exitTilePlaced = false;
        int tileX;
        int tileY;
        int numAttempts = 0;
        int maxAttempts = 100;
        if (!IsAlive())
        {
            DropCoins( );
            DropKey();
            enemies.Remove(this);
            entityLayer[y, x] = null;
            while (!exitTilePlaced && numAttempts <= maxAttempts)
            {
                tileY = rand.Next(y - 5, y + 5);
                tileX = rand.Next(x - 5, x + 5);
                if (!tileLayer[tileY, tileX].DoesCollide() && entityLayer[tileY,
tileX] != player && entityLayer[tileY, tileX] != this)
                {
                    tileLayer[tileY, tileX] = new ExitTile();
                    exitTilePlaced = true;
```

```
                    }
                    numAttempts++;

                }
                if(numAttempts==maxAttempts&&!exitTilePlaced)
                {tileLayer[y, x] = new ExitTile();}
            }
```

```
        }
    }
}
namespace goodbyeworld
{
    public abstract class Item
    {
        protected string itemName;
        protected string itemDesc;
        public Item()
        {
            itemName = "ITEM";
            itemDesc = "Nothing";
        }
        public void InventoryDisplay()
        {
            System.Console.WriteLine($"ITEM: {itemName}\nDESCRIPTION:
{itemDesc}\n");
        }
        public string GetName()
        {
            return itemName;
        }
        public abstract DisplayCharacter GameDisplay();


    }
    public class Coin : Item
    {
        private int numCoins;
        public Coin()
```

```
        {
            itemName = "Coin";
            itemDesc = "Money";
            numCoins = 0;
        }
        public Coin(int numCoinsIn)
        {
            itemName = "Coin";
            itemDesc = "Money";
            numCoins = numCoinsIn;
        }
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("$",ConsoleColor.Yellow);
        }
        public int GetNumCoins()
        {
            return numCoins;
        }
        public void ChangeNumCoins(int amount)
        {
            numCoins = amount;
        }
    }
    public class Key : Item
    {
        private int numKeys;
        public Key()
        {
            itemName = "Key";
            itemDesc = "Opens chests";
            numKeys = 0;
        }
        public Key(int numKeysIn)
        {
            itemName = "Key";
            itemDesc = "Opens chests";
            numKeys = numKeysIn;
```

```
    }
    public int GetNumKeys()
    {
        return numKeys;
    }
    public void ChangeNumKeys(int amount)
    {
        numKeys = amount;
    }
    public override DisplayCharacter GameDisplay()
    {
        return new DisplayCharacter("€",ConsoleColor.Yellow);
    }
}
public abstract class ConsumableItem : Item
{
    protected int numUses;
    public ConsumableItem(int numUsesIn) : base()
    {
```

```
        numUses = numUsesIn;
    }
    public abstract void DoItemEffect(Player player, Dungeon dungeon);
    public void UseItem()
    {
        if(numUses>0)
        {numUses--;}
    }
    public int GetNumUses()
    {
        return numUses;
    }
    public override DisplayCharacter GameDisplay()
    {
        return new DisplayCharacter("$",ConsoleColor.Green);
    }

}
public abstract class DurationConsumableItem:ConsumableItem
```

```
    {
        protected int duration;
        protected int currentDuration;
        public DurationConsumableItem(int numUsesIn, int durationIn) :
base(numUsesIn)
        {
            numUses = numUsesIn;
            duration = durationIn+1;
            currentDuration = 0;
        }
        public  void DecreaseDuration(Player player, Dungeon dungeon)
        {
            if (currentDuration > 0)
            {
                currentDuration--;
                DoItemEffect(player, dungeon);
            }
        }

        public int GetDuration()
        {
            return duration;
        }
        public void ChangeCurrentDuration(int amount)
        {
```

```
            currentDuration = amount;
        }
        public int GetCurrentDuration()
        {
            return currentDuration;
        }
    }
    public abstract class PassiveStatItem : Item
    {
        public PassiveStatItem() : base()
        {

        }
```

```
        public abstract void BoostPlayerStat(Player player, Entity[,] entityLayer);
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("€",ConsoleColor.DarkGreen);
        }
    }
}
namespace goodbyeworld
{
    public class SmallHealPotion : ConsumableItem
    {
        public SmallHealPotion(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Small Healing Potion";
            itemDesc = "Heals a heart";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (numUses > 0)
            {
                player.ChangeHP(player.GetHP() + 1, entityLayer);
            }

        }
    }
    public class MediumHealPotion : ConsumableItem
    {
        public MediumHealPotion(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Medium Healing Potion";
```

```
            itemDesc = "Heals two hearts";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (numUses > 0)
            {
```

```
                player.ChangeHP(player.GetHP() + 2, entityLayer);
            }


        }
    }
    public class LargeHealPotion : ConsumableItem
    {
        public LargeHealPotion(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Large Healing Potion";
            itemDesc = "Heals three hearts";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (numUses > 0)
            {
                player.ChangeHP(player.GetHP() + 3, entityLayer);
            }
        }
    }
    public class StrengthPotion : DurationConsumableItem
    {
        public StrengthPotion(int numUsesIn, int durationIn) : base(numUsesIn,
durationIn)
        {
            itemName = "Strength Potion";
            itemDesc = "Increases damage by 1";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (currentDuration == duration)
            {
                player.ChangeDMG(player.GetDMG() + 1);
            }
            else if (currentDuration == 0)
```

```
            {
```

```
                player.ChangeDMG(player.GetDMG() - 1);
            }


        }
    }
    public class RegenerationPotion : DurationConsumableItem
    {
        protected int healDelay;
        public RegenerationPotion(int numUsesIn, int durationIn,int healDelayIn) :
base(numUsesIn, durationIn)
        {
            itemName = "Regeneration Potion";
            itemDesc = $"Heals player by 1 every {healDelayIn} moves";
            healDelay=healDelayIn;
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            if (currentDuration % healDelay==0)
            {
                player.ChangeHP(player.GetHP() + 1,entityLayer);
            }
        }
    }
    public class ActiveAbilityDurationItem : ConsumableItem
    {
        public ActiveAbilityDurationItem(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Active ability duration item";
            itemDesc = "Increases duration of your active ability by 1";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {

player.GetActiveAbility().ChangeDuration(player.GetActiveAbility().GetDuration()+1)
;
        }
    }
    public class PassiveAbilityDurationItem : ConsumableItem
```

```
    {
        public PassiveAbilityDurationItem(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Passive ability duration item";
            itemDesc = "Increases duration of your passive ability by 1";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {

player.GetPassiveAbility().ChangeDuration(player.GetPassiveAbility().GetDuration()+
1);
        }
    }
    public class ActiveAbilityCooldownItem : ConsumableItem
    {
        public ActiveAbilityCooldownItem(int numUsesIn) : base(numUsesIn)
        {
            itemName = "Active ability cooldown item";
            itemDesc = "Decreases coolown of your active ability by 1";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {

player.GetActiveAbility().ChangeCooldown(player.GetActiveAbility().GetCooldown()-1)
;
            if (player.GetActiveAbility().GetCurrentDuration() >
player.GetActiveAbility().GetDuration())
            {

player.GetActiveAbility().ChangeCurrentDuration(player.GetActiveAbility().GetDurati
on());
            }
        }
    }
    public class PassiveAbilityCooldownItem : ConsumableItem
    {
        public PassiveAbilityCooldownItem(int numUsesIn) : base(numUsesIn)
        {
```

```
            itemName = "Passive ability cooldown item";
            itemDesc = "Decreases coolown of your passive ability by 1";
        }
        public override void DoItemEffect(Player player, Dungeon dungeon)
        {

player.GetPassiveAbility().ChangeCooldown(player.GetPassiveAbility().GetCooldown()-
1);
            if (player.GetPassiveAbility().GetCurrentDuration() >
player.GetPassiveAbility().GetDuration())
```

```
            {

player.GetPassiveAbility().ChangeCurrentDuration(player.GetPassiveAbility().GetDura
tion());
            }
        }
    }
}
namespace goodbyeworld
{
    public class FullHeartItem : PassiveStatItem
    {
        protected int extraHearts;
        public FullHeartItem(int extraHeartsIn) : base()
        {
            extraHearts = extraHeartsIn;
            itemName = $"{extraHearts} Full Heart(s)";
            itemDesc = $"Gives you {extraHearts} extra full heart(s)";
        }
        public override void BoostPlayerStat(Player player, Entity[,] entityLayer)
        {
            player.ChangeMaxHP(player.GetMaxHP() + extraHearts);
            player.ChangeHP(player.GetHP() + extraHearts, entityLayer);
        }



    }
    public class DMGBoostItem : PassiveStatItem
```

```
    {
        protected int dmgBoost;
        public DMGBoostItem(int dmgIn) : base()
        {
            dmgBoost = dmgIn;
            itemName = "Damage boost";
            itemDesc = $"Gives you {dmgBoost} extra damage";
        }
        public override void BoostPlayerStat(Player player, Entity[,] entityLayer)
        {
            player.ChangeDMG(player.GetDMG() + dmgBoost);
        }
    }
}
namespace goodbyeworld{
    public struct Coord(int xin, int yin)
```

```
    {
        public int x = xin;
        public int y = yin;
    }
    public class PriorityQueue:Queue
    {
        private int[,] distanceMap;
        public PriorityQueue(int size) : base(size)
        {
        }
        public void Enqueue(Coord coordIn, int[,] distanceMapIn)
        {
            distanceMap=distanceMapIn;
            bool lessThanEqualTo = false;
            int index = front;
            int coordInWeight = distanceMap[coordIn.y,coordIn.x];
            int coordWeight;
            Coord coord;
            Coord temp;
            if (!IsFull())
            {
                rear++;
```

```
            while (!lessThanEqualTo&&index<rear) //while the input coord weight
is not less than or equal to the current coord weight
            {
                coord = queue[index];
                coordWeight = distanceMap[coord.y,coord.x];
                if (coordWeight > coordInWeight) //if the input coord weight is
less than or equal to the current coord weight
                {
                    lessThanEqualTo=true;
                    coord=queue[index];
                    queue[index]=coordIn;
                    for(int i = index+1; i < rear; i++)
                    {
                        temp=queue[i];
                        queue[i]=coord;
                        coord=temp;
                    }
                    queue[rear]=coord;
                }
                else
                {
                    index++;
```

```
                }
            }
            if (!lessThanEqualTo)
            {
                queue[rear]=coordIn;
            }
        }
        else
        {
            throw new Exception("Attempted to enqueue to a full queue");
        }
    }
}
public class Queue
{
    protected Coord[] queue;
```

```
        protected int front;
        protected int rear;
        public Queue(int size)
        {
            queue = new Coord[size];
            front = 0;
            rear = -1;
        }
        public void Enqueue(Coord coord)
        {
            if (!IsFull())
            {
                rear++;
                queue[rear] = coord;
            }
        }
        public Coord Dequeue()
        {
            if (!IsEmpty())
            {
                front++;
                return queue[front - 1];
            }
            return new Coord(-1, -1);
        }
        public bool IsEmpty()
        {
            if (rear < front)
```

```
            {
                return true;
            }
            return false;
        }
        public bool IsFull()
        {
            if (rear >= queue.Length - 1)
            {
                return true;
```

```
            }
            return false;
        }
    }
}
using System.Numerics;
using System.Runtime.CompilerServices;

namespace goodbyeworld
{
    public abstract class Weapon:Item{
        protected int dmg;
        public Weapon() { }
        public Weapon(int dmgIn)
        {
            dmg = dmgIn;
            itemName = "Weapon";
        }
        public bool IsValid(int height, int width, int x, int y)
        {
            if (x < 0 || x > width - 1 || y < 0 || y > height - 1)
            {
                return false;
            }
            return true;
        }
        public abstract void AttackInDirection(Dungeon dungeon, Player player, int
x, int y, int xInc, int yInc);
        public abstract void UseWeapon(Dungeon dungeon, Player player, ConsoleKey
input);
        public int GetDMG()
        {
            return dmg;
        }
```

```
    }
    public class Sword : Weapon
    {
        public Sword() { }
```

```
        public Sword(int dmgIn)
        {
            dmg = dmgIn;
            itemName = "Sword";
        }
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("^",ConsoleColor.Cyan);
        }
        public override void UseWeapon(Dungeon dungeon, Player player, ConsoleKey
input)
        {
            if (input == ConsoleKey.W)
            {
                AttackInDirection(dungeon, player, player.x, player.y, 0, -1);
            }
            else if (input == ConsoleKey.A)
            {
                AttackInDirection(dungeon, player, player.x, player.y, -1, 0);
            }
            else if (input == ConsoleKey.S)
            {
                AttackInDirection(dungeon, player, player.x, player.y, 0, 1);
            }
            else if (input == ConsoleKey.D)
            {

                AttackInDirection(dungeon, player, player.x, player.y, 1, 0);
            }
        }
        public override void AttackInDirection(Dungeon dungeon, Player player, int
x, int y, int xInc, int yInc)
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();

            if (IsValid(height, width, x + xInc, y + yInc))
```

```
            {
                if (entityLayer[y + yInc, x + xInc] != null && entityLayer[y + yInc,
x + xInc].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y + yInc, x + xInc];
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y+yInc,x+xInc]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);



                }

            }

        }
    }
    public class Longsword : Sword
    {
        public Longsword() { }
        public Longsword(int dmgIn):base(dmgIn)
        {
            itemName = "Longsword";
        }
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("|",ConsoleColor.Cyan);
        }
        public override void AttackInDirection(Dungeon dungeon, Player player, int
x, int y, int xInc, int yInc)
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (IsValid(height, width, x + xInc, y + yInc))
```

```
            {
                if (entityLayer[y + yInc, x + xInc] != null && entityLayer[y + yInc,
x + xInc].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y + yInc, x + xInc];
```

```
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y+yInc,x+xInc]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);

                }
                else if (entityLayer[y + yInc, x + xInc] == null && tileLayer[y +
yInc, x + xInc] != null && !tileLayer[y + yInc, x + xInc].DoesCollide())
                {


                    if (IsValid(height, width, x + (xInc * 2), y + (yInc * 2)))
                    {



                        if (entityLayer[y + (yInc * 2), x + (xInc * 2)] != null &&
entityLayer[y + (yInc * 2), x + (xInc * 2)].GetType().IsSubclassOf(typeof(Enemy)))
                        {

                            Enemy enemy = (Enemy)entityLayer[y + (yInc * 2), x +
(xInc * 2)];
                            enemy.EnemyAttacked(player.GetDMG()+dmg);
                            effectsLayer[y + (yInc * 2), x + (xInc * 2)]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                        }
                    }
                }

            }

        }
```

```
    }
    public class Broadsword : Sword
    {
        public Broadsword(int dmgIn) : base(dmgIn)
        {
            itemName = "Broadsword";
        }
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("/",ConsoleColor.Cyan);
        }
        public override void AttackInDirection(Dungeon dungeon, Player player, int
x, int y, int xInc, int yInc)
```

```
        {
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (IsValid(height, width, x + xInc, y + yInc))

            {
                if (entityLayer[y + yInc, x + xInc] != null && entityLayer[y + yInc,
x + xInc].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y + yInc, x + xInc];
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y+yInc,x+xInc]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                }
                if (xInc == 0)
                {
                    if (IsValid(height, width, x - 1, y + yInc))
                    {
                        if (entityLayer[y + yInc, x - 1] != null && entityLayer[y +
yInc, x - 1].GetType().IsSubclassOf(typeof(Enemy)))
                        {
```

```
                        Enemy enemy = (Enemy)entityLayer[y + yInc, x - 1];
                        enemy.EnemyAttacked(player.GetDMG()+dmg);
                        effectsLayer[y+yInc,x-1]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                    }
                }
                if (IsValid(height, width, x + 1, y + yInc))
                {
                    if (entityLayer[y + yInc, x + 1] != null && entityLayer[y +
yInc, x + 1].GetType().IsSubclassOf(typeof(Enemy)))
                    {

                        Enemy enemy = (Enemy)entityLayer[y + yInc, x + 1];
                        enemy.EnemyAttacked(player.GetDMG()+dmg);
                        effectsLayer[y+yInc,x+1]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                    }
                }
            }
```

```
            else if (yInc == 0)
            {
                if (IsValid(height, width, x + xInc, y - 1))
                {
                    if (entityLayer[y - 1, x + xInc] != null && entityLayer[y -
1, x + xInc].GetType().IsSubclassOf(typeof(Enemy)))
                    {

                        Enemy enemy = (Enemy)entityLayer[y - 1, x + xInc];
                        enemy.EnemyAttacked(player.GetDMG()+dmg);
                        effectsLayer[y-1,x+xInc]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                    }
                }
                if (IsValid(height, width, x + xInc, y +1))
                {
                    if (entityLayer[y +1, x + xInc] != null && entityLayer[y +1,
x + xInc].GetType().IsSubclassOf(typeof(Enemy)))
                    {
```

```
                        Enemy enemy = (Enemy)entityLayer[y +1, x + xInc];
                        enemy.EnemyAttacked(player.GetDMG()+dmg);
                        effectsLayer[y+1,x+xInc]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                    }
                }
            }
        }

    }
    public class Mace : Sword
    {
        public Mace(int dmgIn) : base(dmgIn)
        {
            itemName = "Mace";
        }
        public override DisplayCharacter GameDisplay()
        {
            return new DisplayCharacter("9",ConsoleColor.Cyan);
        }
        public override void AttackInDirection(Dungeon dungeon, Player player, int
x, int y, int xInc, int yInc)
        {
```

```
            DisplayCharacter[,] effectsLayer = dungeon.GetEffectsLayer();
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (IsValid(height, width, x + 1, y))

            {
                if (entityLayer[y, x + 1] != null && entityLayer[y, x +
1].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y, x + 1];
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
```

```
                    effectsLayer[y,x+1]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                }
            }
            if (IsValid(height, width, x - 1, y))

            {
                if (entityLayer[y, x - 1] != null && entityLayer[y, x -
1].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y, x - 1];
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y,x-1]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                }
            }
            if (IsValid(height, width, x, y + 1))

            {
                if (entityLayer[y + 1, x] != null && entityLayer[y + 1,
x].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y + 1, x];
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y+1,x]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                }
            }
            if (IsValid(height, width, x, y-1))
```

```
            {
                if (entityLayer[y-1, x] != null && entityLayer[y-1,
x].GetType().IsSubclassOf(typeof(Enemy)))
                {

                    Enemy enemy = (Enemy)entityLayer[y-1, x];
```

```
                    enemy.EnemyAttacked(player.GetDMG()+dmg);
                    effectsLayer[y-1,x]=new
DisplayCharacter("*",ConsoleColor.DarkBlue);
                }
            }

        }
    }

}

namespace goodbyeworld
{



    public abstract class Ability
    {
        protected int cooldown;
        protected int currentCooldown;
        protected string abilityName;
        protected string abilityDesc;
        protected int duration;
        protected int currentDuration;
        protected int cooldownCap = 0;
        protected int durationCap = 100;

        public Ability()
        {
            abilityName = "NONE";
            abilityDesc = "NONE";
        }
        public Ability(int cooldownIn, int durationIn)
        {
            abilityName = "NONE";
            abilityDesc = "NONE";
            cooldown = cooldownIn;
            currentCooldown = cooldownIn;
            duration = durationIn+1;
            currentDuration = 0;
```

```
        }
        public void Display()
        {
            System.Console.WriteLine($"{abilityName} ABILITY COOLDOWN:
{currentCooldown}/{cooldown}, DURATION: {currentDuration}");
        }

        public bool CanUse()
        {

            if (currentCooldown <= 0)
            {
                return true;
            }
            return false;
        }
        public abstract void UseAbility(Dungeon dungeon, Player player);
        public abstract void DecreaseCooldown(Dungeon dungeon, Player player);
        public void ChangeCooldown(int amount)
        {
            if (amount <= cooldownCap)
            {
                cooldown=cooldownCap;
            }
            else
            {
                cooldown=amount;
            }
        }
        public void ChangeDuration(int amount)
        {
            if (amount <= 0)
            {
                duration=0;
            }
            else if(amount>=durationCap)
            {
                duration=durationCap;
```

```
            }
            else
            {
```

```
                duration=amount;
            }
        }
        public void ChangeCurrentDuration(int currentDurationIn)
        {
            currentDuration = currentDurationIn;
        }
        public int GetDuration()
        {
            return duration;
        }
        public int GetCurrentDuration()
        {
            return currentDuration;
        }
        public int GetCooldown()
        {
            return cooldown;
        }


    }
    public abstract class ActiveAbility : Ability
    {

        public ActiveAbility() : base()
        {
        }
        public ActiveAbility(int cooldownIn, int durationIn) :
base(cooldownIn,durationIn)
        {
        }
        public override void DecreaseCooldown(Dungeon dungeon,Player player)
        {
            if (currentCooldown > 0)
            {
```

```
                if (currentDuration > 0)
                {
                    UseAbility(dungeon, player);
                }
                else
                {
                    currentCooldown -= 1;
                }
            }
```

```
            else
            {
                if (currentDuration > 0)
                {
                    UseAbility(dungeon, player);
                }
            }


        }




    }
    public abstract class PassiveAbility : Ability
    {
        public PassiveAbility():base()
        {
        }
        public PassiveAbility(int cooldownIn,int
durationIn):base(cooldownIn,durationIn)
        {
        }
        public override void DecreaseCooldown(Dungeon dungeon,Player player)
        {
            if (currentCooldown > 0)
            {
                if (currentDuration > 0)
                {
                    UseAbility(dungeon, player);
```

```
                }
                else
                {
                    currentCooldown--;
                }
            }
            else
            {
                currentDuration = duration;
                UseAbility(dungeon,player);
            }
        }

    }
```

```
}
using System.Security.Authentication;

namespace goodbyeworld
{
    public class ActiveTemporaryDMGBoost:ActiveAbility
    {
        protected int dmgBoost;
        public ActiveTemporaryDMGBoost(int cooldownIn, int dmgBoostIn, int
durationIn):base(cooldownIn,durationIn)
        {
            dmgBoost = dmgBoostIn;
            abilityName = "Active Temporary Damage Boost";
            abilityDesc = $"Boosts damage by {dmgBoost} for {cooldownIn} turns";
        }
        public override void UseAbility(Dungeon dungeon,Player player)
        {

            if (currentDuration == duration)
            {
                player.ChangeDMG(player.GetDMG() + dmgBoost);
                currentDuration--;
                if (currentDuration == 0)
```

```
            {
                player.ChangeDMG(player.GetDMG() - dmgBoost);
            }
        }
        else if(currentDuration>0)
        {
            currentDuration--;
            if (currentDuration == 0)
            {
                player.ChangeDMG(player.GetDMG() - dmgBoost);
            }
        }
        currentCooldown = cooldown;
    }
}
public class ActiveHeal : ActiveAbility
{
    protected int healAmount;
    public ActiveHeal(int cooldownIn, int healAmountIn, int
durationIn):base(cooldownIn,durationIn)
    {
```

```
        healAmount = healAmountIn;
        abilityName = "Active Heal";
        abilityDesc = $"Heals {healAmount} HP";
    }
    public override void UseAbility(Dungeon dungeon,Player player)
    {

        Entity[,] entityLayer = dungeon.GetEntityLayer();
        if(currentDuration>0)
        {player.ChangeHP(player.GetHP() + healAmount, entityLayer);
            currentCooldown = cooldown;
            currentDuration--;
        }
    }
}
public class ActiveAOEAttack : ActiveAbility
{
```

```
        protected int dmg;
        public ActiveAOEAttack(int cooldownIn, int dmgIn, int durationIn) :
base(cooldownIn,durationIn)
        {
            dmg = dmgIn;
            abilityName = "Active AOE Attack";
            abilityDesc = "Attacks all of the spaces surrounding the player";
        }
        public override void UseAbility(Dungeon dungeon,Player player)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();

                if (IsValid(dungeon,player.x, player.y - 1))
                {
                    Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x];
                    enemy.EnemyAttacked(dmg);
                }
                if (IsValid(dungeon,player.x, player.y + 1))
                {
                    Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x];
                    enemy.EnemyAttacked(dmg);
                }
                if (IsValid(dungeon,player.x-1, player.y))
                {
```

```
                    Enemy enemy = (Enemy)entityLayer[player.y, player.x-1];
                    enemy.EnemyAttacked(dmg);
                }
                if (IsValid(dungeon,player.x+1, player.y))
                {
                    Enemy enemy = (Enemy)entityLayer[player.y, player.x+1];
                    enemy.EnemyAttacked(dmg);
                }
                if (IsValid(dungeon,player.x-1, player.y - 1))
                {
                    Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x-1];
```

```
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x+1, player.y - 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x+1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x-1, player.y + 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x-1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x+1, player.y + 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x+1];
                enemy.EnemyAttacked(dmg);
            }
            Environment.Exit(0);
            currentCooldown = cooldown;
            currentDuration--;

    }
    public bool IsValid(Dungeon dungeon,int x, int y)
    {
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        int height = dungeon.GetDunHeight();
        int width = dungeon.GetDunWidth();
        if (x < 0 || x > width - 1 || y < 0 || y > height - 1)
        {
            return false;
        }
        else if (tileLayer[y, x] == null)
```

```
        {
            return false;
        }
        else if (tileLayer[y, x].DoesCollide())
        {
```

```
                return false;
            }
            else if (entityLayer[y, x] == null)
            {
                return false;
            }
            if (entityLayer[y, x].GetType().IsSubclassOf(typeof(Enemy)))
            {
                return true;
            }
            return false;
        }
    }
    public class ActiveDamageTileTrail : ActiveAbility
    {
        protected int dmg;
        public ActiveDamageTileTrail(int cooldownIn, int dmgIn,int durationIn) :
base(cooldownIn,durationIn)
        {
            dmg=dmgIn;
            abilityName = "Active Damage Tile Trail";
            abilityDesc = "Place damage tiles, and have immunity to your own damage
tiles";
        }
        public override void UseAbility(Dungeon dungeon, Player player)
        {
            Tile[,] tileLayer = dungeon.GetTileLayer();
            tileLayer[player.y,player.x]=new DamageTile(dmg,false);
            currentDuration--;
            currentCooldown=cooldown;
        }
    }
}
namespace goodbyeworld
{
    public class PassiveHeal : PassiveAbility
    {
        protected int healAmount;
```

```
        public PassiveHeal(int cooldownIn, int healAmountIn, int durationIn) :
base(cooldownIn, durationIn)
        {
            healAmount = healAmountIn;
            abilityName = "Passive Heal";
            abilityDesc = $"Heals {healAmount} HP every {cooldown} moves";
        }
        public override void UseAbility(Dungeon dungeon, Player player)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            player.ChangeHP(player.GetHP() + healAmount, entityLayer);
            currentCooldown = cooldown;
            currentDuration--;
        }
        public void ChangeHeal(int healAmountIn)
        {
            healAmount = healAmountIn;
        }
        public int GetHealAmount()
        {
            return healAmount;
        }
    }
    public class PassiveTemporaryDMGBoost:PassiveAbility
    {
        protected int dmgBoost;
        public PassiveTemporaryDMGBoost(int cooldownIn, int dmgBoostIn, int
durationIn):base(cooldownIn,durationIn)
        {
            dmgBoost = dmgBoostIn;
            abilityName = "Passive Temporary Damage Boost";
            abilityDesc = $"Boosts damage by {dmgBoost} for {cooldownIn} turns";
        }
        public override void UseAbility(Dungeon dungeon,Player player)
        {

            if (currentDuration == duration)
            {
                player.ChangeDMG(player.GetDMG() + dmgBoost);
```

```
                currentDuration--;
                if (currentDuration == 0)
                {
                    player.ChangeDMG(player.GetDMG() - dmgBoost);
                }
```

```
            }
            else if(currentDuration>0)
            {
                currentDuration--;
                if (currentDuration == 0)
                {
                    player.ChangeDMG(player.GetDMG() - dmgBoost);
                }
            }
            currentCooldown = cooldown;
        }
    }
    public class PassiveAOEAttack : PassiveAbility
    {
        protected int dmg;
        public PassiveAOEAttack(int cooldownIn, int dmgIn,int durationIn) :
base(cooldownIn,durationIn)
        {
            dmg = dmgIn;
            abilityName = "Passive AOE Attack";
            abilityDesc = "Attacks all of the spaces surrounding the player";
        }
        public override void UseAbility(Dungeon dungeon,Player player)
        {
            Entity[,] entityLayer = dungeon.GetEntityLayer();
            Tile[,] tileLayer = dungeon.GetTileLayer();
            int height = dungeon.GetDunHeight();
            int width = dungeon.GetDunWidth();
            if (IsValid(dungeon,player.x, player.y - 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x];
                enemy.EnemyAttacked(dmg);
            }
```

```
            if (IsValid(dungeon,player.x, player.y + 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x-1, player.y))
            {
                Enemy enemy = (Enemy)entityLayer[player.y, player.x-1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x+1, player.y))
```

```
            {
                Enemy enemy = (Enemy)entityLayer[player.y, player.x+1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x-1, player.y - 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x-1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x+1, player.y - 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y - 1, player.x+1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon,player.x-1, player.y + 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x-1];
                enemy.EnemyAttacked(dmg);
            }
            if (IsValid(dungeon, player.x + 1, player.y + 1))
            {
                Enemy enemy = (Enemy)entityLayer[player.y + 1, player.x + 1];
                enemy.EnemyAttacked(dmg);
            }
            currentDuration--;
            currentCooldown = cooldown;
        }
```

```
            public bool IsValid(Dungeon dungeon,int x, int y)
    {
        Entity[,] entityLayer = dungeon.GetEntityLayer();
        Tile[,] tileLayer = dungeon.GetTileLayer();
        int height = dungeon.GetDunHeight();
        int width = dungeon.GetDunWidth();
        if (x < 0 || x > width - 1 || y < 0 || y > height - 1)
        {
            return false;
        }
        else if (tileLayer[y, x] == null)
        {
            return false;
        }
        else if (tileLayer[y, x].DoesCollide())
        {
            return false;
        }
        else if (entityLayer[y, x] == null)
        {
            return false;
        }
        if (entityLayer[y, x].GetType().IsSubclassOf(typeof(Enemy)))
        {
            return true;
        }
        return false;
    }
    public void ChangeDMG(int dmgIn)
    {
        dmg = dmgIn;
    }
    public int GetDMG()
    {
        return dmg;
    }
}
public class PassiveDamageTileTrail : PassiveAbility
{
```

```
        protected int dmg;
        public PassiveDamageTileTrail(int cooldownIn, int dmgIn,int durationIn) :
base(cooldownIn,durationIn)
        {
            dmg=dmgIn;
            abilityName = "Passive Damage Tile Trail";
            abilityDesc = "Place damage tiles passively, and have immunity to your
own damage tiles";
        }
        public override void UseAbility(Dungeon dungeon, Player player)
        {
            Tile[,] tileLayer = dungeon.GetTileLayer();
            tileLayer[player.y,player.x]=new DamageTile(dmg,false);
            currentDuration--;
            currentCooldown=cooldown;
        }
    }
}
```